

Seating & Event Availability Tracking System (SEATS)

Project Description, ADRs, Microservice Design and API Specification

2110521 วิศวกรรมสถาปัตยกรรมซอฟต์แวร์ (2110521 Software Architecture)

named Concert Table Reservation System (CTRS) in Deliverables #1 and #2

Group Members

SE 101

Chetsada Winaikoson	Student ID 6772012221
Waris Natirutthakorn	Student ID 6872081621
Natcha Thawinpipat	Student ID 6870083521
Watayut Aiamanan	Student ID 6970267821

Version 2.0 · 29 September 2026 · draft for review by the group

This document is an integral component of the subject 2110521 Software Architecture, Department of Computer Engineering, Faculty of Engineering, Chulalongkorn University, Semester 1 of the academic year 2026.

Contents

0 Document Control 4
0.1 Document Information 4
0.2 Revision History 5

1 Project Description 6
1.1 Problem Description 6
1.2 Target Customer 6
1.3 Scope and Increments 6

2 Scenario (use-case & description) 8
2.1 Use Case Diagram 8
2.2 Use Case Descriptions 10
2.2.1 UC-01 Reserve a Specific Table 10
2.2.2 UC-02 Check In with E-Ticket 14
2.2.3 UC-03 Create Concert Round 17
2.2.4 UC-04 Create Venue Zone Map 20
2.2.5 UC-09 Maintain Customer Profile 23
2.2.6 UC-10 Pay the Full Table Fee 24

3 Requirements 27
3.1 Functional Requirements 27
3.1.1 User Authentication & Login System 27
3.1.2 Venue Zone Map & Concert Round Management System 27
3.1.3 Table Reservation System 28
3.1.4 Payment 29
3.1.5 E-Ticket & Check-In System 30
3.1.6 Notification System 31
3.1.7 Out of Scope 31
3.2 Non-functional Requirements 31
3.2.1 Operational 31
3.2.2 Performance 32
3.2.3 Reliability 32
3.2.4 Security 32
3.2.5 Cultural and Legal 33
3.2.6 Usability 33
3.2.7 Interfaces 34

4 Architecture Decision Records (ADRs) 35
4.1 ADR-01: Frontend Architecture & Client Channel 36
4.2 ADR-02: Real-Time Floor Plan Communication 37
4.3 ADR-03: Backend Language & Framework 38
4.4 ADR-04: Notification Engine 39
4.5 ADR-05: Interactive Floor Plan Rendering 40
4.6 ADR-06: Primary Database Engine 41

4.7 ADR-07: Internal Role Authentication (Front Staff, Manager) 42
4.8 ADR-08: Table Hold and Concurrency Control 43
4.9 ADR-09: Table Status Updates in the MVP: Polling 44
4.10 ADR-10: Customer Notifications through the LINE Messaging API Only 45
4.11 ADR-11: Simulated Payment Gateway for the MVP 46
4.12 ADR-12: Communication: REST at the API Gateway, gRPC inside the Backend 47
4.13 ADR-13: The Booking Owns the Hold; the Table Map Is a Read Model 49
4.14 ADR-14: Modular Monolith Mode for Development and Tests 50

5 Microservice Design 51
5.1 Scope of the Microservice Design 51
5.2 Parts of the System and Communication 52
5.3 Service–Operations–Collaborators 55
5.4 Architecture Diagram 58

6 Domain Model and API Specification 59
6.1 Domain Model 59
6.2 From Model to Contract 63
6.3 Concert Round Service 64
6.4 Table Availability Service 67
6.5 Booking Service 69
6.6 Payment, Notification and Staff Account Services 71
6.7 API Gateway 72

Appendix A Use Case Traceability 77
A.1 Operations by Use Case 77
A.2 Step by Step 80
Appendix B Business Rules 88
Appendix C Glossary 90
Appendix D Screens and the Routes They Call 94
D.1 Customer Web App 94
D.2 Back-office Web App 103
D.3 Coverage 113

0 Document Control

0.1 Document Information

Table 0.1 Document information

Document	Project Description, ADRs, Microservice Design and API Specification of the Seating & Event Availability Tracking System (SEATS)
Product name	SEATS; Deliverables #1 and #2 called the system Concert Table Reservation System (CTRS)
Course	2110521 Software Architecture, Semester 1, Academic Year 2026
Group	SE 101
Version	2.0, 30 September 2026
Status	Version 2.0, the updated ADRs and microservice design of Deliverable #3 (progress 1, 3 October 2026); reviewed by the group before submission
Previous version	1.1: Deliverable #1 (version 1.0) and Deliverable #2 as submitted on 8 September 2026
Changes	The change log is a separate document of the same version, "SEATS Project Document — Change Log": every change since version 1.1 (CH-nn), how each teacher comment and known issue was resolved, and how to compare the versions

0.2 Revision History

Table 0.2 Revision history

Version	Date	By	Description	Source
1.0	2026-09-08	SE 101	Deliverable #1 submitted: project description, four use cases, requirements, ADR-01 to ADR-07.	—
1.1	2026-09-08	SE 101	Deliverable #2 submitted: microservice design version 1 (Service–Operations–Collaborators table, architecture diagram).	—
2.0	2026-09-30	SE 101	Deliverable #3: the document revised after the teacher's feedback on Deliverables #1 and #2 and the known issues. Scope in increments with an MVP; the product named SEATS; the use cases as tables with the domain model behind them; the requirements completed; ADR-06 corrected, ADR-08 to ADR-14 added; microservice design version 2 with REST at the API Gateway and gRPC inside, the parts of the system, the operations of the MVP and their traceability; Chapter 6 with the domain model, the data model and the API of each service and the gateway; the appendices with the traceability, the business rules, the glossary and the screens. Changes CH-01 to CH-68, listed in the change-log document; the drafts 1 to 35 of this version are the git tags doc-v2.0-draft1 to doc-v2.0-draft35.	Teacher feedback on Deliverables #1 and #2, KI-01 to KI-18, the group's review

1 Project Description

1.1 Problem Description

Pubs and bars still rely on manual chat messaging and phone calls to handle table reservations, leading to delayed staff responses, lost bookings, lack of real-time **zone map** visibility, and frequent allocation errors where customers do not get their requested tables. Currently, handling table inquiries and confirming seat statuses manually consumes substantial staff time and often leads to miscommunications during peak hours. The system should streamline table reservations through a web app accessed via LINE Messenger, providing a real-time interactive **zone map**, payment of the **full table fee** when booking, **e-tickets** for check-in, and one clear arrival rule: check-in opens 2 hours before the show, the table is kept until 30 minutes after the start, and a booking not checked in by then becomes a **no-show**.

1.2 Target Customer

- **Manager (venue manager):** Full access to the **back-office**: create the venue **zone map** (**zones**, tables, and **table types**), create and publish **concert rounds** with their prices, follow bookings and table occupancy on the **live view**, and rule on check-in **escalations**. The venue owner uses the same **back-office** with read-only access.
- **Front Staff (front of house):** Scan the QR codes of **e-tickets** for fast check-in, monitor real-time table statuses, and seat **walk-in** guests at the tables of **no-shows**, paid by hand at the venue.
- **Customer:** Browse interactive **zone maps**, reserve and pay for a specific table, receive the **e-ticket** by LINE, and check in via digital QR pass.

1.3 Scope and Increments

The system is built in increments, so that a first working version fits a build of two to three months. The use case descriptions in Section 2 still describe the complete behaviour; every flow that the MVP does not build is marked *(Increment 2)*, and the requirements and ADRs carry the same marks. The microservice design of Section 5 shows the MVP only.

Increment	What is built	Use cases and flows
MVP	Create a venue **zone map** on an uploaded image of the venue: **zones**, tables, **table types** and capacities; validate and activate it.	UC-04 basic flow, AF-1, AF-3, EF-1, EF-2, EF-3

Increment	What is built	Use cases and flows
MVP	Create, validate and publish a **concert round** with its prices and **booking-open time**, or save it as a draft.	UC-03 basic flow, AF-1, AF-3, EF-1, EF-2
MVP	Reserve a table: LINE Login, rounds, **zone map** refreshed within 2 seconds, 15-minute **hold** (**first lock wins**) with automatic expiry, **party size** and fee, profile and consent (UC-09), **booking terms**, **simulated payment** with success or decline (UC-10), confirmation, QR **e-ticket** and LINE confirmation message.	UC-01 basic flow, AF-1 to AF-5, EF-1 to EF-3; UC-09 basic flow, AF-1, AF-2, EF-1; UC-10 basic flow, AF-1, EF-4
MVP	Check in with the QR code or the typed **booking reference**, verified against the **check-in window** and the **grace period**; used, unknown and out-of-window tickets are refused.	UC-02 basic flow, AF-2 to AF-5, EF-1, EF-2, EF-5
Increment 2	Real payment gateway (Beam Checkout in its sandbox, the gateway selected for the system) with status polling, automatic refund of a late payment, and **degraded mode** with transfer-slip review by the **Manager**.	UC-10 EF-1, EF-2, EF-3
Increment 2	Table status pushed to open maps by WebSocket instead of polling.	ADR-09
Increment 2	Escalation of invalid tickets to the **Manager**, look-up by name or phone, automatic **no-show** marking.	UC-02 AF-1, EF-3, EF-4
Increment 2	Copy a round or a **zone map**; withdraw a published round; handle a **zone map** changed during editing.	UC-03 AF-2, AF-4, EF-3; UC-04 AF-2
Out of scope	Planned for a later release: reminders and the "on my way" grace extension, **waitlist**, booking transfer, reports and analytics, and seat-view photographs of the tables.	—

The product is named Seating & Event Availability Tracking System (SEATS); Deliverables #1 and #2 called it Concert Table Reservation System (CTRS). Requirements are identified as FR-nn (functional) and NFR-nn (non-functional), and business rules as BRULE-nn (Appendix B).

2 Scenario (use-case & description)

2.1 Use Case Diagram

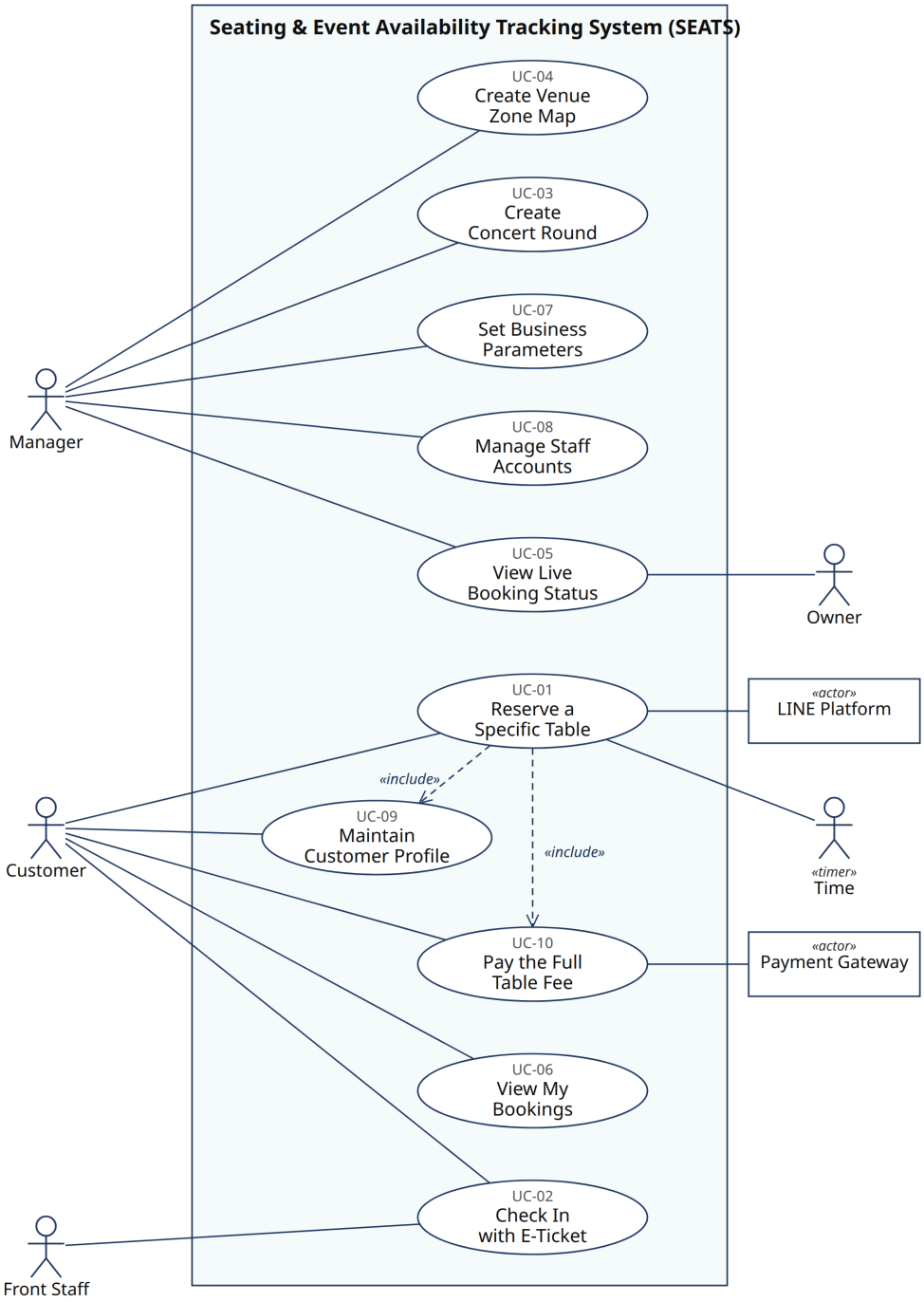


Figure 2.1 Use case diagram of the Seating & Event Availability Tracking System (SEATS)

The diagram names each use case as its description does and shows every actor that takes part in it: the **Customer**, the **Front Staff**, the **Manager** and the **Owner**, and the secondary actors **LINE Platform** (LINE Login

and the Messaging API), **Payment Gateway** and Time. UC-01 to UC-04, UC-09 and UC-10 are the business use cases and are described in Section 2.2; UC-05 to UC-08 are the reads and the back-office settings that the same actors need, each one operation or a few reads of Section 5, and they are listed by name only. UC-01 «includes» UC-09 Maintain Customer Profile and UC-10 Pay the Full Table Fee: both always run inside a reservation, and UC-09 also stands alone from My Bookings. The Increment 2 parts of the **Manager** in UC-10 (transfer-slip review) and UC-02 (**escalation**) and of Time in UC-02 (**no-show** marking) are listed in Table 2.1 and not drawn, to keep the diagram readable. There is no «extend» relationship, and UC-04, UC-03, UC-01 and UC-02 follow one another only because each produces what the next one needs (**zone map**, **concert round**, confirmed booking), which is an order of use, not a relationship between use cases.

Table 2.1 Use cases, actors and increments

Use case	Primary actor	Secondary actors	Increment
UC-01 Reserve a Specific Table	**Customer**	**LINE Platform**, Time (hold expiry); includes UC-09 and UC-10	MVP
UC-02 Check In with E-Ticket	**Front Staff**	**Customer**, **Manager** (**escalation**, Increment 2), Time (**no-show** marking, Increment 2)	MVP
UC-03 Create Concert Round	**Manager**	—	MVP
UC-04 Create Venue Zone Map	**Manager**	—	MVP
UC-05 View Live Booking Status	**Manager**	**Owner** (reads the same view)	MVP
UC-06 View My Bookings	**Customer**	—	MVP
UC-07 Set Business Parameters	**Manager**	—	MVP
UC-08 Manage Staff Accounts	**Manager**	**Front Staff**, **Owner** (sign in and log out with the accounts)	MVP
UC-09 Maintain Customer Profile	**Customer**	— (included by UC-01; also from My Bookings)	MVP
UC-10 Pay the Full Table Fee	**Customer**	**Payment Gateway**, **Manager** (transfer-slip review, Increment 2); included by UC-01	MVP

2.2 Use Case Descriptions

The six business use cases are described in a table after Dennis, Wixom and Tegarden: name, identifier and importance, actors and type, stakeholders, brief description, trigger, relationships, pre- and postconditions, then the basic flow with its phases in braces and its extension points in bold, the subflows, the alternative flows and the exception flows. Flows that the MVP does not build are marked *(Increment 2)*.

2.2.1 UC-01 Reserve a Specific Table

Table 2.2 Use-case description of Reserve a Specific Table

Use Case Name Reserve a Specific Table	**ID** UC-01	**Importance Level** High — MVP.
Primary Actor **Customer** Secondary actors: **LINE Platform** (LINE Login and the Messaging API) and Time (hold expiry). The **Payment Gateway** and, from Increment 2, the **Manager** take part through the included use case UC-10.		**Use-Case Type** Business / Transactional
Stakeholders and Interests • **Customer**: Wants to select a suitable table and receive reliable booking confirmation. • **Front Staff**: Wants accurate reservations with fewer manual inquiries and table allocation errors. • **Manager**: Wants efficient table utilization and visibility into bookings.		
Brief Description **Customer** accesses the web app through LINE Messenger, browses the upcoming **concert rounds**, views the real-time **zone map** and holds a specific table for 15 minutes. After completing the **customer profile** (UC-09), accepting the **booking terms** and paying the **full table fee** through the **Payment Gateway** (UC-10), the system confirms the booking, issues an **e-ticket** with a QR code and sends it to the **Customer** by LINE.		
Trigger **Customer** wants to reserve a table for a **concert round**. Trigger type: external.		
Relationships • Association: **Customer** (primary actor); **LINE Platform** and Time (secondary actors). • Include: UC-09 Maintain Customer Profile at **{Complete the Customer Profile}** • Include: UC-10 Pay the Full Table Fee at **{Pay the Full Table Fee}** • Extend: none. • Generalization: none. • Related use cases: Relies on LINE Login for customer authentication, the LINE Messaging API for confirmation messages, the **Payment Gateway** for payment (UC-10), and the **concert round** created in UC-03 on a **zone map** from UC-04; the **e-ticket** is presented in UC-02 and the bookings are listed under My Bookings (UC-06). • Business rules: BRULE-01, 02, 03, 07, 08, 09, 11, 12, 16, 17.		

Preconditions

- **Customer** has the LINE application and is a friend of the shop's **LINE Official Account**.
- **Manager** has created the **concert round** with its **zone map**, **table types**, **package prices** and **booking-open time** (UC-03).
- The **Payment Gateway** is available (UC-10): in the MVP the simulated gateway.

Postconditions

Success (basic flow; UC-10 EF-2 slip approved, Increment 2):

- Booking state = Confirmed; the payment is recorded; the table is booked for the round; the **e-ticket** is issued; the confirmation has been sent by LINE or is retrievable under My Bookings.

Cancelled (AF-4 cancel; AF-5 profile not completed; EF-2 login failed):

- Booking state = Cancelled (or no booking was created); the **hold** is released and the table is available again; no payment exists.

Expired (EF-1 hold expired; UC-10 EF-4 hold ended during payment; UC-10 EF-2 slip rejected or missing and UC-10 EF-3 no result, Increment 2):

- Booking state = Expired; the table is available again and the **zone map** is refreshed; no money is held from the **Customer**; the **Customer** has been informed on screen and by LINE.

Refunded (UC-10 EF-1 payment result after the hold expired, table no longer available; Increment 2):

- Booking state = Expired; the payment and the refund are both recorded; the table remains with the other customer; the **Customer** has been informed by LINE that the payment is being returned.

Basic Flow

{Open the Customer Frontend}

1. The use case begins when the **Customer** opens the Rich Menu of the **LINE Official Account** and chooses "Reserve a table".
2. System signs the **Customer** in through LINE Login and uses the LINE user id as the customer identity. **{LINE Login Result}**

{Browse Rounds}

3. System displays the upcoming **concert rounds** with artist, date, start time, **booking-open time** and status: not yet open, open, sold out.
4. **Customer** selects a round that is open for booking.

{View the Zone Map}

5. System displays the **zone map** of the round: every table with its number, **zone**, **table type**, **package price** and status (available, held, booked), refreshed within 2 seconds of any change (FR-06).
6. **Customer** selects an available table.

{Hold the Table}

7. System verifies that the table is still available and places a **hold** on it for the **hold period** of 15 minutes.
8. System marks the table as held for every other customer (**first lock wins**) and displays the remaining hold time.
9. System shows the booking summary (round, table, **package** content and **package price**) and asks for the **party size**.
10. **Customer** enters the **party size**.
11. System computes and displays the **full table fee**: the **package price** of the selected **table type** in its **zone** plus the **extra-person fee** for every person above the capacity of the **table type**.

{Complete the Customer Profile}

12. System includes UC-09 Maintain Customer Profile: on the first booking the **customer profile** is created with the **Customer**'s consent, otherwise it is shown and confirmed. **{Profile Complete}**

{Accept the Booking Terms}

13. System displays the **booking terms**: full payment confirms the booking, the **check-in window** opens 2 hours before the show, the **grace period** is 30 minutes after the start, and a **no-show** is not refunded.
14. **Customer** accepts the terms.

{Pay the Full Table Fee}

15. System includes UC-10 Pay the Full Table Fee: the **Customer** pays the **full table fee** through the **Payment Gateway** before the **hold** expires, and the payment result is verified and recorded. **{Payment Verified}**

{Confirm the Booking}

16. System sets the booking to Confirmed and marks the table as booked. **{Booking Confirmed}**

{Issue the E-Ticket}

17. System issues the **e-ticket**: it generates a signed **booking reference** valid only for that **concert round** and table, encodes it in a QR code, stores it with the booking, and makes it available under My Bookings (UC-06).
18. System displays the confirmation and the **e-ticket** on screen.
19. System sends the confirmation message with the **e-ticket** and the **booking terms** to the **Customer** through the LINE Messaging API (subflow S-1). **{Confirmation Delivery Result}**
20. The use case ends.

Subflows

S-1 Notify the Customer by LINE

1. System composes the message from the booking data and the **booking terms** template.
2. System sends it as a push message through the LINE Messaging API and records the delivery result.
3. The subflow returns to the step that invoked it.

Alternative Flows

AF-1 Round Not Yet Open for Booking

At **{Browse Rounds}**, if the **Customer** selects a round whose **booking-open time** has not arrived,

1. System shows the round's details and its **booking-open time** and does not display the **zone map**.
2. Resume the basic flow at **{Browse Rounds}**.

AF-2 Round Sold Out

At **{Browse Rounds}**, if the **Customer** selects a round whose status is sold out, or at **{View the Zone Map}**, if the last available table of the round is taken while the map is displayed,

1. System shows the round as sold out and closes the **zone map** if it is displayed.
2. Resume the basic flow at **{Browse Rounds}**.

AF-3 Table Just Taken by Another Customer

At **{Hold the Table}**, if the verification finds that the selected table has already been held or booked by another customer (**first lock wins**),

1. System informs the **Customer** that the table has just been taken and refreshes the **zone map**.
2. Resume the basic flow at **{View the Zone Map}**.

AF-4 Customer Cancels During the Hold

At any point between **{Hold the Table}** and **{Payment Verified}**, if the **Customer** cancels the booking,

1. System releases the **hold** immediately, sets the booking to Cancelled and returns the table to available.
2. The use case ends.

Leaving the web app without cancelling does not release the **hold**; it runs until it expires (EF-1).

AF-5 Profile Not Completed

At **{Profile Complete}**, if UC-09 ends without a **customer profile** because the **Customer** declined the consent to the data collection,

1. System explains that the booking cannot continue, releases the **hold** and sets the booking to Cancelled.
2. The use case ends.

Exception Flows

EF-1 Hold Expires Before Payment

At any point between **{Hold the Table}** and **{Payment Verified}**, if the **hold period** of 15 minutes ends without a verified payment (Time actor),

1. System releases the table, sets the booking to Expired and refreshes the **zone map** for every customer.

2. System informs the **Customer** by a LINE message that the **hold** has expired and that the table may be selected again if still available (subflow S-1).
3. The use case ends; a payment result that arrives afterwards is handled by UC-10 EF-1 (Increment 2).

EF-2 LINE Login Fails or Is Cancelled

At **{LINE Login Result}**, if LINE Login fails or the **Customer** cancels it,

1. System explains that reservation requires a LINE account and returns to the Rich Menu.
2. The use case ends.

EF-3 Confirmation Message Cannot Be Sent

At **{Confirmation Delivery Result}**, if the LINE Messaging API rejects the message or the account's push quota is exhausted,

1. System records the failure and retries three times over five minutes.
2. The use case ends normally; the booking is Confirmed and the **e-ticket** stays available under My Bookings.

2.2.2 UC-02 Check In with E-Ticket

Table 2.3 Use-case description of Check In with E-Ticket

Use Case Name	ID	Importance Level
Check In with E-Ticket	UC-02	High — MVP; AF-1, EF-3 and EF-4 in Increment 2.
Primary Actor **Front Staff** Secondary actors: **Customer** (presents the **e-ticket**) and, from Increment 2, the **Manager** (rules on escalated tickets) and Time (**no-show** marking).		**Use-Case Type** Business / Transactional
Stakeholders and Interests • **Customer**: Wants quick check-in and access to the correct reserved table. • **Front Staff**: Wants to verify reservations quickly and prevent duplicate check-ins. • **Manager**: Wants accurate attendance information and a **live view** of table occupancy and of **no-show** tables free for **walk-in** guests. • Time: Triggers the automatic **no-show** marking at the end of the **grace period**.		
Brief Description **Customer** arrives at the door and shows the **e-ticket** QR code on the phone. **Front Staff** scans the code with the **back-office** on a smartphone; System verifies the signed **booking reference** against the central record (Confirmed, tonight's round, not yet checked in, **check-in window** open) and displays the table, **package**, **party size** and payment status within 2 seconds. **Front Staff** checks the **party size**, confirms the entry, System sets the booking to Checked-in with time and staff account and updates the **live view** for the **Manager**. Invalid or duplicate tickets are escalated to the **Manager**; bookings not checked in by the end of the **grace period** are marked **no-show**, and their tables are shown as free for **walk-in** guests, sold by hand at the venue.		
Trigger **Customer** arrives at the door and shows the **e-ticket** QR code on the phone. Trigger type: external.		
Relationships • Association: **Front Staff** (primary actor); **Customer** and, from Increment 2, the **Manager** and Time (secondary actors). • Include: none. • Extend: none. • Generalization: none. • Related use cases: Uses the Confirmed booking and **e-ticket** issued in UC-01 and the venue-issued **back-office** account of the **Front Staff**. Updates the **live view** seen by the **Manager**. Escalations are ruled on by the **Manager**; a **no-show** frees its table for **walk-in** guests, paid by hand at the venue and never offered to a **waitlist** (BRULE-06). • Business rules: BRULE-04, 05, 06, 09.		
Preconditions • **Customer** holds a Confirmed booking for tonight's **concert round** and has the **e-ticket** on the phone (UC-01). • **Front Staff** is signed in to the **back-office** on a smartphone with a camera using a venue-issued username and password account. • The **check-in window** of the round is open: from 2 hours before the start until the end of the **grace period**.		
Postconditions **Success (basic flow; AF-1 to AF-4; EF-3 accepted):** • Booking state = Checked-in with time and staff account; the party is seated at its table; the shared **live view** shows the table as occupied. **Failure (AF-3 window not open; EF-1; EF-2; EF-3 refused; EF-5):** • Booking keeps its previous state; an **escalation** or a refusal is recorded.		

No-show (AF-5; EF-4):

- Booking state = **No-show**; the table is shown as free for **walk-in** guests; the **full table fee** is forfeited.

Basic Flow

{Present the E-Ticket}

1. The use case begins when the **Customer** arrives at the door and shows the **e-ticket** QR code on the phone.

{Scan and Verify}

2. **Front Staff** scans the QR code with the **back-office** on the phone. **{Booking Reference Obtained}**
3. System decodes the signed **booking reference** and verifies, against the central record, that the booking exists, is Confirmed, belongs to tonight's **concert round**, has not been checked in before, and that the **check-in window** is open (subflow S-1). **{Verification Result}**
4. System displays the verification result within 2 seconds: table number and **zone**, **table type** and **package**, **party size** paid for, the name on the **customer profile**, and payment status "paid in full".

{Confirm Entry}

5. **Front Staff** confirms the number of guests present against the **party size** paid for. **{Party Size Confirmed}**
6. **Front Staff** confirms the entry; System sets the booking to Checked-in, records the time and the staff account, and shows the table number to guide the party.
7. System updates the **live view** of the **back-office** so that the **Manager** sees the table as occupied.

{Use Case Ends}

8. The use case ends when the party has been shown to the table.

Subflows

S-1 Verify the Booking Reference

1. System checks the signature of the **booking reference**.
2. System loads the booking and checks its status, round, table and prior check-in.
3. System compares the current time with the round's **check-in window** and **grace period**.
4. The subflow returns to the step that invoked it.

Alternative Flows

AF-1 Customer Cannot Show the QR Code *(Increment 2)*

At **{Present the E-Ticket}**, if the **Customer** cannot show the **e-ticket** (no battery, no signal, deleted message),

1. **Front Staff** looks the booking up in the **back-office** by name, phone number or **booking reference**.
2. System shows the matching Confirmed bookings for tonight's round.
3. **Front Staff** selects the one whose name matches the customer's identification.
4. Resume the basic flow at **{Booking Reference Obtained}** with the selected booking.

AF-2 QR Code Unreadable

At **{Scan and Verify}**, if the camera cannot read the code (damaged screen, glare),

1. **Front Staff** types the **booking reference** printed under the QR code.
2. Resume the basic flow at **{Booking Reference Obtained}**.

AF-3 Check-In Window Not Open Yet

At **{Verification Result}**, if the current time is more than 2 hours before the start,

1. System shows the time at which the **check-in window** opens and does not change the booking.
2. **Front Staff** asks the **Customer** to return at that time. The use case ends.

AF-4 More Guests Than the Party Size Paid For

At **{Confirm Entry}**, if more guests are present than the **party size** paid for,

1. If the guests fit within the capacity of the **table type**, **Front Staff** lets them in; no fee is due.
2. If they exceed the capacity, the extra persons are handled by hand at the venue, outside the system (BRULE-09); System records nothing.
3. Resume the basic flow at **{Party Size Confirmed}**.

AF-5 Arrival After the Grace Period

At **{Verification Result}**, if the current time is more than 30 minutes after the concert start and the booking has not been checked in,

1. System shows that the booking is a **no-show**: the **grace period** has passed and the **full table fee** is not refunded.
2. If the table is still free, **Front Staff** may seat the party as **walk-in** guests on the **Manager**'s authority, paid by hand at the venue outside the system; the booking stays **No-show**. The use case ends.
3. Otherwise **Front Staff** informs the **Customer**. The use case ends.

Exception Flows

EF-1 Ticket Already Used

At **{Verification Result}**, if the booking is already Checked-in,

1. System shows "already checked in" with the time and the staff account of the earlier check-in.
2. If the party is joining a group already seated at the same table (latecomers of the same booking), **Front Staff** lets them through without a second check-in; if the guests exceed the capacity of the **table type**, AF-4 applies. The use case ends.
3. Otherwise **Front Staff** escalates the case to the **Manager** as a duplicate ticket (EF-3; in the MVP **Front Staff** refuses the entry and calls the **Manager** by hand).

EF-2 Ticket for Another Round or an Unknown Booking

At **{Verification Result}**, if the reference is not found, its signature is invalid, or the booking belongs to a different **concert round** or was Transferred or Cancelled,

1. System shows the reason (unknown ticket, wrong date, transferred, cancelled).
2. **Front Staff** escalates the case to the **Manager** (EF-3; in the MVP **Front Staff** refuses the entry and calls the **Manager** by hand).

EF-3 Escalation to the Manager *(Increment 2)*

At EF-1 or EF-2, when **Front Staff** escalates an invalid ticket or a duplicate ticket,

1. System records the **escalation** with the scanned data, the reason and the staff account, and notifies the **Manager** on the shared view.
2. **Manager** examines the booking and payment records and rules: accept, with a note, or refuse.
3. If accepted, System sets the booking to Checked-in on the **Manager**'s decision and resumes the basic flow at **{Party Size Confirmed}**.
4. If refused, System records the refusal.
5. **Front Staff** informs the **Customer** that entry cannot be granted. The use case ends.

EF-4 No-Show Marking After the Grace Period *(Increment 2)*

At the end of the round's **grace period**, for every Confirmed booking not yet Checked-in (Time actor),

1. **Front Staff** marks the booking as **no-show**, or System marks it automatically at the end of the **grace period**.
2. System shows the table as free on the **live view** and records that the fee is forfeited; the table is never offered to a **waitlist** (BRULE-06).
3. If **Front Staff** seats **walk-in** guests at the table, paid by hand at the venue, **Front Staff** marks the table as occupied (FR-72).
4. The use case ends.

EF-5 Check-In Cannot Be Saved

At **{Confirm Entry}**, if System cannot save the check-in when **Front Staff** confirms the entry,

1. System informs **Front Staff** that the check-in has not been confirmed; the booking stays Confirmed and the **live view** is unchanged.
2. **Front Staff** retries.
3. System checks the existing check-in status first so that no duplicate check-in is recorded.
4. Resume the basic flow at **{Party Size Confirmed}**.

2.2.3 UC-03 Create Concert Round

Table 2.4 Use-case description of Create Concert Round

Use Case Name	ID	Importance Level
Create Concert Round	UC-03	High — MVP; AF-2, AF-4 and EF-3 in Increment 2.
Primary Actor **Manager** Secondary actors: none.		**Use-Case Type** Business / Creation

Stakeholders and Interests

- **Manager**: Wants to publish a **concert round** quickly with the right **zone map**, **table types**, **package prices** and **booking-open time**, priced according to the venue's policy, and to see it in the **back-office live view** so that bookings and check-ins can be handled.
- **Customer**: Wants accurate round information, table availability and prices before reserving (UC-01).
- **Front Staff**: Wants the round's **check-in window** and **grace period** defined so that check-in works on the night (UC-02).

Brief Description

Manager creates a **concert round** in the **back-office**: enters the concert details, date, start time and **booking-open time**, chooses the venue **zone map** and marks the tables for sale, sets the **package price** of each **table type** in each **zone**, reviews the round as **Customers** will see it and publishes it. From its **booking-open time** the round is open for reservation in UC-01, and its **check-in window** and **grace period** drive check-in in UC-02.

Trigger

Manager wants to open a new **concert round** for booking. Trigger type: external.

Relationships

- Association: **Manager** (primary actor); no secondary actor.
- Include: none.
- Extend: none.
- Generalization: none.
- Related use cases: Requires the venue **zone map** (**zones**, tables and **table types**) created in UC-04 and the venue-issued **back-office** account of the **Manager**. Produces the **concert round** used in UC-01 (rounds, **zone map**, fees) and in UC-02 (**check-in window** and **grace period**). The **live view** of published rounds is shared by the **Manager**.
- Business rules: BRULE-04, 05, 07, 08.

Preconditions

- **Manager** is signed in to the **back-office** using a venue-issued username and password account.
- An Active venue **zone map** with **zones**, tables and **table types** exists (UC-04).
- The venue's pricing policy for the **packages** is known to the **Manager**; the **extra-person fee** (600 THB, BRULE-09), the **check-in window** and the **grace period** are **business parameters** of the venue (FR-38).

Postconditions

Success (basic flow; AF-2 copy; AF-3 edit):

- Round state = Published; the round appears to **Customers** in UC-01 from its **booking-open time**; its **check-in window** and **grace period** are available to UC-02; the **live view** shows the round.

Draft (AF-1 save as draft; AF-4 unpublish; EF-1 not corrected):

- Round state = Draft; the round is not visible to **Customers** and no booking is possible.

Unchanged (EF-2 cannot be saved):

- No round is created or changed; previously published rounds are unaffected.

Basic Flow

{Open Round Creation}

1. The use case begins when the **Manager** opens the **concert rounds** section of the **back-office** and chooses to create a new round.
2. System displays a new, empty round form.

{Enter the Concert Details}

3. **Manager** enters the concert name, the artist or programme, the date, the doors-open time and the start time.
4. **Manager** enters the **booking-open time** from which **Customers** may reserve.
5. System derives the **check-in window** of the round from the **business parameters** (from 2 hours before the start until 30 minutes after the start, BRULE-04, BRULE-05) and displays it for confirmation. **{Schedule Set}**

{Choose the Zone Map}

6. **Manager** selects the venue **zone map** to use for the round.
7. System displays the **zones**, tables and **table types** of the map.
8. **Manager** marks any table that is not for sale in this round.

{Set the Prices}

9. **Manager** sets the **package price** and the **package** content of each **table type** in each **zone** (BRULE-08).
10. System computes and displays the number of tables for sale and the capacity of each **zone**. **{Prices Set}**

{Review and Publish}

11. **Manager** chooses to review the round.
12. System validates the round (subflow S-1). **{Validation Result}**
13. System shows a preview of the round as **Customers** will see it in UC-01.
14. **Manager** publishes the round.
15. System saves the round as Published, makes it visible to **Customers** from its **booking-open time** and updates the **back-office live view**. **{Round Saved}**
16. The use case ends.

Subflows

S-1 Validate the Round

1. System checks that the doors-open time, the start time and the **booking-open time** are in order and that the **booking-open time** is before the start.
2. System checks that every **table type** for sale has a **package price** in each **zone**.
3. System checks that no other Published round overlaps the same date and time.
4. The subflow returns to the step that invoked it.

Alternative Flows

AF-1 Save as Draft

At any point before **{Review and Publish}**, if the **Manager** chooses to save without publishing,

1. System saves the round as Draft; it is not visible to **Customers**.
2. The use case ends.

AF-2 Copy from an Earlier Round *(Increment 2)*

At **{Open Round Creation}**, if the **Manager** chooses to copy an earlier round,

1. System pre-fills the form with the earlier round's **zone map**, table selection and prices; the date, the times and the **booking-open time** are left blank.
2. Resume the basic flow at **{Enter the Concert Details}**.

AF-3 Edit a Published Round

At **{Open Round Creation}**, if the **Manager** opens a Published round instead of creating a new one,

1. System shows the round with the number of Confirmed bookings on each table.
2. If the **booking-open time** of the round has not passed, **Manager** may change any field; resume the basic flow at **{Enter the Concert Details}**.
3. Otherwise the **zone map**, the tables for sale and the prices are fixed (BRULE-07, FR-35): System allows changes only to the concert details and, when the round has no Confirmed booking, to the date and the times, displaying the number of Confirmed bookings.
4. Resume the basic flow at **{Review and Publish}**.

AF-4 Unpublish a Round *(Increment 2)*

At **{Review and Publish}**, if the **Manager** withdraws a Published round that has no Confirmed booking,

1. System sets the round back to Draft and removes it from UC-01.
2. The use case ends.

Exception Flows

EF-1 Validation Fails

At **{Validation Result}**, if the round does not pass validation,

1. System marks the invalid fields and states the reason (times out of order, unpriced **table type**, overlapping round).
2. **Manager** corrects the fields.
3. Resume the basic flow at **{Review and Publish}**.

EF-2 Round Cannot Be Saved

At **{Round Saved}**, if System cannot save the round,

1. System informs the **Manager** that the round has not been published and keeps the entered data in the form.
2. **Manager** retries.
3. System checks whether the round already exists so that no duplicate round is created and saves the round.
4. The use case ends when the round is saved or the **Manager** abandons the form.

EF-3 Zone Map Changed While Editing *(Increment 2)*

At **{Choose the Zone Map}**, if the **zone map** has been changed in the **back-office** since the form was opened,

1. System reloads the map and highlights the tables that changed.
2. Resume the basic flow at **{Choose the Zone Map}**.

2.2.4 UC-04 Create Venue Zone Map

Table 2.5 Use-case description of Create Venue Zone Map

Use Case Name	ID	Importance Level
Create Venue Zone Map	UC-04	Medium — MVP; AF-2 in Increment 2.
Primary Actor **Manager** Secondary actors: none.		**Use-Case Type** Business / Creation

Stakeholders and Interests

- **Manager**: Wants the digital **zone map** to match the physical venue so that every **concert round** can be set up on it without rework, with the **zones**, tables and **table types** ready to select when creating a **concert round** (UC-03), and the layout reflecting the venue's capacity and policy.
- **Customer**: Wants accurate table positions and **table types** before reserving (UC-01).
- **Front Staff**: Wants the live **zone map** used at check-in to match the actual room (UC-02).

Brief Description

Manager creates or edits the venue **zone map** in the **back-office**: uploads an image of the venue, defines the **zones** on it, places the tables on the map, gives each table a table number, a **table type** and a seating capacity, and activates the map. System validates the map (named **zones**, unique table numbers, every table typed and sized) before it can be activated. Seat-view photographs of the tables are out of scope (a future feature). An Active **zone map** can be selected by the **Manager** when creating a **concert round** in UC-03 and is rendered as the **zone map** seen by **Customers** in UC-01 and by **Front Staff** in UC-02.

Trigger

Manager wants to set up the **zone map** of the venue for the first time or change it. Trigger type: external.

Relationships

- Association: **Manager** (primary actor); no secondary actor.
- Include: none.
- Extend: none.
- Generalization: none.
- Related use cases: Requires the venue-issued **back-office** account of the **Manager**. Produces the venue **zone map** (**zones**, tables and **table types**) that UC-03 requires when a **concert round** is created and that is rendered in UC-01 and UC-02.
- Requirements: FR-37, FR-39.

Preconditions

- **Manager** is signed in to the **back-office** using a venue-issued username and password account.
- The venue's **table types** (e.g. standard, high table, VIP sofa) and their seating capacities are known to the **Manager**.

Postconditions

Success (basic flow; AF-1 edit; AF-2 copy):

- Zone map state = Active; the map can be selected in UC-03 and is rendered in UC-01 and UC-02 for the rounds that use it.

Draft (AF-3 save as draft; EF-1 not corrected):

- Zone map state = Draft; the map cannot be selected in UC-03.

Unchanged (AF-1 blocked change; EF-2 cannot be saved):

- No **zone map** is created or changed; rounds already using the map are unaffected.

Basic Flow

{Open Map Editor}

1. The use case begins when the **Manager** opens the venue section of the **back-office** and chooses to create a new **zone map**.
2. System displays a new, empty map editor.

{Define the Zones}

3. **Manager** uploads the image of the venue's **zone map** (e.g. the printed map of the venue) (FR-39).
4. **Manager** draws the **zones** of the venue on the image (e.g. front stage, middle, bar) and names each **zone**.

{Place the Tables}

5. **Manager** places each table in its **zone** on the map.
6. **Manager** enters, for each table, its table number, **table type** and seating capacity (FR-37). **{Tables Defined}**
7. System displays the number of tables and the total capacity of each **zone**.

{Validate and Activate}

8. **Manager** chooses to activate the **zone map**.
9. System validates the **zone map** (subflow S-1). **{Validation Result}**
10. System shows a preview of the map as **Customers** will see it in UC-01.
11. **Manager** confirms the activation.
12. System saves the **zone map** as Active and makes it available for selection in UC-03. **{Map Saved}**
13. The use case ends.

Subflows

S-1 Validate the Zone Map

1. System checks that every **zone** has a name and contains at least one table.
2. System checks that table numbers are unique across the whole map.
3. System checks that every table has a **table type** and a seating capacity.
4. The subflow returns to the step that invoked it.

Alternative Flows

AF-1 Edit an Active Zone Map

At **{Open Map Editor}**, if the **Manager** opens an Active **zone map** instead of creating a new one,

1. System shows the map with the Published **concert rounds** that use it and the number of Confirmed bookings on each table.
2. If no Published round uses the map, **Manager** may change any **zone** or table; resume the basic flow at **{Define the Zones}**.
3. Otherwise System allows changes only to fields that do not affect existing bookings (**zone** names, tables with no Confirmed booking) and prevents the removal or relocation of tables with Confirmed bookings, displaying the affected bookings.
4. Resume the basic flow at **{Validate and Activate}**.

AF-2 Copy an Existing Zone Map *(Increment 2)*

At **{Open Map Editor}**, if the **Manager** chooses to copy an existing **zone map**,

1. System pre-fills the editor with the image, **zones**, tables and **table types** of the existing map as a new Draft.
2. Resume the basic flow at **{Define the Zones}**.

AF-3 Save as Draft

At any point before **{Validate and Activate}**, if the **Manager** chooses to save without activating,

1. System saves the **zone map** as Draft; it cannot be selected in UC-03.
2. The use case ends.

Exception Flows

EF-1 Validation Fails

At **{Validation Result}**, if the **zone map** does not pass validation,

1. System marks the invalid **zones** or tables and states the reason (unnamed or empty **zone**, duplicate table number, missing **table type** or capacity).
2. **Manager** corrects the **zones** or tables.
3. Resume the basic flow at **{Validate and Activate}**.

EF-2 Zone Map Cannot Be Saved

At **{Map Saved}**, if System cannot save the **zone map**,

1. System informs the **Manager** that the map has not been saved and keeps the entered data in the editor.
2. **Manager** retries.
3. System checks whether the map already exists so that no duplicate map is created and saves the map.
4. The use case ends when the map is saved or the **Manager** abandons the editor.

EF-3 Zone Map Image Cannot Be Uploaded

At **{Define the Zones}**, if the image of the venue cannot be uploaded,

1. System informs the **Manager** that the upload failed and keeps the **zones** already defined.
2. **Manager** retries the upload or chooses another image.
3. Resume the basic flow at **{Define the Zones}**.

2.2.5 UC-09 Maintain Customer Profile

Table 2.6 Use-case description of Maintain Customer Profile

Use Case Name Maintain Customer Profile	**ID** UC-09	**Importance Level** Medium — MVP.
Primary Actor **Customer** Secondary actors: none.		**Use-Case Type** Business / Maintenance

Stakeholders and Interests

- **Customer**: Wants to give name and phone once, with a clear purpose, and to keep them correct.
- **Manager**: Wants a correct name and phone for every booking and the consent that the personal data rules require.

Brief Description

The **customer profile** (name and phone, tied to the LINE user id) is created once, with the **Customer**'s consent to the data collection, when UC-01 reaches the profile step of the first booking; afterwards the **Customer** confirms or corrects it during a booking, or views and corrects it from My Bookings (BRULE-11).

Trigger

UC-01 reaches **{Complete the Customer Profile}**, or the **Customer** opens the profile from My Bookings. Trigger type: external.

Relationships

- Association: **Customer** (primary actor); no secondary actor.
- Include: none. Included by UC-01 at **{Complete the Customer Profile}**.
- Extend: none.
- Generalization: none.
- Related use cases: Runs inside UC-01 on every reservation and stands alone from My Bookings (UC-06). Relies on LINE Login for the identity of the **Customer** (BRULE-12).
- Business rules: BRULE-11, 12.

Preconditions

- **Customer** is signed in through LINE Login (UC-01 step 2).

Postconditions

Complete (basic flow; AF-2 corrected):

- A **customer profile** with name, phone and the time of consent is stored for the LINE user id; the including use case continues.

No profile (AF-1):

- Nothing is stored; the including use case is told that the profile is missing (UC-01 AF-5).

Basic Flow

{Look Up the Profile}

1. The use case begins when UC-01 reaches **{Complete the Customer Profile}**, or when the **Customer** opens the profile from My Bookings.
2. System looks the **customer profile** up by the LINE user id. **{Profile Found}**

{Create the Profile}

3. If no profile exists, System shows the purpose of the data collection (BRULE-11) and asks for consent, name and phone.
4. **Customer** gives consent and enters name and phone.

5. System validates the data and stores the profile with the time of consent. Resume the basic flow at **{Use Case Ends}**.

{Confirm the Profile}

6. Otherwise System shows the stored name and phone.
7. **Customer** confirms them, or corrects them; System validates and stores the correction.

{Use Case Ends}

8. The use case ends and returns to the step that included it.

Subflows

None.

Alternative Flows

AF-1 Consent Refused

At **{Create the Profile}**, if the **Customer** declines the consent to the data collection,

1. System explains that a booking cannot be made without it and stores nothing.
2. The use case ends without a profile.

AF-2 Invalid Profile Data

At **{Create the Profile}** or **{Confirm the Profile}**, if the name is empty or the phone number is not a valid Thai mobile number,

1. System marks the field and asks for a correction; a running **hold** timer keeps running.
2. Resume the basic flow at the same phase.

Exception Flows

EF-1 Profile Cannot Be Saved

At **{Create the Profile}** or **{Confirm the Profile}**, if System cannot save the profile,

1. System informs the **Customer** that the profile has not been saved and keeps the entered data.
2. **Customer** retries.
3. Resume the basic flow at the same phase.

2.2.6 UC-10 Pay the Full Table Fee

Table 2.7 Use-case description of Pay the Full Table Fee

Use Case Name Pay the Full Table Fee	**ID** UC-10	**Importance Level** High — MVP; EF-1, EF-2 and EF-3 in Increment 2.
Primary Actor **Customer** Secondary actors: **Payment Gateway** (simulated in the MVP, ADR-11) and, from Increment 2, the **Manager** (transfer-slip review). The **hold** timer stays with UC-01.		**Use-Case Type** Business / Transactional
Stakeholders and Interests • **Customer**: Wants to pay once, safely, and to know at once whether the payment went through. • **Manager**: Wants the **full table fee** received and verified before any booking is confirmed (BRULE-01). • **Payment Gateway**: Wants correct payment requests and reliable delivery of payment results.		
Brief Description The **Customer** pays the **full table fee** of a held booking in the hosted checkout of the **Payment Gateway**. The gateway reports the result by signed webhook; System verifies the result and the amount, records the payment once and returns to UC-01, which confirms the booking. In the		

MVP the gateway is simulated (ADR-11); the real gateway, the automatic refund of a late payment and the **degraded mode** come in Increment 2.

Trigger

UC-01 reaches **{Pay the Full Table Fee}** with the **booking terms** accepted. Trigger type: external.

Relationships

- Association: **Customer** (primary actor); **Payment Gateway** and, from Increment 2, the **Manager** (secondary actors).
- Include: none. Included by UC-01 at **{Pay the Full Table Fee}**.
- Extend: none.
- Generalization: none.
- Related use cases: Pays the booking held in UC-01; the **hold** and its expiry (UC-01 EF-1) bound the payment. The payment status is shown under My Bookings (UC-06).
- Business rules: BRULE-01, 17.

Preconditions

- The booking is Held with its **party size**, its **full table fee** and the accepted **booking terms** (UC-01 steps 7 to 14).
- The **Payment Gateway** is available: in the MVP the simulated gateway; from Increment 2 the merchant account is active and the shop's bank account for **degraded mode** is configured.

Postconditions

Paid (basic flow; EF-2 slip approved, Increment 2):

- The payment is recorded once, with its gateway reference; UC-01 continues at **{Confirm the Booking}**.

Not paid (AF-1; EF-4; EF-2 slip rejected or missing, Increment 2):

- No payment is recorded; the **hold** is unchanged after a decline (AF-1) or has ended (EF-4, EF-2), in which case UC-01 EF-1 has set the booking to Expired.

Refunded (EF-1, table no longer available; Increment 2):

- The payment and its refund are both recorded; the table stays with the other customer; the **Customer** has been informed by LINE.

Basic Flow

{Create the Payment Request}

1. The use case begins when UC-01 reaches **{Pay the Full Table Fee}**.
2. System creates a payment request for the **full table fee** with the **Payment Gateway** and opens the gateway's hosted checkout inside the web app.

{Pay}

3. **Customer** chooses a payment method (PromptPay QR, card, mobile banking, e-wallet); in the MVP the simulated gateway offers a successful and a declined payment.
4. **Customer** submits the payment through the chosen method before the **hold** expires.

{Payment Result}

5. **Payment Gateway** verifies the payment and sends the payment result to System by signed webhook. **{Payment Result Received}**
6. System verifies the authenticity of the payment result and the amount and records the payment; if the same payment result arrives again, System ignores the duplicate. **{Payment Verified}**

{Use Case Ends}

7. The use case ends and returns to UC-01 at **{Confirm the Booking}**.

Subflows

None.

Alternative Flows

AF-1 Payment Declined

At **{Payment Result Received}**, if the **Payment Gateway** reports that the payment was declined,

1. System shows the decline and the remaining hold time and informs the **Customer** by LINE message (FR-21).
2. Resume the basic flow at **{Pay}**.

Exception Flows

EF-1 Payment Result Arrives After the Hold Expired *(Increment 2)*

At **{Payment Result Received}**, if a successful payment result arrives after the **hold** has expired (UC-01 EF-1),

1. System checks whether the table is still available.
    - a. If the table is still available, System places a new **hold** and records the payment; the use case ends and UC-01 resumes at **{Confirm the Booking}**.
    - b. If the table has been taken, System requests a refund of the payment through the gateway's refund API, records the refund, and informs the **Customer** by LINE message that the table was taken in the meantime and the payment is being returned (UC-01 subflow S-1). The use case ends.

EF-2 Payment Gateway Unreachable (Degraded Mode) *(Increment 2)*

At **{Create the Payment Request}**, if the **Payment Gateway** cannot be reached,

1. System keeps the **hold**, shows the shop's bank account and the **full table fee**, and asks the **Customer** to transfer the fee and attach the **transfer slip**.
2. **Customer** transfers the fee in a banking application and attaches the slip in the frontend.
3. System stores the slip with the booking, marks the booking "awaiting slip verification", extends the **hold** until the **Manager**'s decision (**degraded mode**) and notifies the **Manager**.
4. **Manager** reviews the slip in the **back-office**.
    - a. If the **Manager** confirms the payment, System records the payment; the use case ends and UC-01 resumes at **{Confirm the Booking}**.
    - b. If the **Manager** rejects the slip, or no slip is attached before the **hold** ends, System releases the table, sets the booking to Expired and informs the **Customer** by LINE message (UC-01 subflow S-1). The use case ends.

EF-3 Payment Result Not Received *(Increment 2)*

At **{Payment Result}**, if no payment result has arrived within 60 seconds after the gateway's checkout reported completion,

1. System queries the gateway's payment status API every 10 seconds until a result is known or until 10 minutes after the **hold** ends (NFR-23).
    - a. If the payment is confirmed, System records it; resume the basic flow at **{Payment Verified}**.
    - b. If the **hold** ends first, EF-4 applies; any payment result that arrives later follows EF-1.

EF-4 Hold Expires During Payment

At any point before **{Payment Verified}**, if the **hold period** ends (UC-01 EF-1),

1. UC-01 EF-1 applies: the table is released, the booking is Expired and the **Customer** is informed.
2. The use case ends without a payment.

3 Requirements

The functional and non-functional requirements below are those that the use cases of Section 2 need. The Increment column refers to Section 1.3, and the business rules that the requirements and the use cases refer to (BRULE-nn) are listed in Appendix B.

3.1 Functional Requirements

3.1.1 User Authentication & Login System

Table 3.1 User authentication and login

ID	Requirement	Use case	Increment
FR-01	The system shall allow customers to authenticate using LINE Login; the LINE user id is the customer identity and there is no separate registration.	UC-01	MVP
FR-02	If LINE Login fails or is cancelled, the system shall explain that a LINE account is required and return to the Rich Menu.	UC-01	MVP
FR-40	The system shall allow customers to access only their own reservations and **e-tickets**: My Bookings lists each booking with its round, table, status and **e-ticket**.	UC-01	MVP
FR-65	The system shall authenticate all internal role accounts (**Manager**, **Front Staff**, and the **Owner** with read-only access) with a username and password issued by the venue, separately from customer LINE Login; the **Manager** creates and disables the accounts and assigns their roles.	UC-02 to UC-04	MVP
FR-66	The system shall give each role access only to the functions it owns; the **Owner** can read everything.	UC-02 to UC-04	MVP
FR-73	The system shall allow all users to log out securely.	all	MVP

3.1.2 Venue Zone Map & Concert Round Management System

Table 3.2 Venue zone map and concert round management

ID	Requirement	Use case	Increment
FR-37	The system shall allow the **Manager** to define **zones**, **table types** (capacity, **package** content and price) and table numbers.	UC-04, UC-03	MVP
FR-39	The system shall allow the **Manager** to upload the image of the venue's **zone map** and place each table on it.	UC-04	MVP

ID	Requirement	Use case	Increment
FR-74	The system shall validate a **zone map** before activation: every **zone** named and non-empty, table numbers unique, and every table with a **table type** and a seating capacity.	UC-04	MVP
FR-34	The system shall allow the **Manager** to create and edit a **concert round** with its artist, date, start time, **booking-open time** and status (not yet open, open, sold out, finished, cancelled).	UC-03	MVP
FR-35	The system shall assign a **zone map** to each round and lock it once the **booking-open time** has passed.	UC-03	MVP
FR-75	The system shall validate a **concert round** before publishing and prevent overlapping published rounds.	UC-03	MVP
FR-36	The system shall allow the **Manager** to withdraw a round that has no Confirmed booking.	UC-03	Increment 2
FR-38	The system shall keep the **business parameters** (**hold period**, **grace period**, **check-in window**, **extra-person fee**) as settings that change without a code change and apply to rounds that open for booking afterwards.	UC-03	MVP
FR-03	The system shall list the upcoming rounds with artist, date, start time, **booking-open time** and status (not yet open, open, sold out).	UC-01	MVP
FR-04	The system shall let customers select the tables of a round only from its **booking-open time**.	UC-01	MVP
FR-05	The system shall display the **zone map** of a published round with each table's number, **zone**, **table type**, **package price** and status (available, held, booked).	UC-01	MVP
FR-06	The system shall show a table status change on every open map within 2 seconds.	UC-01, UC-02	MVP (polling, ADR-09)

3.1.3 Table Reservation System

Table 3.3 Table reservation

ID	Requirement	Use case	Increment
FR-07	The system shall hold an available table for the **hold period** (15 minutes, BRULE-02) and show the remaining time.	UC-01	MVP
FR-08	The system shall allow at most one hold or booking per table per round; a later selection is refused and the map refreshed (**first lock wins**).	UC-01	MVP
FR-09	The system shall ask for the **party size** and compute the **full table fee**: the **package price** plus the **extra-person fee** for each person above the capacity of the **table type**.	UC-01	MVP
FR-10	On the first booking, the system shall show the purpose of data collection and obtain consent before collecting name and phone into a **customer profile**, which is pre-filled for later bookings; without consent the booking is cancelled.	UC-09	MVP

ID	Requirement	Use case	Increment
FR-11	The system shall let the customer cancel a **hold**; the table becomes available at once and nothing is charged.	UC-01	MVP
FR-12	The system shall display the **booking terms** (full payment confirms the booking, **check-in window**, **grace period**, no refund for a **no-show**) and require their acceptance before payment.	UC-01	MVP
FR-23	The system shall release the table when the **hold** ends (±5 seconds), set the booking to Expired and notify the customer, except while a **transfer slip** awaits the **Manager**'s decision.	UC-01	MVP

3.1.4 Payment

Table 3.4 Payment

ID	Requirement	Use case	Increment
FR-13	The system shall create a payment request for the **full table fee** and present the gateway's payment methods in its hosted checkout inside the web app.	UC-10	MVP (simulated gateway, ADR-11)
FR-14	After a declined or failed payment, the system shall allow a retry while the **hold** remains.	UC-10	MVP
FR-16	The system shall verify the signature and the amount of every payment result, process it exactly once, record the payment and confirm the booking.	UC-10	MVP
FR-17	A successful payment result that arrives after the **hold** expired shall confirm the booking if the table is still free; otherwise the system refunds it through the gateway and informs the customer.	UC-10	Increment 2
FR-18	If no payment result arrives within 60 seconds of checkout completion, the system shall poll the payment status every 10 seconds until a result is known or until 10 minutes after the **hold** ends.	UC-10	Increment 2
FR-69	If the gateway does not answer within 30 seconds, the system shall switch new payments to **degraded mode** (bank transfer, slip, **Manager** confirmation) and return to gateway mode within 1 minute after it answers again.	UC-10	Increment 2
FR-15	In **degraded mode** the system shall keep the **hold**, show the shop's bank account, accept a **transfer slip**, extend the **hold** until the **Manager** decides and notify the **Manager**.	UC-10	Increment 2
FR-44	The system shall queue the slip bookings for the **Manager** to confirm or reject; a confirmed slip continues the booking as a gateway payment would.	UC-10	Increment 2
FR-71	The system shall accept cards only through the gateway's hosted checkout; no card data is entered in or stored by the system.	UC-10	Increment 2

3.1.5 E-Ticket & Check-In System

Table 3.5 E-ticket and check-in

ID	Requirement	Use case	Increment
FR-19	The system shall issue one **e-ticket** per Confirmed booking, with a signed **booking reference** valid only for that round and table, shown on screen and under My Bookings.	UC-01	MVP
FR-24	The system shall let **Front Staff** scan the QR code with the phone camera and decode the signed **booking reference**.	UC-02	MVP
FR-25	The system shall accept a **booking reference** typed by hand.	UC-02	MVP
FR-26	The system shall verify in real time that the booking exists, is Confirmed, is for tonight's round and not yet checked in, and that the **check-in window** is open, and show the result within 2 seconds with table, **zone**, **package**, **party size**, name and payment status.	UC-02	MVP
FR-27	On a failed verification the system shall give the reason: already checked in (with time and staff account), invalid signature, wrong round, transferred, cancelled, window not open, or **grace period** passed.	UC-02	MVP
FR-28	The system shall set the booking to Checked-in with the time and the staff account and update the **live view**.	UC-02	MVP
FR-42	The system shall give the **Manager** a **live view** of the current round's bookings and tables with their status; the **Owner** reads the same view.	UC-02	MVP
FR-30	The system shall find tonight's Confirmed bookings by name, phone or **booking reference** and allow check-in from the result.	UC-02	Increment 2
FR-31	The system shall let **Front Staff** escalate an invalid or duplicate ticket to the **Manager** with the scanned data, the reason and the staff account, and apply the **Manager**'s decision.	UC-02	Increment 2
FR-48	The system shall present escalated tickets to the **Manager**, who accepts (with a note) or refuses; the decision is recorded and returned to **Front Staff**.	UC-02	Increment 2
FR-33	After the **grace period** the system shall let **Front Staff** mark a **no-show**, or mark it automatically; the table is shown as free, the forfeited fee is recorded, and the table is never offered to a **waitlist**.	UC-02	Increment 2
FR-72	The system shall let **Front Staff** mark a freed **no-show** table as occupied by **walk-in** guests, who are seated and paid by hand at the venue.	UC-02	Increment 2

3.1.6 Notification System

Table 3.6 Notification

ID	Requirement	Use case	Increment
FR-20	The system shall send the booking confirmation with the **e-ticket** and the **booking terms** through the LINE Messaging API within 1 minute.	UC-01	MVP
FR-21	The system shall send a LINE message when a **hold** expires, when a payment fails and, from Increment 2, when a late payment is refunded.	UC-01, UC-10	MVP
FR-22	If a message cannot be delivered, the system shall retry 3 times within 5 minutes, record the outcome and keep the **e-ticket** available in the web app.	UC-01	MVP

3.1.7 Out of Scope

The following are planned for a later release, and this project does not build them: scheduled LINE reminders, the "on my way" grace extension (pending the owner's decision), the **waitlist**, booking transfer, and reports and analytics. Seat-view photographs of the tables are a future feature.

3.2 Non-functional Requirements

3.2.1 Operational

Table 3.7 Operational requirements

ID	Requirement
NFR-03	The customer web app shall work in the current LINE in-app browser on iOS and Android; the **back-office** in the last two versions of Chrome, Safari and Edge and in the staff's phone browsers.
NFR-34	The customer web app shall open from the Rich Menu of the **LINE Official Account**, and the payment checkout shall run inside the same LIFF session.
NFR-06	The system shall back up reservation and venue data daily; a restore loses at most 24 hours of data.
NFR-31	Test and production shall be separated (simulated or sandbox payment gateway, LINE test channel); no production key is kept in the source code.
NFR-32	The operating cost shall stay within 1,000 THB per month at 60 bookings per concert night.

3.2.2 Performance

Table 3.8 Performance requirements

ID	Requirement
NFR-01	The system shall accept at least 5 hold requests per second at booking open.
NFR-04	Page loads, holds and ticket verification shall complete within 2 seconds at the 95th percentile under the load of NFR-01.
NFR-25	A table status change shall be visible on every open map within 2 seconds.
NFR-26	A booking shall be Confirmed with its **e-ticket** within 10 seconds of the payment result, and the LINE confirmation sent within 60 seconds of the confirmation.
NFR-27	The system shall serve 200 customers browsing at once and rounds of up to 60 tables, with 50 % headroom.
NFR-05	Check-in from the scan to Checked-in shall take at most 15 seconds per table.

3.2.3 Reliability

Table 3.9 Reliability requirements

ID	Requirement
NFR-19	The system shall be available at least 99.5 % of the opening hours (17:00 to 02:00) of concert nights, over a rolling 12 months.
NFR-02	The system shall recover within 2 hours of a failure during opening hours.
NFR-20	Of concurrent holds on one table exactly one shall succeed, tested with at least 50 simultaneous attempts.
NFR-21	An unpaid hold shall be released within 10 seconds of the end of its 15 minutes, with a timer accuracy of ±5 seconds.
NFR-22	Duplicate or out-of-order payment results shall be processed exactly once; unknown or already settled results are logged and ignored.
NFR-23	Every payment shall reach a final state (paid, failed, expired, refunded) within the **hold period** plus 10 minutes, through the webhook or, from Increment 2, the status query.

3.2.4 Security

Table 3.10 Security requirements

ID	Requirement
NFR-36	The identity tokens of LINE Login (OpenID Connect) shall be validated on the server.
NFR-38	The payment webhook shall verify the signature of every call and reject calls older than 5 minutes or with a reused id.
NFR-39	All traffic shall use HTTPS with TLS 1.2 or later; there is no plain-text endpoint.

ID	Requirement
NFR-44	The system shall protect customer information and authentication credentials; access to reservations and **e-tickets** follows FR-40 and FR-66.
NFR-42	No card data shall pass through the system; the PCI DSS scope stays with the gateway (Increment 2, real gateway).

3.2.5 Cultural and Legal

Table 3.11 Cultural and legal requirements

ID	Requirement
NFR-15	The interface shall be in Thai and English, following the LINE language setting and switchable.
NFR-45	The system shall display booking and check-in times in Thailand's local time **zone**.
NFR-40	The system shall handle personal information in accordance with Thailand's PDPA: purpose, consent, access and correction, retention and deletion, and breach handling, with a privacy notice.
NFR-33	Personal data shall be kept for at most 24 months by default and then deleted or anonymised.
NFR-41	Package displays, prices and messages shall be factual and shall not advertise alcohol (Alcoholic Beverage Control Act B.E. 2551).

3.2.6 Usability

Table 3.12 Usability requirements

ID	Requirement
NFR-13	The customer web app shall fit a 360 × 640 px viewport in the LINE in-app browser without horizontal scrolling, with touch targets of at least 44 px.
NFR-14	A first-time customer shall get from the Rich Menu to the start of payment within 3 minutes, tested with 5 users.
NFR-16	Table status shall be shown by colour plus a label or pattern, with a contrast of at least 4.5:1.
NFR-17	The **back-office** shall work on a desktop and on the staff's phone; the verification result is one tap after the scan.
NFR-18	Every customer-facing failure message shall say what happened and what to do next.

3.2.7 Interfaces

Table 3.13 Interface requirements

ID	Requirement
NFR-37	The payment gateway shall sit behind one payment-service interface (create payment, receive result, query status, refund) so that the gateway can be replaced without changing the reservation logic.
NFR-35	The QR code shall be read by the phone camera in the browser, with no dedicated scanner.

4 Architecture Decision Records (ADRs)

The ADRs follow the course template (Michael Nygard's format: Title, Context, Decision, Status, Consequences); each ADR is a table of these five fields and starts on a new page. An accepted ADR is not rewritten when its decision changes: a new ADR supersedes it, and the old record keeps its text with a status that names its successor. Corrections to how a record is written, such as the teacher's comment on ADR-06 (FB-D1-01), are made in place and listed in the change-log document. The Part column names the part of the system that each decision concerns: the Frontend, the Backend or the External systems (Section 5.2).

Table 4.1 Architecture decision records

ADR	Title	Part of the system	Status
ADR-01	Frontend Architecture & Client Channel	Frontend	Accepted
ADR-02	Real-Time Floor Plan Communication	Frontend, Backend	Superseded by ADR-08 and ADR-09
ADR-03	Backend Language & Framework	Backend	Accepted; Socket.io deferred by ADR-09
ADR-04	Notification Engine	Backend, External systems	Superseded by ADR-10
ADR-05	Interactive Floor Plan Rendering	Frontend	Accepted
ADR-06	Primary Database Engine	Backend	Accepted (revised after FB-D1-01)
ADR-07	Internal Role Authentication (**Front Staff**, **Manager**)	Frontend, Backend	Accepted
ADR-08	Table Hold and Concurrency Control	Backend	Accepted; the lock moved to the Booking Service by ADR-13
ADR-09	Table Status Updates in the MVP: Polling	Frontend, Backend	Accepted
ADR-10	**Customer** Notifications through the LINE Messaging API Only	Backend, External systems	Accepted
ADR-11	Simulated **Payment Gateway** for the MVP	Backend, External systems	Accepted
ADR-12	Communication: REST at the API Gateway, gRPC inside the Backend	All three parts	Accepted
ADR-13	The Booking Owns the Hold; the Table Map Is a Read Model	Backend	Accepted
ADR-14	Modular Monolith Mode for Development and Tests	Backend	Accepted

4.1 ADR-01: Frontend Architecture & Client Channel

Table 4.2 ADR-01 Frontend Architecture & Client Channel

Title	Frontend Architecture & Client Channel
Context	Target users primarily interact via mobile phones while coordinating nightlife plans on LINE. Requiring an App Store/Play Store download would cause high user drop-off. The client must support mobile responsiveness and rich interactive graphics for table selection.
Decision	Use Next.js (React) bundled as a LINE Front-end Framework (LIFF) Web App, with fallback access via standard mobile browsers.
Status	Accepted
Consequences	• Eliminates app install friction; customers book directly inside the LINE app. • Allows seamless integration with LINE Login and LINE Messaging API. • Requires compliance with LIFF webview constraints and in-app browser caching behaviors.

4.2 ADR-02: Real-Time Floor Plan Communication

Table 4.3 ADR-02 Real-Time Floor Plan Communication

Title	Real-Time Floor Plan Communication
Context	When multiple customers look at the **zone map** simultaneously on busy nights, table statuses must update instantaneously to prevent double-booking conflicts and outdated views.
Decision	Implement WebSocket (or Server-Sent Events - SSE) for real-time table status broadcasting, combined with short-term Redis distributed locks for table selection checkout.
Status	Superseded by ADR-08 (table hold) and ADR-09 (table status updates) on 2026-09-29.
Consequences	• Delivers sub-second UI updates when tables are locked, booked, or released. • Avoids wasteful client polling against the backend server. • Requires maintaining persistent connection pools and managing socket disconnect/reconnect states on mobile networks.

4.3 ADR-03: Backend Language & Framework

Table 4.4 ADR-03 Backend Language & Framework

Title	Backend Language & Framework
Context	The system requires rapid development cycles while supporting RESTful APIs, real-time bi-directional communication for **zone map** updates, seamless integration with the LINE Messaging API/LIFF SDK, and scheduled background tasks to automatically release expired table reservations past the cutoff window. Options considered include Go, Java, and Node.js (JavaScript). Given that the frontend is built using Next.js/React, adopting a unified language stack across client and server significantly streamlines development.
Decision	Use Node.js (JavaScript) with the Express.js framework for the core API services, combined with Socket.io for real-time WebSocket communication and node-cron for background cutoff scheduling.
Status	Accepted. Amended on 2026-09-29: the Socket.io part is deferred to Increment 2 by ADR-09, and node-cron runs the scheduled jobs of ADR-08 (hold expiry) instead of cutoff scheduling.
Consequences	• Enables a unified full-stack JavaScript environment, allowing the team to share utility logic, minimize context switching, and accelerate feature delivery. • Leverages Node.js's event-driven, non-blocking I/O model to handle concurrent client connections efficiently during peak reservation hours. • Because JavaScript is dynamically typed, the backend requires robust runtime request validation and clear documentation to prevent data type mismatches.

4.4 ADR-04: Notification Engine

Table 4.5 ADR-04 Notification Engine

Title	Notification Engine
Context	Reminders before cutoff times are critical to reducing lost tables. Traditional SMS is costly per transaction, whereas email is rarely checked by nightlife patrons in real time.
Decision	Use LINE Messaging API (Push Messages / Flex Messages) as the primary notification channel, supplemented by Web Push notifications.
Status	Superseded by ADR-10 on 2026-09-29.
Consequences	1. Near 100% open rate with interactive rich cards ("On My Way" / "Postpone 30 mins"). 2. Low messaging cost within standard official account quotas. 3. Requires users to add or link the venue's **LINE Official Account** (OA).

4.5 ADR-05: Interactive Floor Plan Rendering

Table 4.6 ADR-05 Interactive Floor Plan Rendering

Title	Interactive Floor Plan Rendering
Context	The **zone map** needs to support responsive rendering, pinch-to-zoom, **zone** highlighting, and seat view popups without lagging on mid-range smartphones.
Decision	Use HTML5 Canvas / SVG (via Fabric.js or Konva.js) embedded within the Next.js client.
Status	Accepted
Consequences	• Smooth vector scaling and interactive **zone**/table hit-testing. • **Managers** can visually manipulate coordinates for tables and stages. • Requires custom responsive coordinate mapping to ensure the **zone map** adapts across various mobile screen sizes.

4.6 ADR-06: Primary Database Engine

Table 4.7 ADR-06 Primary Database Engine

Title	Primary Database Engine
Context	The system needs to store diverse data structures, including flexible venue **zone maps** (coordinates, shapes, multi-table **zone** configs, the image of the venue), booking transaction records, **customer profiles**, and time-stamped status transitions. Traditional relational databases (RDBMS) enforce rigid table schemas that make storing deeply nested **zone map** geometries and dynamic table attributes cumbersome. A **zone map** is naturally one nested document (**zones** that contain tables with their coordinates, types and capacities) whose shape differs from venue to venue, so the **zone maps** need a flexible schema that stores such a document whole, without complex multi-table joins. Additionally, the backend runtime is Node.js/JavaScript, making native JSON/BSON document handling particularly advantageous for rapid development.
Decision	Use MongoDB as the primary NoSQL document database, accessed via the Mongoose Object Data Modeling (ODM) library in the Node.js backend.
Status	Accepted. Revised on 2026-09-29 after the teacher's feedback (FB-D1-01): the need for a flexible schema moved from Consequences to Context.
Consequences	• **Seamless Node.js Integration:** Documents map directly to JavaScript objects (JSON) reducing serialization overhead and development friction. • **Atomic Single-Document Updates:** A table's status can be changed with one conditional update, which gives the first-lock-wins hold of ADR-08 without a separate lock server. • **Horizontal Scalability:** Supports horizontal partitioning (sharding) should the platform expand to serve multiple nightlife venues concurrently. • **Validation Moves into the Application:** The flexible schema does not check the documents, so every service validates what it stores with Mongoose schemas. • **Multi-Document Changes Need Care:** A change that spans several documents, such as a booking and its payment, needs a multi-document transaction, which MongoDB supports only on a replica set, or separate steps that can be retried safely.

4.7 ADR-07: Internal Role Authentication (Front Staff, Manager)

Table 4.8 ADR-07 Internal Role Authentication (Front Staff, Manager)

Title	Internal Role Authentication (**Front Staff**, **Manager**)
Context	**Customers** reach the web app through LINE and are authenticated with LINE Login, as decided in ADR-01. **Front Staff** and **Managers** use internal accounts created by the venue, and front-of-house staff work from shared devices. Binding those accounts to personal LINE profiles would tie venue access to individual social accounts and make shift handover and staff turnover difficult. Options considered: LINE Login for every role, venue-issued username and password accounts, and an external identity provider.
Decision	Use venue-issued username and password accounts with JWT sessions for all internal roles (**Front Staff**, **Manager**, and the **Owner** with read-only access, FR-65), kept separate from the customer LINE Login flow. Each account carries a role that determines which **back-office** functions it may use (e.g. check-in for **Front Staff**; **zone map** and round creation for the **Manager**). The **Manager** creates, updates, and deactivates internal accounts from the venue configuration.
Status	Accepted
Consequences	• Venue access stays under the owner's control and survives staff turnover, because accounts are not tied to personal LINE profiles. • Requires the system to implement password storage, session expiry, and account lockout instead of delegating all authentication to LINE. • Two authentication paths must be maintained and tested, one for customers and one for internal roles.

4.8 ADR-08: Table Hold and Concurrency Control

Table 4.9 ADR-08 Table Hold and Concurrency Control

Title	Table Hold and Concurrency Control
Context	Selecting a table gives the **Customer** a **hold** of 15 minutes (BRULE-02, FR-07). When several customers select the same table, exactly one hold may succeed (**first lock wins**, BRULE-03, FR-08); NFR-20 tests this with at least 50 simultaneous attempts. An unpaid hold must be released within 10 seconds of its end, with a timer accuracy of ±5 seconds, and the **Customer** informed (FR-23, NFR-21). Version 1.1 named three different mechanisms for this: short-term Redis distributed locks (ADR-02), MongoDB TTL indexes (ADR-06) and a node-cron job (ADR-03). Options considered: a Redis lock with an expiry time, a MongoDB TTL index on hold documents, and a conditional update of the table's status with a scheduled release job.
Decision	The Table Availability Service owns the status of every table of every round in its own database. holdTable() is one conditional update that changes a table from available to held only if it is still available, and stores the booking and the end of the **hold**; exactly one of several concurrent requests succeeds. The Booking Service owns the **hold** timer of the booking: a scheduled job (node-cron, ADR-03) runs every 5 seconds, calls expireUnpaidBookings(), sets every overdue Held booking to Expired, calls releaseHold() of the Table Availability Service and asks the Notification Service to send the hold-expired notice. No Redis server and no TTL index are used.
Status	Accepted on 2026-09-29. Supersedes the locking part of ADR-02 and the TTL consequence of ADR-06. Superseded in part on 2026-09-30 by ADR-13: the Booking Service owns the hold and the Table Availability Service keeps the read model; the timer job of this record stays.
Consequences	• One atomic operation in one database gives **first lock wins**, and it can be load-tested directly (NFR-20). • A 5-second job meets the 10-second release requirement. A MongoDB TTL index would not: its background task runs only every 60 seconds, and it deletes the document instead of releasing the table and informing the **Customer**. • No extra infrastructure is needed, which helps keep the operating cost within the limit of NFR-32. • The two services must agree after a crash between the booking update and the table release: the job retries releaseHold() for Expired bookings whose table is still held, so releaseHold() must be idempotent.

4.9 ADR-09: Table Status Updates in the MVP: Polling

Table 4.10 ADR-09 Table Status Updates in the MVP: Polling

Title	Table Status Updates in the MVP: Polling
Context	A table status change must appear on every open map within 2 seconds (FR-06, NFR-25), with up to 200 customers browsing at once and rounds of up to 60 tables (NFR-27). ADR-02 chose WebSocket or Server-Sent Events, and the design constraints of the system also prefer pushed map updates to polling. The teacher's feedback on Deliverable #2 (FB-D2-01) recommends leaving real-time WebSocket out of the MVP and adding it later.
Decision	In the MVP the Customer Web App and the Back-office Web App poll getRoundTableStatus() of the Table Availability Service through the API Gateway every 2 seconds while a **zone map** is open; the response carries a version number of the round's table status, so an unchanged map costs one small response. In Increment 2 the Table Availability Service pushes status changes to the open maps by WebSocket (Socket.io, ADR-03), and polling stays as the fallback after a lost connection.
Status	Accepted on 2026-09-29. Supersedes the real-time part of ADR-02.
Consequences	• A change reaches every open map within 2 seconds plus one response time, as FR-06 requires, and the MVP has no persistent connections to manage. • 200 customers polling every 2 seconds make about 100 small requests per second at booking open; the answer is one status document per round, which the Table Availability Service can cache. • Between two polls a customer can still select a table that has just been taken; the conditional update of ADR-08 refuses the **hold** and UC-01 AF-3 refreshes the map. • Requests are wasted when nothing changes, which is the cost ADR-02 wanted to avoid; Increment 2 removes most of it.

4.10 ADR-10: Customer Notifications through the LINE Messaging API Only

Table 4.11 ADR-10 Customer Notifications through the LINE Messaging API Only

Title	**Customer** Notifications through the LINE Messaging API Only
Context	ADR-04 chose LINE Messaging API push messages supplemented by Web Push, mainly for reminders before cutoff times with "On My Way" and "Postpone 30 mins" buttons. The requirements now use the **check-in window** and the **grace period** as the arrival rule; reminders and the grace extension are planned for a later release and are out of scope (Section 3.1.7). The customer web app runs inside LINE's in-app browser (LIFF), which does not support the Push API that Web Push needs, and the system is LINE-only. The messages in scope are the booking confirmation with the **e-ticket**, the hold-expired and payment-failed notices and, from Increment 2, the refund and slip-decision notices (FR-20, FR-21), each retried 3 times within 5 minutes (FR-22) and sent within the Official Account's monthly push quota.
Decision	Send every customer notification as a LINE Messaging API push message (a Flex Message for the **e-ticket**) from the Notification Service through the LINE Messaging Adapter. No Web Push, SMS or e-mail.
Status	Accepted on 2026-09-29. Supersedes ADR-04.
Consequences	• One channel that every customer already has, since the LINE account is the customer's identity (BRULE-12). • The **Customer** must be a friend of the venue's **LINE Official Account**, as the precondition of UC-01 already requires. • Push messages count against the monthly quota; the MVP sends only transactional messages. • When LINE is unavailable the confirmation is delayed, but the **e-ticket** stays available under My Bookings (FR-22).

4.11 ADR-11: Simulated Payment Gateway for the MVP

Table 4.12 ADR-11 Simulated Payment Gateway for the MVP

Title	Simulated **Payment Gateway** for the MVP
Context	UC-01 pays the **full table fee** through a payment gateway (BRULE-01, FR-13, FR-16). A payment gateway feasibility study selected Beam Checkout, with Opn Payments as the fallback; the requirements ask for one payment-service interface so that the gateway can be replaced (NFR-37) and a sandbox for testing (NFR-31). Onboarding with a real gateway needs a registered merchant. The teacher's feedback on Deliverable #2 (FB-D2-01) recommends a simulated payment in the MVP, and automatic refunds, transfer-slip review and the degraded payment mode later.
Decision	The Payment Service reaches a gateway only through the Payment Gateway Adapter, the port of Section 5 (createCheckoutSession(), verifyWebhookSignature(), queryPaymentStatus(), refundPayment()). In the MVP the adapter is backed by a simulated gateway: a small checkout page, opened inside the web app like a hosted checkout, where the tester chooses to pay or to decline, after which it sends a signed webhook to the API Gateway as a real gateway would. In Increment 2 a Beam Checkout adapter replaces it, first against Beam's sandbox, together with status polling, automatic refund of a late payment and the **degraded mode**.
Status	Accepted on 2026-09-29.
Consequences	• The MVP demonstrates the whole booking flow, including a declined payment and the signed webhook, without a merchant account or real money. • The Payment Service and the Booking Service do not change when the real gateway arrives; only the adapter does (NFR-37). • The simulated gateway must never run in production; only the test configuration enables it (NFR-31). • What only a real gateway shows (late or missing results, refunds, outages) is not exercised until Increment 2.

4.12 ADR-12: Communication: REST at the API Gateway, gRPC inside the Backend

Table 4.13 ADR-12 Communication: REST at the API Gateway, gRPC inside the Backend

Title	Communication: REST at the API Gateway, gRPC inside the Backend
Context	SEATS has three parts (Section 5.2): the Frontend in the users' browsers, the Backend on the servers of SEATS, and the External systems. Version 1 of the architecture (Deliverable #2) used REST for every call, as the Deliverable #2 brief allowed for a first version; the brief asks the later versions to choose REST, gRPC or asynchronous messaging for each part according to its work, and the minimum requirements of the course project include at least one REST service and one gRPC service. Three kinds of call cross or stay inside the parts: • The web apps run in LINE's in-app browser (LIFF, ADR-01) and in ordinary browsers, which speak HTTP and JSON; the **Payment Gateway** reports payment results by an HTTPS webhook. • Inside the Backend the services call each other on the busiest paths: holding a table calls the Concert Round Service and the Table Availability Service within one customer request, which must answer within 2 seconds at the 95th percentile (NFR-04) at 5 hold requests per second (NFR-01). • The backend is written in JavaScript (ADR-03), which is dynamically typed, so nothing checks that a caller and a service agree on the shape of their messages. Options considered: REST everywhere; gRPC everywhere, with gRPC-Web for the browsers; REST into the API Gateway and through it to the services, with gRPC only between the services; REST at the API Gateway only, which translates each call into gRPC; asynchronous messages through a message broker.
Decision	• **Frontend to Backend:** REST, JSON over HTTPS, only through the API Gateway, which authenticates each request and routes it to the service that owns the operation. • **External systems to Backend:** the payment webhook is a REST call over HTTPS to the API Gateway, which passes it to the Payment Service. • **API Gateway to services:** gRPC. The API Gateway is the only component that speaks REST: it maps each of its routes to one gRPC method of the service that owns the operation, turns the path, the query and the JSON body into the request message, the response message into JSON and the gRPC status into the HTTP status (Section 6.7). It holds no business logic. • **Service to service:** gRPC, Protocol Buffers over HTTP/2, with @grpc/grpc-js. Every service has exactly one API, described by one .proto file, from which the API Gateway and the other services generate their clients; every collaboration between services in Table 5.3 is a gRPC call. Every call carries a deadline, and only calls that are safe to repeat, such as releaseHold() (ADR-08), are retried. • **Backend to External systems:** through the adapters, over the provider's HTTPS API. • **No message broker in the MVP:** every call is synchronous.
Status	Accepted on 2026-09-30.
Consequences	• The browsers, the LIFF app and the payment webhook use plain HTTPS and JSON, so no gRPC-Web proxy is needed.

	• Each service has one API and one contract, its .proto file, which generates the client and server code and catches the type mismatches that ADR-03 warns about when a service changes its messages, in the API Gateway as in every other caller. • The calls between services use a compact binary encoding over long-lived HTTP/2 connections, which keeps the internal calls of a **hold** small within the 2-second budget of NFR-04. • The API Gateway is the REST service of the system: it offers the create, read, update and delete of **zone maps** and **concert rounds** over REST, served by the Concert Round Service over gRPC, and every service is a gRPC service, so the design contains the REST service and the gRPC service that the course project requires. • The API Gateway grows by one route per operation: the route table and the mapping of gRPC statuses to HTTP statuses live in one place, and the polled read of the table map (ADR-09) answers 304 at the gateway, which compares the version of the read model with the If-None-Match header of the request. The team has to learn gRPC and Protocol Buffers. • gRPC messages are binary and harder to inspect than JSON; debugging needs tools such as grpcurl and server reflection. • Every call is synchronous: when a service is down, the calls to it fail and so does the request that made them. Calls that can be delayed without harm, such as the LINE messages, are the first candidates for asynchronous messaging when a later ADR revisits it. • The services find each other by service names that the deployment resolves; how services are discovered is left to a later ADR.

4.13 ADR-13: The Booking Owns the Hold; the Table Map Is a Read Model

Table 4.14 ADR-13 The Booking Owns the Hold; the Table Map Is a Read Model

Title	The Booking Owns the **Hold**; the Table Map Is a Read Model
Context	ADR-08 put the lock in the Table Availability Service: holdTable() is a conditional update of the table's status, and the Booking Service writes its own Held record afterwards. One business fact, a table taken by a customer, is therefore written twice, in two databases, and kept consistent by retries and an idempotent releaseHold(). The domain model (Section 6.1) shows that the booking history already contains every fact the table map shows: a table is unavailable exactly when an active booking, Held, Confirmed or Checked-in, exists for it in that round. An audit trail of booking state changes is wanted for disputes at the door, and the minimum requirements of the course project include a service fed through a message broker. Options considered: keep the lock in the Table Availability Service (ADR-08); make the booking the single truth with a uniqueness rule and derive the table map as a read model, fed synchronously now and by events later; full event sourcing with an event store and state rebuilt by replay.
Decision	The Booking Service owns the **hold**. A booking is created as Held by one insert that a unique partial index on (round, table, active status) in the Booking DB accepts for exactly one caller, which is **first lock wins** (BRULE-03, FR-08); every later transition is appended to the booking's history. The Table Availability Service keeps the read model of the table map, one document per round: createRoundTableStatus() creates it from the round when the round is published, and the Booking Service updates it after each transition with holdTable(), releaseHold(), markTableBooked() and markTableOccupied(). The read model never decides who gets a table. In the MVP the updates are synchronous gRPC calls; from Increment 2 they are events on a message broker that the Table Availability Service consumes. There is no event store: a booking stores its current state and its history.
Status	Accepted on 2026-09-30. Supersedes the locking part of ADR-08; the 5-second hold-expiry job of ADR-08 stays.
Consequences	• One source of truth: the invariant of BRULE-03 is a database constraint, tested directly under the concurrent holds of NFR-20, and no lock crosses a service boundary. • Every booking carries its own audit trail, and the table map can be rebuilt at any time from the bookings and the round. • The message broker of Increment 2 gets a real consumer, the read model, instead of being added for its own sake. • The map lags a booking change by one call in the MVP, or one event later; the customer who holds sees the result in the same request, other customers within the 2-second poll of ADR-09. • A failed update leaves the map stale until the Booking Service retries it; the updates are idempotent, and a stale map only costs a customer a refused hold (UC-01 AF-3), never a double booking. • The polled read stays with the Table Availability Service, so the Booking DB is not hit by the map polling.

4.14 ADR-14: Modular Monolith Mode for Development and Tests

Table 4.15 ADR-14 Modular Monolith Mode for Development and Tests

Title	Modular Monolith Mode for Development and Tests
Context	The MVP runs as seven processes, the API Gateway and six services, which speak gRPC (ADR-12). Running or debugging one business flow, a **hold** or the publication of a **concert round**, means seven processes, their ports and their logs, and an end-to-end test needs all of them up. The rules of a service live in its domain module, which reaches the other services only through small client objects (Section 6.2, step 4), and the unit tests already replace those objects with stubs; what the unit tests cannot cover is a flow across services. The course requires the microservices, and the team wants to test the logic before scaling them out. Options considered: the flows tested only by hand against the seven processes; docker compose for every test; an in-process mode from the same code.
Decision	A modular monolith mode for development and tests, beside the microservices, from the same code: • Every service keeps an API layer, one function per method of its .proto file that takes the request message and returns the response message with the caller passed explicitly; its gRPC server is a thin wrapper around it. • In monolith mode one process loads the API Gateway and the six services, and one composition root, the only module that knows every service, points the client objects of the gateway and of the services at the API layers of the other services. No service imports another service. • Every call still passes its request and its response through the Protocol Buffers serializer and deserializer of the method, in memory, so the contract, the defaults and the optional fields behave exactly as over gRPC; only the network is gone. Each service keeps its own store; nothing reads another service's data. • The end-to-end tests of the two Deliverable #3 flows run in this mode under the unit-test command; the smoke test against the seven processes stays. • The mode is never the deployment target: the MVP is deployed as the microservices of Section 5.
Status	Accepted on 2026-09-30.
Consequences	• A flow is debugged in one process with one log and one stack trace, and an end-to-end test runs in milliseconds without ports or containers, on every change. • The API layer is one more file per service, and the boundary between transport and rules is explicit: the gRPC server and the in-process client wrap the same functions. • In-process calls carry no deadline and never fail with UNAVAILABLE, so the timeouts, the retries and the failure handling of ADR-08 and ADR-12 are exercised only over gRPC; the smoke test against the seven processes therefore stays the check before a release. • The composition root is the one place that may depend on every service; that no service imports another is what keeps the split real. The caller's identity travels as a call context built by the same gateway code that fills the gRPC metadata, so the role checks hold in both modes.

5 Microservice Design

5.1 Scope of the Microservice Design

This section presents the second version of the microservice architecture for the Seating & Event Availability Tracking System (SEATS). It defines the parts of the system and how they communicate (Section 5.2), lists the services with their operations and collaborators (Section 5.3) and draws the architecture (Section 5.4); Appendix A traces every use case to the operations that carry it out. The section describes the MVP of Section 1.3 only: the operations, collaborations and components that Increment 2 and later increments add are not shown, and they are added to this section when those increments are designed.

The architecture covers the business use cases end to end, including the alternative and exception flows that the MVP builds:

- **UC-01 Reserve a Specific Table** — the **Customer** browses **concert rounds**, holds a table for 15 minutes, completes the **customer profile** (included UC-09 Maintain Customer Profile), accepts the terms, pays the **full table fee** through the simulated **Payment Gateway** (included UC-10 Pay the Full Table Fee, ADR-11) and receives a QR **e-ticket** by LINE.
- **UC-02 Check In with E-Ticket** — **Front Staff** scans the **e-ticket** at the door, the **booking reference** is verified against the **check-in window** and the **grace period**, the entry is confirmed and the table becomes occupied on the **live view**.
- **UC-03 Create Concert Round** — the **Manager** creates a **concert round** on the venue **zone map**, prices the **table types**, validates and publishes it, which makes the round bookable in UC-01 and defines the **check-in window** used in UC-02.
- **UC-04 Create Venue Zone Map** — the **Manager** uploads the image of the venue, defines and names venue **zones** on it, places tables with unique table numbers, assigns **table types** and seating capacities, validates and activates the map, which makes it selectable in UC-03 and provides the **zone map** used in UC-01 and UC-02 for rounds that use it.

The Deliverable #2 brief asks for at least three business use cases; the design covers all six, and the four reads and settings UC-05 to UC-08. The venue **zone map** is owned by the same service that owns the rounds, because a round cannot be created without it. Which tables of a round are booked is read from the Table Availability Service, which keeps the read model of every table's status, so the Concert Round Service does not depend on the Booking Service. The **back-office** accounts of ADR-07 are owned by the Staff Account Service.

The services follow the bounded contexts of the domain, and each is named by the glossary term for what it owns:

- **Concert Round Service** — the venue and events context: **zone maps**, **table types**, **concert rounds**, prices and **business parameters**. A supporting subdomain.
- **Booking Service** — the booking context: the booking from Held to Checked-in, with the **customer profile** and the **e-ticket** as parts of the same aggregate, and the rule that one active booking exists per table per round (**first lock wins**, ADR-13). A core subdomain, since double bookings are the problem the system solves.
- **Table Availability Service** — the table availability context: the read model of the table map, the status of every table of every round as the bookings change it (ADR-13), polled by every open map. A core subdomain, kept apart for its load.
- **Payment Service**, **Notification Service** and **Staff Account Service** — the generic subdomains of payment, messaging and staff identity, each behind an external provider or a standard mechanism, and each changing for its own reasons.

Table Availability and Booking stay two services although every booking state change has a matching table status change: the booking is the truth and the table map its read model (ADR-13), created by the Concert Round Service when a round is published, updated by the Booking Service after each transition, and read by every open map.

The architecture diagram of Section 5.4 (Figure 5.1) is drawn in the ports-and-adapters style: every service is a hexagon whose business logic is reached only through its one API, a gRPC tab drawn on each side from which the API Gateway or another service calls it, and that reaches the external systems only through adapters. The API Gateway carries the only REST tab: it is the only API that the web apps and the payment webhook use, and it turns each call into a gRPC call (ADR-12). An arrow A → B means A invokes B; response paths are not drawn. The three dashed boundaries are the parts of Section 5.2. The hosted checkout page of the **Payment Gateway** is opened by the customer's browser inside the web app and is therefore not shown as a service call. The web apps read the table status by polling through the API Gateway (ADR-09), and the **Payment Gateway** is the simulated gateway of ADR-11.

5.2 Parts of the System and Communication

SEATS is divided into three parts: the Frontend, the Backend and the External systems. Figure 5.1 draws their boundaries, and each ADR names the part it concerns (Table 4.1). The Frontend runs in the phones and browsers

of the **Customers**, the **Front Staff**, the **Manager** and the **Owner**; it is not trusted, and every request it sends is authenticated and checked in the Backend. The Backend runs on the servers that run SEATS; the API Gateway is the only component reachable from the internet, and the services with their databases sit on a private network behind it. The External systems are run by third parties, outside the control of SEATS, and are reached only through adapters, which are modules inside the component that uses them; the simulated gateway of the MVP (ADR-11) is built by the group but deployed and treated as an external system, so that the real gateway can replace it. Table 5.1 lists the components of each part, and Table 5.2 how they communicate (ADR-12).

Table 5.1 Parts of the system and their components

Part	Component	Responsibility	API it offers	Data it owns
Frontend	Customer Web App	The LIFF app inside LINE: **concert rounds**, the **zone map**, the **hold**, the **customer profile**, the **booking terms**, the hosted checkout and My Bookings (UC-01)	None; it calls the API Gateway by REST	None
	Back-office Web App	**Zone maps**, **concert rounds** and **business parameters** for the **Manager**, the **live view** for the **Manager** and the **Owner**, the QR scan and check-in for the **Front Staff** (UC-02 to UC-04)	None; it calls the API Gateway by REST	None
Backend	API Gateway	The only public entry: authenticates the caller (the LINE ID token through the LINE Login Adapter, the staff session token), checks the role of the route (FR-66) and routes the request to the service that owns the operation	REST, the only REST API: for the Frontend and for the payment webhook, each route mapped to one gRPC method	None
	Concert Round Service	The venue and events context: **zone maps**, **table types**, **concert rounds**, **package prices** and **business parameters**; stores the image of the venue through the Media Storage Adapter	gRPC: the **back-office** and customer routes of the API Gateway, the reads of the Booking Service	Round DB
	Table Availability Service	The read model of the table map: the status of every table of every round, kept from the booking changes (ADR-13), and the sold-out counts	gRPC: every change and read, including the polled read of the map that the API Gateway serves (ADR-09)	Table Status DB
	Booking Service	The booking from Held to Checked-in, one active booking per table per round (**first lock wins**, ADR-13), the **customer profile**, the **booking terms**, the **e-ticket**, the check-in and the **live view**; its own job expires unpaid **holds**	gRPC: the customer and staff routes of the API Gateway, the payment confirmation of the Payment Service	Booking DB

Part	Component	Responsibility	API it offers	Data it owns
	Payment Service	Payment requests and results, reached through the Payment Gateway Adapter (ADR-11)	gRPC: the payment request, the webhook result passed on by the API Gateway, the payment status	Payment DB
	Notification Service	LINE messages to the **Customer** through the LINE Messaging Adapter (ADR-10); its own job retries failed messages	gRPC	Notification DB
	Staff Account Service	**Back-office** accounts, roles and sign-in (ADR-07)	gRPC: sign-in and the staff accounts, through the API Gateway	Staff Account DB
External systems	**LINE Platform**	LINE Login and LIFF for the customer's identity, the Messaging API for push messages	The provider's API	—
	Payment Gateway	The hosted checkout, the signed payment results and, from Increment 2, refunds; simulated in the MVP	The provider's API and its webhook	—
	Cloud Object Storage	The image of the venue behind each **zone map**	The provider's API	—

Table 5.2 Communication between and inside the parts (ADR-12)

From	To	How	Used for
Frontend	API Gateway	REST: JSON over HTTPS	Every operation that an actor invokes (Appendix A)
Payment Gateway	API Gateway	REST: signed webhook over HTTPS	Payment results
API Gateway	Services	gRPC: Protocol Buffers over HTTP/2 on the private network	Each REST route is one gRPC method of the service that owns the operation (Section 6.7)
Service	Service	gRPC: Protocol Buffers over HTTP/2	Every collaboration between services in Table 5.3
Service or API Gateway	Its adapters	Call inside the same process	Reaching an external system
Adapter	External system	The provider's HTTPS API	LINE ID token check, LINE push messages, checkout and payment results, storing the image of the venue
Frontend	External systems	The provider's own SDK or page	LINE Login in the LIFF app; the hosted checkout of the **Payment Gateway**
Service	Its database	Database driver (MongoDB, ADR-06)	A service reads and writes only its own database

Every service has one API, gRPC, described by its .proto file, and the API Gateway is the only REST API of the system: it turns each route into one gRPC call on the owning service (ADR-12, Section 6.7). The API Gateway authenticates every request before routing it: a **Customer** by the LINE ID token, checked through the LINE Login Adapter, and a member of staff by the session token issued by the Staff Account Service (ADR-07), whose role decides which operations the request may reach (FR-66). The payment webhook is the exception: the gateway passes it on, and the Payment Service verifies its signature (NFR-38).

5.3 Service–Operations–Collaborators

Operations are the business operations that a service exposes as gRPC methods, to the web apps through a route of the API Gateway or to other services; each operation is one function, so creating and updating a record are two operations, and Sections 6.3 to 6.7 give their methods, messages and routes. What a service does on its own timer, or as a step inside another operation, is not an operation and is described below the table. Collaborators are the services or adapters that the service invokes to complete its own operations; an em dash means the service completes its work without calling anyone else. A call to another service is a gRPC call, and a call to an adapter stays inside the service (Section 5.2).

Note: the table lists only what the MVP builds. The operations and collaborators that Increment 2 and later increments add (Section 1.3) are not written here.

Two services also run a job on their own timer, which is not an operation: the hold-expiry job of the Booking Service sets every overdue Held booking to Expired every 5 seconds and calls releaseHold() and sendHoldExpiredNotice() (ADR-08, UC-01 EF-1), and the retry job of the Notification Service resends a failed message 3 times within 5 minutes (FR-22, UC-01 EF-3). Two steps are part of a larger operation: setPartySize() computes the **full table fee** (UC-01 steps 10–11), and confirmBookingPayment() issues the **e-ticket** (UC-01 steps 16–17). A Published round is changed with updateRound(), which then allows only the changes of UC-03 AF-3 (BRULE-07), and the preview of a round (UC-03 step 13) is the customer's own view, getRound() and getRoundTables().

Table 5.3 Service–Operations–Collaborators of the MVP

Service	Operations	Collaborators
Concert Round Service	createZoneMap() updateZoneMap() uploadZoneMapImage() defineTableType() listTableTypes() listZoneMaps() getZoneMap() validateZoneMap() activateZoneMap() getBusinessParameters() updateBusinessParameters() createRound() updateRound() validateRound() publishRound() getUpcomingRounds() getRound() getRoundTables() getRoundPricing() getCheckInWindow()	**Table Availability Service** createRoundTableStatus() getRoundTableStatus() countAvailableTables() **Media Storage Adapter** storeZoneMapImage()
Table Availability Service	createRoundTableStatus() getRoundTableStatus() countAvailableTables() holdTable() releaseHold() markTableBooked() markTableOccupied()	—

Service	Operations	Collaborators
Booking Service	createHeldBooking() getBooking() setPartySize() getCustomerProfile() createCustomerProfile() updateCustomerProfile() getBookingTerms() acceptBookingTerms() startPayment() confirmBookingPayment() getETicket() getCustomerBookings() cancelBooking() verifyBookingReference() checkInBooking() getRoundBookings()	**Concert Round Service** getRound() getRoundPricing() getCheckInWindow() **Table Availability Service** holdTable() releaseHold() markTableBooked() markTableOccupied() **Payment Service** createPaymentRequest() **Notification Service** sendBookingConfirmation() sendHoldExpiredNotice()
Payment Service	createPaymentRequest() receivePaymentResult() getPaymentStatus()	**Payment Gateway Adapter** createCheckoutSession() verifyWebhookSignature() **Booking Service** confirmBookingPayment() **Notification Service** sendPaymentFailedNotice()
Notification Service	sendBookingConfirmation() sendHoldExpiredNotice() sendPaymentFailedNotice()	**LINE Messaging Adapter** pushLineMessage()
Staff Account Service	signIn() signOut() createStaffAccount() listStaffAccounts() updateStaffAccount() disableStaffAccount()	—

5.4 Architecture Diagram

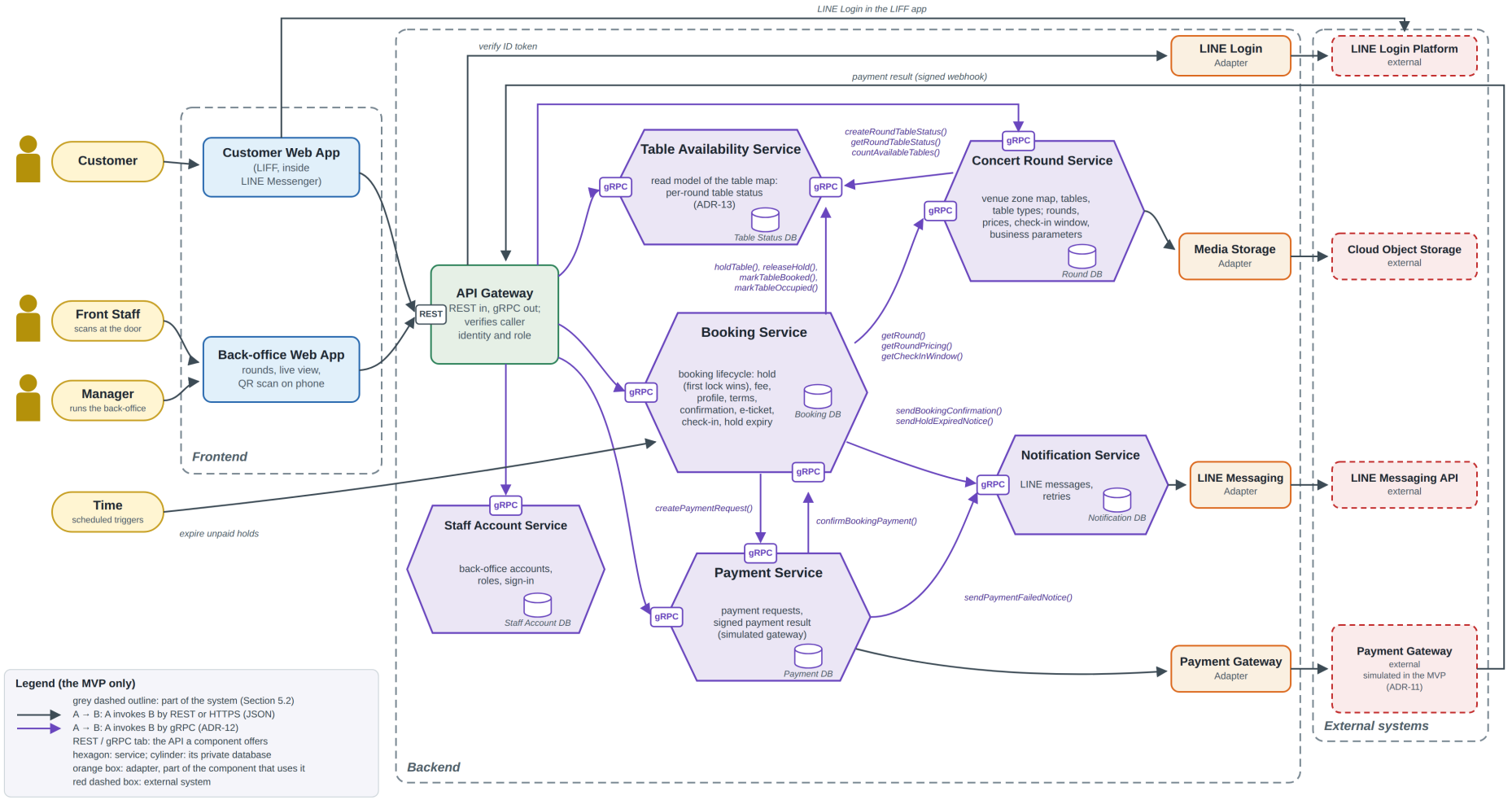


Figure 5.1 SEATS microservice architecture, version 2

6 Domain Model and API Specification

This chapter is the bridge between the business and the code. It states the domain model that every part of the document shares (6.1) and the rule by which a model becomes a contract (6.2). Then one section per service gives what its implementer must hold and serve: the slice of the model in its own database, its gRPC API and the fields of its messages (6.3 to 6.6). The last section is the API Gateway, the public REST API, one route per gRPC method (6.7). Deliverable #3 demonstrates these contracts: REST with create, read, update and delete on the routes of the Concert Round Service, and gRPC with the same four on the Table Availability Service. In every table a field marked **?** is optional and, in an update, unchanged when left out. The contracts themselves live in the code repository, cuse-software-engineering/SEATS: one `.proto` file per service under `proto/` and the OpenAPI document of the gateway, `docs/openapi.yaml`; each section links to its file.

6.1 Domain Model

The domain model is the conceptual model of the concert-table business: the things that exist in the domain, their attributes, how they relate, and the rules that constrain them. It is the shared vocabulary behind the use cases (Section 2), the requirements (Section 3), the glossary (Appendix C) and the services of Section 5: every entity is a glossary term, every invariant is a business rule of Appendix B, and the booking lifecycle supplies the alternative and exception flows of UC-01 and UC-02. Actors (**Customer**, **Manager**, **Front Staff**, **Owner**) are not entities; the **Customer** entity is the **customer profile** that a LINE account owns. The model is drawn without instance data: the current **zones**, **table types**, **package prices** and the **extra-person fee** are maintained by the **Manager** and stated as BRULE-08 and BRULE-09, not as values on the diagram.

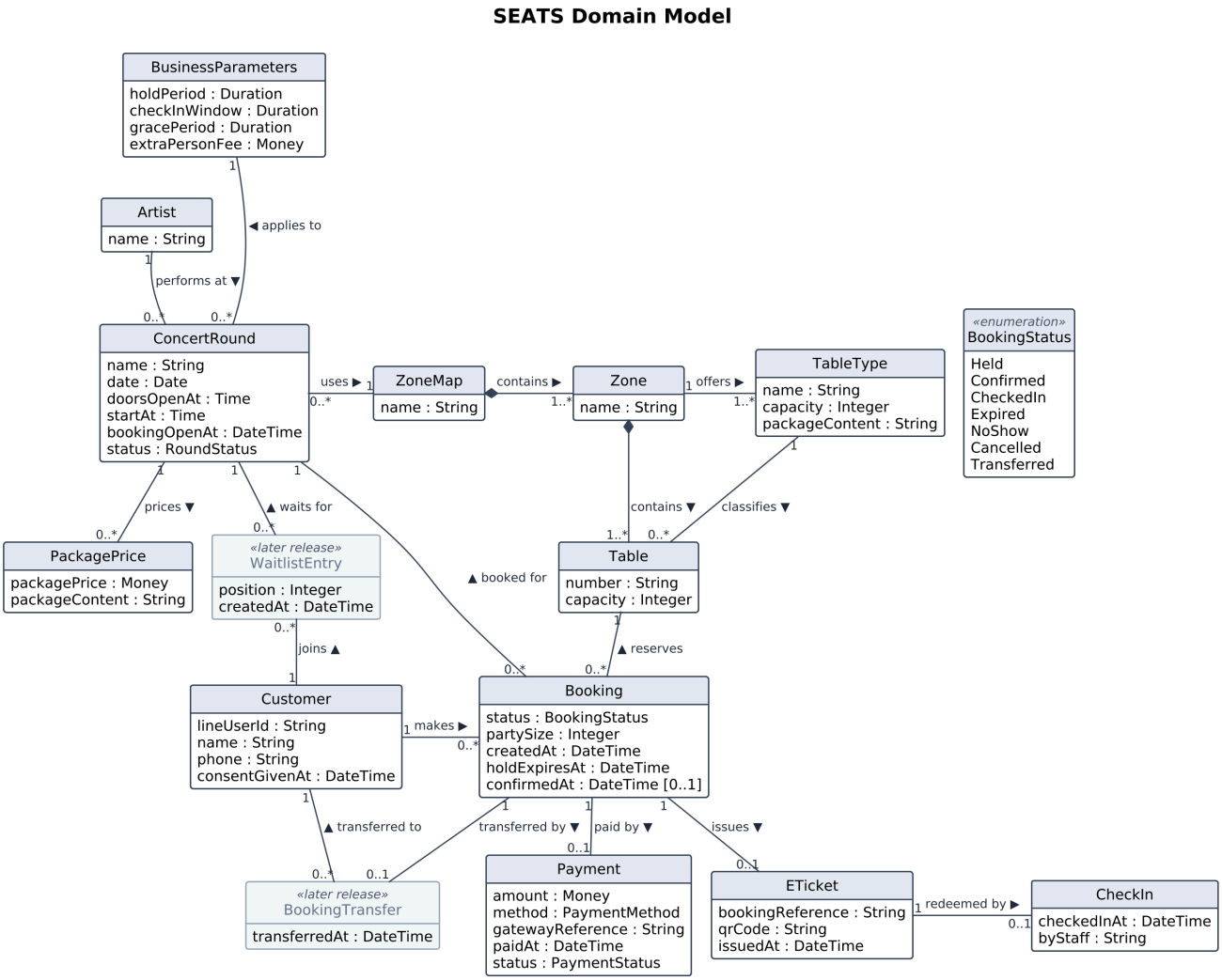


Figure 6.1 Domain model of SEATS (UML class diagram); WaitlistEntry and BookingTransfer belong to a later release

Table 6.1 Business entities

Entity	Definition	Key attributes	Glossary term
ConcertRound	One concert night at the venue, for which tables are sold in advance	name, date, doors-open time, start time, **booking-open time**, status (Draft, Published; then not yet open, open, sold out, finished, cancelled as seen by the **Customer**)	**concert round**
Artist	The performer of a concert night	name	—
ZoneMap	The plan of the venue; one map serves many rounds, and the map of a round is fixed once its booking opens (BRULE-07)	name, image of the venue	**zone map**
Zone	A pricing area of the **zone map**, defined by the **Manager** per map	name	**zone**

Entity	Definition	Key attributes	Glossary term
Table	A physical, numbered table inside a **zone**	number, capacity, position on the image	table
TableType	The kind of table with its **package**, defined by the **Manager**; the same type may have a different price in each **zone**	name, capacity, package content	**table type**, **package**
PackagePrice	The **package price** of one **table type** in one **zone** for one round (BRULE-08)	price, package content	**package price**
BusinessParameters	The venue's settings; a round uses the values in force at its **booking-open time** (FR-38)	**hold period**, **check-in window**, **grace period**, **extra-person fee**	**business parameters**
Customer	A person identified by a LINE account who reserves tables (BRULE-12)	LINE user id, name, phone, consent given at	**Customer**, **customer profile**
Booking	The reservation of one table for one round by one customer; the central record of the system	status (BookingStatus, Figure 6.2), **party size**, created at, **hold** expires at, confirmed at	booking, **hold**
Payment	The payment of the **full table fee** for a booking through the **Payment Gateway**	amount, method, gateway reference, paid at, status	**full table fee**
ETicket	The proof of a Confirmed booking, presented as a QR code at the door	**booking reference** (signed), QR code, issued at, valid for (round, table)	**e-ticket**
CheckIn	The event of admitting the **Customer** at the door	checked-in at, by staff, guests present	check-in
WaitlistEntry	A **Customer**'s request to be offered a table of a full round; later release	position, created at	**waitlist**
BookingTransfer	The official reassignment of a Confirmed booking to another **Customer**; later release	transferred at, to customer	—

The invariants of the model are the business rules of Appendix B: a table has at most one active booking (Held, Confirmed or Checked-in) per round (BRULE-03, **first lock wins**); a booking is Confirmed only after the **full table fee** is verified (BRULE-01); a Held booking expires after the **hold period** (BRULE-02); check-in is accepted within the **check-in window** and the **grace period**, after which the booking is a **no-show** (BRULE-04, BRULE-05, BRULE-06); the **zone map** of a round does not change once booking is open (BRULE-07); and a customer is identified by exactly one LINE account (BRULE-12).

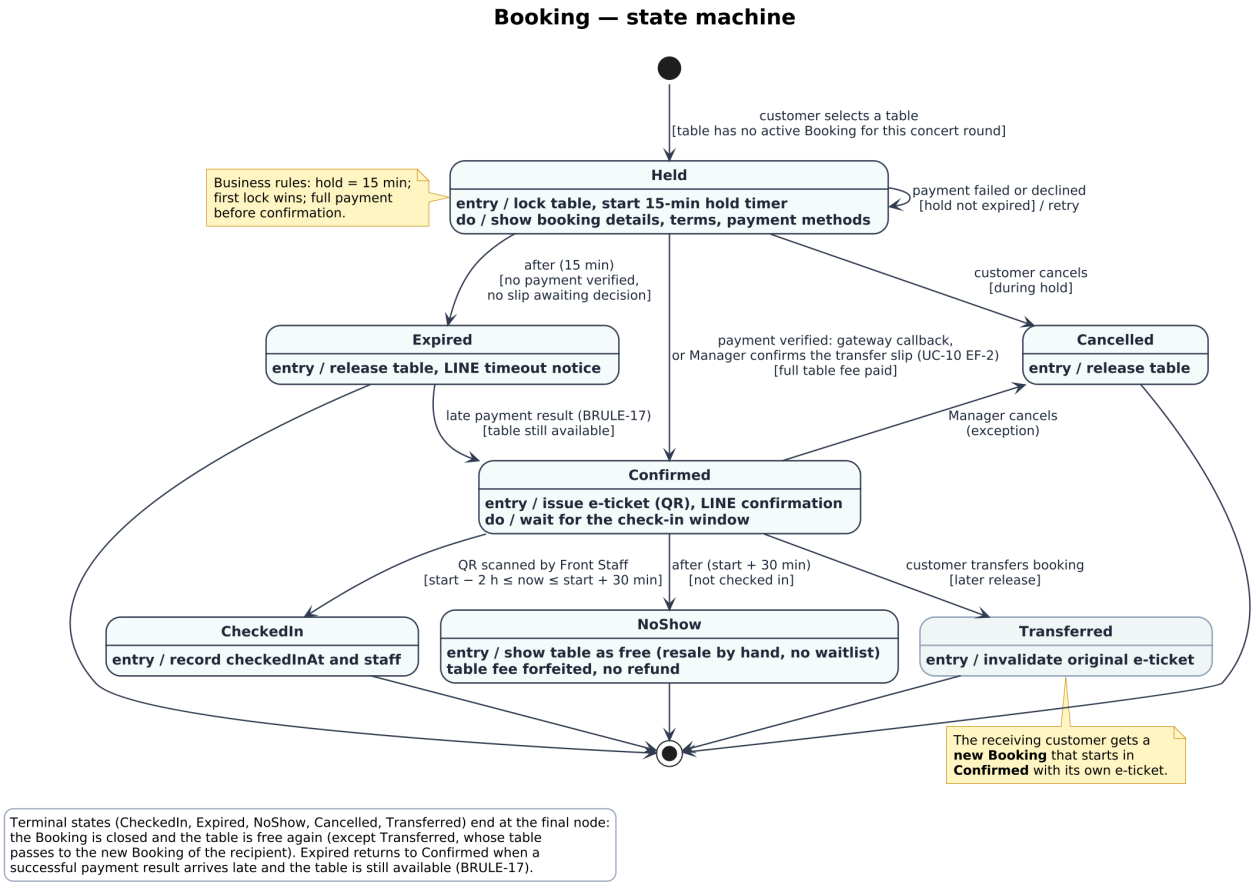


Figure 6.2 Lifecycle of a booking (UML state machine); Transferred belongs to a later release

Table 6.2 Booking states

State	Entered when	Leaves when
Held	the **Customer** selects a free table (UC-01 step 7)	the payment is verified (UC-10 step 6), or the **Manager** confirms the **transfer slip** in **degraded mode** (UC-10 EF-2, Increment 2) → Confirmed; the **hold period** ends with no payment verified → Expired (EF-1); the **Customer** cancels or the profile is not completed → Cancelled (AF-4, AF-5)
Confirmed	the payment result is verified and the **e-ticket** issued (UC-01 steps 16–17)	the **e-ticket** is scanned within the **check-in window** → Checked-in (UC-02); the **grace period** passes → **No-show** (UC-02 EF-4, Increment 2); the **Manager** cancels as an exception → Cancelled
Checked-in	**Front Staff** confirms the entry (UC-02 step 6)	final
Expired	the **hold** ends with no payment verified; the table is released and the **Customer** notified (UC-01 EF-1)	a successful payment result arrives late while the table is still available → Confirmed (UC-10 EF-1, BRULE-17, Increment 2); otherwise final

State	Entered when	Leaves when
No-show	the **grace period** ends without a check-in; the table is shown as free, the fee is forfeited (BRULE-06)	final
Cancelled	the **Customer** cancels during the **hold**, or the **Manager** cancels as an exception	final
Transferred	later release: the booking is reassigned and its **e-ticket** invalidated	final

6.2 From Model to Contract

The types of the code, the gRPC messages and the routes of the API Gateway are all derived from the model, in that order, and a change is made in that order too:

Table 6.3 From the domain model to the contracts

Step	Artifact	What it holds	Example: the booking
1	Domain model (6.1)	The business entity, its attributes and its rules	Booking: status, **party size**, **hold** end; one active booking per table per round (BRULE-03)
2	Data model of the owning service (6.3 to 6.5)	The entity as stored, with the copies and references the service needs	Booking DB: `zoneId`, `tableTypeId` and `capacity` copied from the round; `history` of state changes; unique index on (`roundId`, `tableNumber`, active)
3	Type in the service (`model.ts`)	The stored shape, used by the operations and the store	`Booking`, `BookingStatus`, `Fee`, `CustomerProfile`
4	gRPC message (`.proto`)	Only what a caller needs, the API Gateway or another service: a projection of the model, never the stored document	`Booking` carries the fee and the remaining hold seconds; `HoldTableRequest` carries only `round_id`, `table_number`, `booking_id`, `hold_ends_at`, nothing of the customer
5	Route of the API Gateway (6.7)	The method and path the web app calls, mapped one to one onto a gRPC method: the JSON body is the request message and the JSON answer the response message	`POST /bookings {roundId, tableNumber}` is CreateHeldBooking and answers the booking with `remainingHoldSeconds`; the unique index and the store stay inside the service

Two consequences of the rule shape the design. First, a gRPC message is owned by the service that serves it: `concert_round.proto`, `table_availability.proto` and `booking.proto` are the contracts of the three services, and a change to a message is a compile error in every caller, the API Gateway included.

Second, the table map is a read model and not a second truth: the Booking Service decides who holds a table, and the Table Availability Service keeps the projection that the web apps poll (ADR-13). The projection is created from the round when it is published (createRoundTableStatus()), because which tables exist and which are not for sale come from the round, not from any booking; from then on every booking transition updates it, by a gRPC call in the MVP and by an event on the message broker from Increment 2.

6.3 Concert Round Service

Each service owns one database and one slice of the domain model (ADR-06, Table 5.1); a service never reads another service's database, so a `roundId` in the Booking DB is a reference by identifier, not a foreign key, and what a service needs from another it asks by gRPC. A message named after an entity of the service's data model carries the stored fields plus what its row lists.

The venue and events context: **zone maps**, **table types**, **concert rounds**, **package prices** and **business parameters**, in the Round DB. Its gRPC API (port 5001, concert_round.proto) is called by the API Gateway for the **back-office** and customer routes that Table 6.12 maps onto it, and by the Booking Service for GetRound, GetRoundPricing and GetCheckInWindow; it calls the Table Availability Service when a round is published and for the sold-out counts, and the Media Storage Adapter for the image of the venue.

Round DB — Concert Round Service

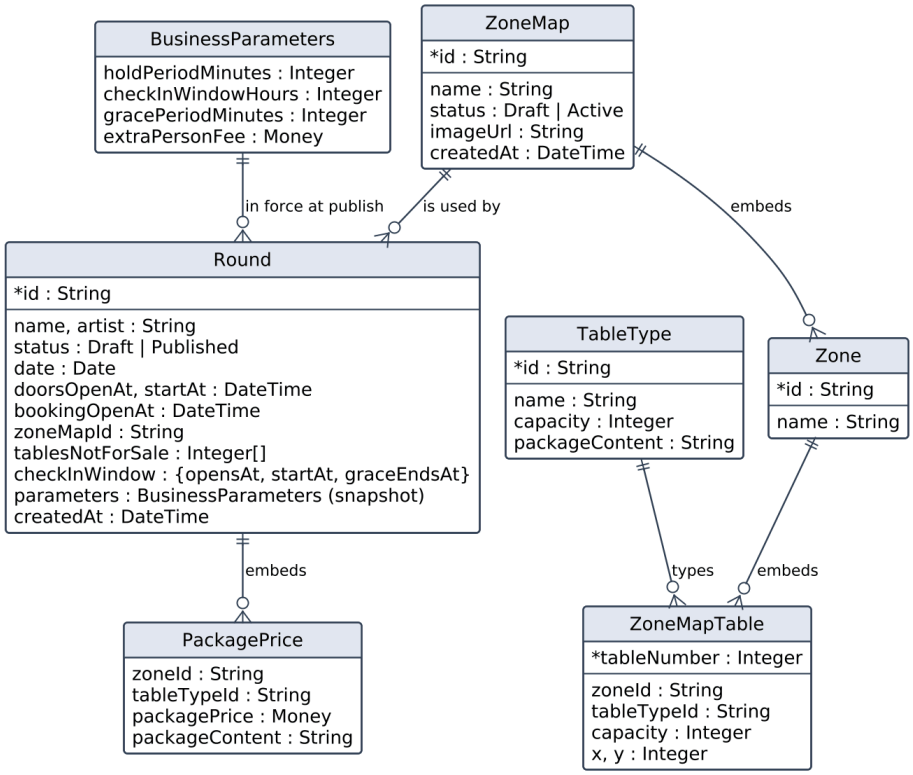


Figure 6.3 Round DB of the Concert Round Service

A **zone map** is one document with its **zones** and tables embedded; a round embeds its **package prices**, its **check-in window** and the snapshot of the **business parameters** in force when it was published (FR-38). The tables of a round are the tables of its map minus those not for sale.

Table 6.4 Concert Round Service, gRPC API (port 5001)

Operation	gRPC method	Request message	Response message
defineTableType()	DefineTableType	`DefineTableTypeRequest {id, name, capacity, package_content}`	`TableType`
listTableTypes()	ListTableTypes	`Empty`	`TableTypeList {table_types[]}`
getBusinessParameters()	GetBusinessParameters	`Empty`	`BusinessParameters`
updateBusinessParameters()	UpdateBusinessParameters	`UpdateBusinessParametersRequest {hold_period_minutes?, check_in_window_hours?, grace_period_minutes?, extra_person_fee?}`	`BusinessParameters`
createZoneMap()	CreateZoneMap	`CreateZoneMapRequest {name?}`	`ZoneMap` (Draft)
listZoneMaps()	ListZoneMaps	`ListZoneMapsRequest {status?}`	`ZoneMapList {zone_maps[]}` of `ZoneMapSummary`
getZoneMap()	GetZoneMap	`ZoneMapRef {zone_map_id}`	`ZoneMap` with `summary[]` per zone
updateZoneMap()	UpdateZoneMap	`UpdateZoneMapRequest {zone_map_id, name?, zones?[], tables?[]}`	`ZoneMap`; on an Active map only the changes of UC-04 AF-1
uploadZoneMapImage()	UploadZoneMapImage	`UploadZoneMapImageRequest {zone_map_id, file_name}`	`ZoneMap` with `image_url`
validateZoneMap()	ValidateZoneMap	`ZoneMapRef`	`ValidationResult`
activateZoneMap()	ActivateZoneMap	`ZoneMapRef`	`ZoneMap` (Active); INVALID_ARGUMENT with the problems when not valid
discardDraftZoneMap()	DiscardDraftZoneMap	`ZoneMapRef`	`Removed`; FAILED_PRECONDITION unless Draft
createRound()	CreateRound	`CreateRoundRequest {name?}`	`Round` (Draft)
getUpcomingRounds()	GetUpcomingRounds	`Empty`	`UpcomingRoundList {rounds[]}` of `UpcomingRound`
getRound()	GetRound	`RoundRef {round_id}`	`Round` with its `tables[]` and `hold_period_minutes`

Operation	gRPC method	Request message	Response message
getRoundTables()	GetRoundTables	`RoundRef`	`RoundTableList {tables[]}` of `RoundTable`
updateRound()	UpdateRound	`UpdateRoundRequest {round_id, name?, artist?, date?, doors_open_at?, start_at?, booking_open_at?, zone_map_id?, tables_not_for_sale?[], prices?[]}`	`Round`; a Published round accepts only the changes of UC-03 AF-3 (BRULE-07)
validateRound()	ValidateRound	`RoundRef`	`ValidationResult`
publishRound()	PublishRound	`RoundRef`	`Round` (Published); calls CreateRoundTableStatus; idempotent
discardDraftRound()	DiscardDraftRound	`RoundRef`	`Removed`; FAILED_PRECONDITION unless Draft
getRoundPricing()	GetRoundPricing	`RoundRef`	`RoundPricing {round_id, prices[], extra_person_fee}`
getCheckInWindow()	GetCheckInWindow	`RoundRef`	`CheckInWindow`

Table 6.5 Messages of the Concert Round Service

Message	Fields
`Round`	The round of Figure 6.3: `id, name, artist, status, date, doors_open_at, start_at, booking_open_at, zone_map_id, tables_not_for_sale[], prices[]` of `PackagePrice`, `check_in_window, parameters, created_at`; plus `hold_period_minutes` in force for the round and `tables[]` of `RoundTable`
`RoundTable`	`table_number, zone_id, zone_name, table_type_id, table_type_name, capacity, x, y, for_sale, package_price?, package_content`: a table of the map joined with its **table type** and **package price**
`UpcomingRound`	`id, name, artist, date, start_at, booking_open_at, status` (not yet open, open, sold out), `available_tables, tables_for_sale`
`ZoneMap`	`id, name, status, image_url, zones[]` of `Zone {id, name}`, `tables[]` of `ZoneMapTable {table_number, zone_id, table_type_id, capacity, x, y}`, `created_at, summary[]` of `ZoneSummary {zone_id, name, tables, capacity}`
`ZoneMapSummary`	`id, name, status, tables`
`TableType`	`id, name, capacity, package_content`
`BusinessParameters`	`hold_period_minutes, check_in_window_hours, grace_period_minutes, extra_person_fee`
`PackagePrice`	`zone_id, table_type_id, package_price, package_content`
`RoundPricing`	`round_id, prices[]` of `PackagePrice`, `extra_person_fee`
`CheckInWindow`	`round_id, opens_at, start_at, grace_ends_at`

Message	Fields
`ValidationResult`, `Removed`	`valid, problems[]`; `removed`

6.4 Table Availability Service

The read model of the table map (ADR-13): the status of every table of every published round, in the Table Status DB. Its gRPC API (port 5003, table_availability.proto) is called by the Concert Round Service (create, count, remove), by the Booking Service after every booking transition (hold, release, booked, occupied) and by the API Gateway for the polled read of the table map (Table 6.12); it calls no one. It is the gRPC service with create, read, update and delete that Deliverable #3 demonstrates.

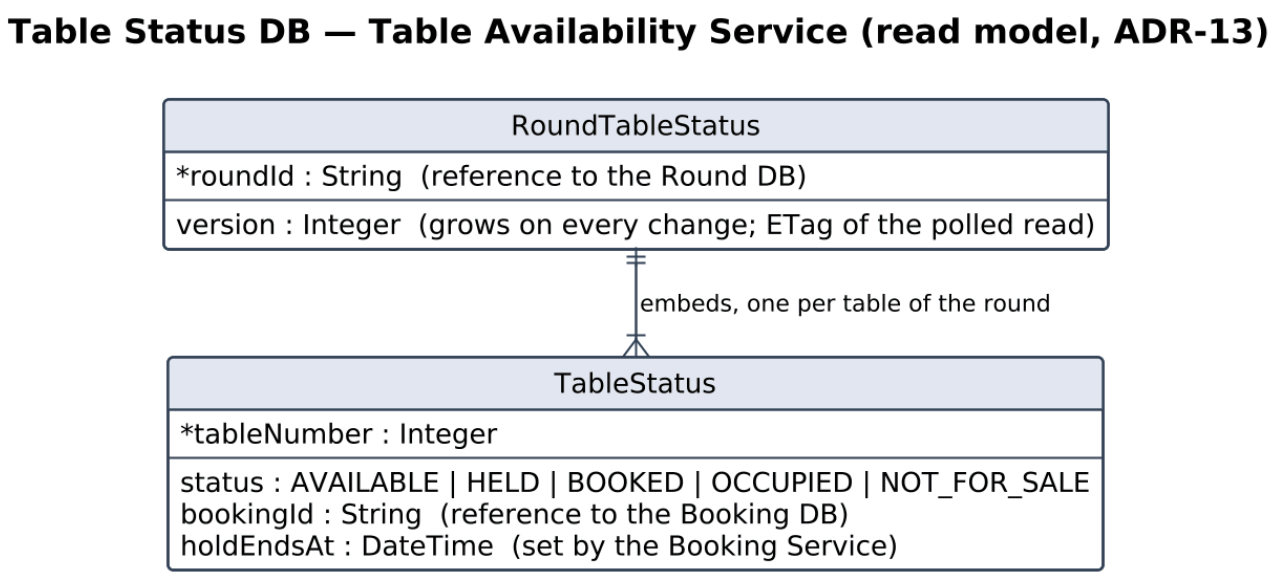


Figure 6.4 Table Status DB of the Table Availability Service

One document per round with its tables embedded: the read model of the table map (ADR-13). Its version grows on every change and is the ETag of the polled read (ADR-09).

Table 6.6 Table Availability Service, gRPC API (port 5003)

Operation	gRPC method	Request message	Response message
createRoundTableStatus()	CreateRoundTableStatus	`CreateRoundTableStatusRequest {round_id, tables[{table_number, for_sale}]}`	`RoundTableStatus`; idempotent on retry
getRoundTableStatus()	GetRoundTableStatus	`RoundRef {round_id}`	`RoundTableStatus`

Operation	gRPC method	Request message	Response message
countAvailableTables()	CountAvailableTables	`CountAvailableTablesRequest {round_ids[]}`	`CountAvailableTablesResponse {counts[{round_id, available, for_sale}]}`
holdTable()	HoldTable	`HoldTableRequest {round_id, table_number, booking_id, hold_ends_at}`	`TableStatus`; AVAILABLE → HELD, otherwise FAILED_PRECONDITION
releaseHold()	ReleaseHold	`TableRef {round_id, table_number, booking_id}`	`TableStatus`; HELD → AVAILABLE, a no-op when already AVAILABLE
markTableBooked()	MarkTableBooked	`TableRef`	`TableStatus`; HELD → BOOKED
markTableOccupied()	MarkTableOccupied	`TableRef`	`TableStatus`; BOOKED → OCCUPIED
removeRoundTableStatus()	RemoveRoundTableStatus	`RoundRef`	`RemoveRoundTableStatusResponse {removed}`; refused while a table is held or booked

Table 6.7 Messages of the Table Availability Service

Message	Fields
`RoundTableStatus`	The document of Figure 6.4: `round_id, version, tables[]` of `TableStatus`
`TableStatus`	`table_number, status` (AVAILABLE, HELD, BOOKED, OCCUPIED, NOT_FOR_SALE), `booking_id, hold_ends_at`

6.5 Booking Service

The booking context: the booking from Held to Checked-in, the **customer profile**, the **booking terms** and the **e-ticket**, in the Booking DB; the booking owns the **hold** (ADR-13). Its gRPC API (port 5002, booking.proto) is called by the API Gateway for the customer and staff routes that Table 6.12 maps onto it, with the caller in the metadata `x-user-id`, and by the Payment Service for ConfirmBookingPayment; it calls the Concert Round Service (GetRound, GetRoundPricing, GetCheckInWindow), the Table Availability Service (HoldTable, ReleaseHold, MarkTableBooked, MarkTableOccupied) and, from progress 2, the Payment and Notification Services. Its own job expires unpaid **holds** (ADR-08).

Booking DB — Booking Service (source of truth of the hold, ADR-13)

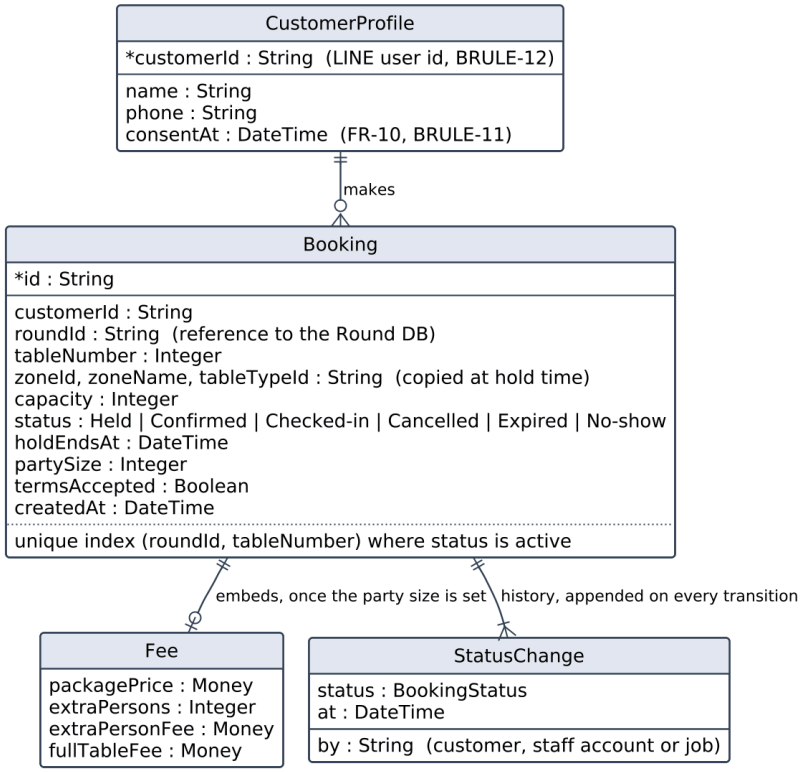


Figure 6.5 Booking DB of the Booking Service

The booking is the source of truth of a table's occupation: at most one active booking (Held, Confirmed, Checked-in) exists per table per round, enforced by a unique index (ADR-13, BRULE-03), and every state change is appended to the booking's history. The booking copies zone, **table type** and capacity from the round when the table is held, so that the fee and the ticket do not change if the map is edited later (BRULE-07).

Table 6.8 Booking Service, gRPC API (port 5002); the caller is the metadata x-user-id

Operation	gRPC method	Request message	Response message
createHeldBooking()	CreateHeldBooking	`CreateHeldBookingRequest {round_id, table_number}`	`Booking` (Held) with `remaining_hold_seconds`; FAILED_PRECONDITION when the table was just taken
getBooking()	GetBooking	`BookingRef {booking_id}`	`Booking`; own bookings only
setPartySize()	SetPartySize	`SetPartySizeRequest {booking_id, party_size}`	`Booking` with `fee`
getCustomerProfile()	GetCustomerProfile	`Empty`	`CustomerProfile`; NOT_FOUND before the first booking
createCustomerProfile()	CreateCustomerProfile	`CustomerProfileRequest {name, phone, consent}`	`CustomerProfile`
updateCustomerProfile()	UpdateCustomerProfile	`CustomerProfileRequest {name?, phone?}`	`CustomerProfile`
getBookingTerms()	GetBookingTerms	`BookingRef`	`BookingTerms`
acceptBookingTerms()	AcceptBookingTerms	`BookingRef`	`Booking` with `terms_accepted`
startPayment()	StartPayment	`BookingRef`	`PaymentRequest`; UNIMPLEMENTED in Increment 1
cancelBooking()	CancelBooking	`BookingRef`	`Booking` (Cancelled)
getCustomerBookings()	GetCustomerBookings	`Empty`	`BookingList {bookings[]}`
getETicket()	GetETicket	`BookingRef`	`ETicket`
verifyBookingReference()	VerifyBookingReference	`BookingReference {booking_reference}`	`VerificationResult`
checkInBooking()	CheckInBooking	`BookingReference`	`Booking` (Checked-in)
getRoundBookings()	GetRoundBookings	`RoundRef {round_id}`	`BookingList` for the **live view**
confirmBookingPayment()	ConfirmBookingPayment	`ConfirmBookingPaymentRequest {booking_id, payment_id, amount}`	`Booking` (Confirmed) with its **e-ticket**; idempotent

Table 6.9 Messages of the Booking Service

Message	Fields
`Booking`	The booking of Figure 6.5: `id, customer_id, round_id, table_number, zone_id, zone_name, table_type_id, capacity, status, hold_ends_at, party_size?, fee, terms_accepted, created_at, history[]` of `StatusChange {status, at, by}`; plus `remaining_hold_seconds`
`Fee`	`package_price, extra_persons, extra_person_fee, full_table_fee`
`CustomerProfile`	`customer_id, name, phone, consent_at`
`BookingTerms`	`booking_id, terms[], check_in_window`

Message	Fields
`ETicket`	`booking_id, booking_reference, qr_payload`
`VerificationResult`, `PaymentRequest`	`valid, booking, reason`; `payment_id, checkout_url`

6.6 Payment, Notification and Staff Account Services

The three generic services of Table 5.1 are built after the Deliverable #3 demo. Their contracts are fixed now (payment.proto, notification.proto, staff_account.proto), so that the Booking Service and the API Gateway are written against them; their databases are drawn when they are built. The Payment Service is the simulated **Payment Gateway** of the MVP (ADR-11), the Notification Service sends through the LINE Messaging Adapter (ADR-10), and the Staff Account Service issues the session tokens of the **back-office** (ADR-07).

Table 6.10 Payment, Notification and Staff Account Services, gRPC APIs (MVP contracts)

Service	Operation	gRPC method	Request message	Response message
Payment (port 5004)	createPaymentRequest()	CreatePaymentRequest	`{booking_id, amount, customer_id}`	`PaymentRequest {payment_id, checkout_url}`
	receivePaymentResult()	ReceivePaymentResult	`{payment_id, status, amount, signature}`, the signed result of the **Payment Gateway**	`{accepted}`; a duplicate is ignored
	getPaymentStatus()	GetPaymentStatus	`{payment_id}`	`PaymentStatus {payment_id, booking_id, status, amount}`
Notification (port 5005)	sendBookingConfirmation(), sendHoldExpiredNotice(), sendPaymentFailedNotice()	SendBookingConfirmation, SendHoldExpiredNotice, SendPaymentFailedNotice	`{customer_id, booking_id, …}`	`{message_id, delivered}`
Staff Account (port 5006)	signIn(), signOut()	SignIn, SignOut	`{username, password}`; `{token}`	`Session {token, role}`; `Empty`
	createStaffAccount(), updateStaffAccount()	CreateStaffAccount, UpdateStaffAccount	`{username, role, password}`; `{staff_account_id, role?, password?}`	`StaffAccount {staff_account_id, username, role, status}`
	listStaffAccounts(), disableStaffAccount()	ListStaffAccounts, DisableStaffAccount	`Empty`; `{staff_account_id}`	`StaffAccountList {accounts[]}`; `StaffAccount` (disabled)

6.7 API Gateway

Every service has one API, gRPC, described by its `.proto` file, and the API Gateway is the only REST API of the system (ADR-12). The two are specified separately because they differ in shape: a gRPC method takes one request message and returns one response message, named as in the `.proto` file with `snake_case` fields, while a route of the gateway is a method and a path whose path parameters, query and JSON body are turned into the request message and whose answer is the response message as JSON, with the same fields in `camelCase` (`round_id` becomes `roundId`) and a list message unwrapped to an array. Not every method has a route: the methods that only another service calls stay private to the Backend. Table 6.11 is the public REST API: every route with the web app of Table 5.1 that calls it, the Customer Web App or the Back-office Web App, the roles that may call it (FR-66), its body, its answer and its status codes; the payment webhook is called by the **Payment Gateway** itself. Table 6.12 maps each route onto the gRPC method of the owning service, which is what the gateway implements. Appendix D maps the screens of the two web apps to these routes. The same routes are kept as an OpenAPI document, docs/openapi.yaml, from which the web apps take their types, and are implemented in gateway/src/routes.ts; the Status column of Table 6.11 comes from the OpenAPI document. Every route with a role may also answer 401 (no identity) and 403 (role), which the column leaves out; every error body is `{error, details?}`.

The gateway authenticates the caller and passes its identity and role to the service as the gRPC metadata `x-user-id` and `x-role`. An error is a gRPC status with a message and, where useful, details, which the gateway maps to an HTTP status and a JSON body `{error, details?}`: INVALID_ARGUMENT to 400 (invalid input), UNAUTHENTICATED to 401 (no identity), PERMISSION_DENIED to 403 (role), NOT_FOUND to 404, FAILED_PRECONDITION to 409 (a rule refuses the change, for example the table was just taken), UNIMPLEMENTED to 501 (not built in this increment) and UNAVAILABLE to 502 (the service or one of its collaborators is down).

Table 6.11 The REST API of the API Gateway (port 4000)

Route	Web app	Roles	Request body	Response	Status
`PUT /table-types/{id}`	Back-office	manager	`{name, capacity, packageContent}`	TableType	200, 400
`GET /table-types`	Back-office	manager, owner	—	[TableType]	200
`GET /business-parameters`	Back-office	manager, owner	—	BusinessParameters	200
`PUT /business-parameters`	Back-office	manager	`{holdPeriodMinutes?, checkInWindowHours?, gracePeriodMinutes?, extraPersonFee?}`	BusinessParameters	200, 400

Route	Web app	Roles	Request body	Response	Status
`POST /zone-maps`	Back-office	manager	`{name?}`	ZoneMap	200
`GET /zone-maps?status=`	Back-office	manager, owner	—	[ZoneMapSummary]	200
`GET /zone-maps/{id}`	Back-office	manager, owner	—	ZoneMap	200, 404
`PUT /zone-maps/{id}`	Back-office	manager	`{name?, zones?, tables?}`	ZoneMap	200, 404, 409
`POST /zone-maps/{id}/image`	Back-office	manager	`{fileName}`	ZoneMap	200, 400, 404
`POST /zone-maps/{id}/validate`	Back-office	manager	—	ValidationResult	200, 404
`POST /zone-maps/{id}/activate`	Back-office	manager	—	ZoneMap	200, 400, 404
`DELETE /zone-maps/{id}`	Back-office	manager	—	Removed	200, 404, 409
`POST /rounds`	Back-office	manager	`{name?}`	Round	200
`GET /rounds`	both	everyone	—	[UpcomingRound]	200
`GET /rounds/{id}`	both	everyone	—	Round	200, 404
`GET /rounds/{id}/tables`	both	everyone	—	[RoundTable]	200, 404
`PUT /rounds/{id}`	Back-office	manager	`{name?, artist?, date?, doorsOpenAt?, startAt?, bookingOpenAt?, zoneMapId?, tablesNotForSale?, prices?}`	Round	200, 400, 404, 409
`POST /rounds/{id}/validate`	Back-office	manager	—	ValidationResult	200, 404
`POST /rounds/{id}/publish`	Back-office	manager	—	Round	200, 400, 404, 502
`DELETE /rounds/{id}`	Back-office	manager	—	Removed	200, 404, 409
`GET /rounds/{id}/table-status`	both	everyone	header `If-None-Match: <version>` (ADR-09)	RoundTableStatus with `ETag: <version>`, or 304	200, 304, 404
`POST /bookings`	Customer	customer	`{roundId, tableNumber}`	Booking	200, 400, 404, 409
`GET /bookings/{id}`	Customer	customer	—	Booking	200, 404
`PUT /bookings/{id}/party-size`	Customer	customer	`{partySize}`	Booking	200, 400, 404, 409
`GET /bookings/{id}/terms`	Customer	customer	—	BookingTerms	200, 404
`POST /bookings/{id}/terms-acceptance`	Customer	customer	—	Booking	200, 404, 409

Route	Web app	Roles	Request body	Response	Status
`POST /bookings/{id}/payment`	Customer	customer	—	PaymentRequest (501 in Increment 1)	200, 404, 409, 501
`POST /bookings/{id}/cancel`	Customer	customer	—	Booking	200, 404, 409
`GET /bookings/{id}/e-ticket`	Customer	customer	—	ETicket	200, 404, 501
`GET /customers/me`	Customer	customer	—	CustomerProfile	200, 404
`POST /customers/me`	Customer	customer	`{name, phone, consent}`	CustomerProfile	200, 400, 409
`PUT /customers/me`	Customer	customer	`{name?, phone?}`	CustomerProfile	200, 400, 404
`GET /customers/me/bookings`	Customer	customer	—	[Booking]	200
`POST /check-ins/verify`	Back-office	front staff, manager	`{bookingReference}`	VerificationResult	200, 501
`POST /check-ins`	Back-office	front staff, manager	`{bookingReference}`	Booking	200, 409, 501
`GET /rounds/{id}/bookings`	Back-office	manager, owner	—	[Booking]	200
`POST /payments/webhook`	**Payment Gateway**	the **Payment Gateway**, by its signature (NFR-38)	the signed result	200	200, 400, 404, 409
`GET /payments/{id}`	Customer	customer	—	PaymentStatus	200, 404
`POST /sessions`	Back-office	anyone	`{username, password}`	Session	200, 401
`DELETE /sessions/current`	Back-office	staff	—	—	200
`POST /staff-accounts`	Back-office	manager	`{username, role, password}`	StaffAccount	200, 400, 409
`GET /staff-accounts`	Back-office	manager, owner	—	[StaffAccount]	200
`PUT /staff-accounts/{id}`	Back-office	manager	`{role?, password?}`	StaffAccount	200, 400, 404
`DELETE /staff-accounts/{id}`	Back-office	manager	—	StaffAccount	200, 404

Table 6.12 Routes of the API Gateway and the gRPC methods behind them

Service	Route	gRPC method
Concert Round	`PUT /table-types/{id}`	DefineTableType
	`GET /table-types`	ListTableTypes
	`GET /business-parameters`	GetBusinessParameters
	`PUT /business-parameters`	UpdateBusinessParameters

Service	Route	gRPC method
	`POST /zone-maps`	CreateZoneMap
	`GET /zone-maps?status=`	ListZoneMaps
	`GET /zone-maps/{id}`	GetZoneMap
	`PUT /zone-maps/{id}`	UpdateZoneMap
	`POST /zone-maps/{id}/image`	UploadZoneMapImage
	`POST /zone-maps/{id}/validate`	ValidateZoneMap
	`POST /zone-maps/{id}/activate`	ActivateZoneMap
	`DELETE /zone-maps/{id}`	DiscardDraftZoneMap
	`POST /rounds`	CreateRound
	`GET /rounds`	GetUpcomingRounds
	`GET /rounds/{id}`	GetRound
	`GET /rounds/{id}/tables`	GetRoundTables
	`PUT /rounds/{id}`	UpdateRound
	`POST /rounds/{id}/validate`	ValidateRound
	`POST /rounds/{id}/publish`	PublishRound
	`DELETE /rounds/{id}`	DiscardDraftRound
Table Availability	`GET /rounds/{id}/table-status`	GetRoundTableStatus
Booking	`POST /bookings`	CreateHeldBooking
	`GET /bookings/{id}`	GetBooking
	`PUT /bookings/{id}/party-size`	SetPartySize
	`GET /bookings/{id}/terms`	GetBookingTerms
	`POST /bookings/{id}/terms-acceptance`	AcceptBookingTerms
	`POST /bookings/{id}/payment`	StartPayment
	`POST /bookings/{id}/cancel`	CancelBooking
	`GET /bookings/{id}/e-ticket`	GetETicket
	`GET /customers/me`	GetCustomerProfile
	`POST /customers/me`	CreateCustomerProfile
	`PUT /customers/me`	UpdateCustomerProfile
	`GET /customers/me/bookings`	GetCustomerBookings
	`POST /check-ins/verify`	VerifyBookingReference
	`POST /check-ins`	CheckInBooking
	`GET /rounds/{id}/bookings`	GetRoundBookings
Payment	`POST /payments/webhook`	ReceivePaymentResult

Service	Route	gRPC method
	`GET /payments/{id}`	GetPaymentStatus
Staff Account	`POST /sessions`	SignIn
	`DELETE /sessions/current`	SignOut
	`POST /staff-accounts`	CreateStaffAccount
	`GET /staff-accounts`	ListStaffAccounts
	`PUT /staff-accounts/{id}`	UpdateStaffAccount
	`DELETE /staff-accounts/{id}`	DisableStaffAccount

Appendix A Use Case Traceability

A.1 Operations by Use Case

Table A.1 traces the ten use cases to the operations of Table 5.3: a cell names the steps and flows of the use case that the operation serves, whether the actor invokes it through the API Gateway or another service invokes it as a collaborator; for UC-05 to UC-08, which have no description, a tick marks the operations they use. Every operation has at least one cell, and the two jobs on a timer are listed below the operations. Section A.2 gives the same trace step by step, with who invokes each operation, the collaborations it needs, the data it stores and the requirements it realises; only the flows that the MVP builds are traced (Section 1.3).

Table A.1 Operations by use case

Service	Operation	UC-01	UC-02	UC-03	UC-04	UC-05	UC-06	UC-07	UC-08	UC-09	UC-10
Concert Round Service	createZoneMap()				1–2						
	updateZoneMap()				4; 5–6; AF-1; AF-3						
	uploadZoneMapImage()				3, EF-3						
	defineTableType()				5–6						
	listTableTypes()				5–6						
	listZoneMaps()			6	AF-1						
	getZoneMap()			7	7; 10; AF-1						
	validateZoneMap()				8–9, S-1, EF-1						
	activateZoneMap()				11–12, EF-2						
	getBusinessParameters()			3–5				✓			
	updateBusinessParameters()							✓			
	createRound()			1–2							
	updateRound()			3–5; 8; 9–10; AF-1; AF-3							
	validateRound()			11–12, S-1, EF-1							
	publishRound()			14–15, EF-2							
	getUpcomingRounds()	3, AF-2									
	getRound()	4, AF-1; 6–8, AF-3		13; AF-3							
	getRoundTables()	5, AF-2		13							
	getRoundPricing()	10–11									
	getCheckInWindow()	13	2–4, S-1, AF-2, AF-3, AF-5, EF-1, EF-2								
Table Availability Service	createRoundTableStatus()			14–15, EF-2							
	getRoundTableStatus()	5, AF-2	7	AF-3	AF-1	✓					
	countAvailableTables()	3, AF-2									
	holdTable()	6–8, AF-3									
	releaseHold()	AF-4, AF-5; EF-1									
	markTableBooked()	16–17									
	markTableOccupied()		6, EF-5								
Booking Service	createHeldBooking()	6–8, AF-3									
	getBooking()	9; 18									
	setPartySize()	10–11									

Service	Operation	UC-01	UC-02	UC-03	UC-04	UC-05	UC-06	UC-07	UC-08	UC-09	UC-10
	getCustomerProfile()									1–2; 6–7, AF-2, EF-1	
	createCustomerProfile()									3–5, AF-1, AF-2, EF-1	
	updateCustomerProfile()									6–7, AF-2, EF-1	
	getBookingTerms()	13									
	acceptBookingTerms()	14									
	startPayment()										1–2
	confirmBookingPayment()	16–17									5–6, AF-1
	getETicket()	18					✓				
	getCustomerBookings()						✓				
	cancelBooking()	AF-4, AF-5									
	verifyBookingReference()		2–4, S-1, AF-2, AF-3, AF-5, EF-1, EF-2								
	checkInBooking()		6, EF-5								
	getRoundBookings()		7			✓					
Payment Service	createPaymentRequest()										1–2; 2
	receivePaymentResult()										5–6, AF-1
	getPaymentStatus()										AF-1
Notification Service	sendBookingConfirmation()	19, S-1									
	sendHoldExpiredNotice()	EF-1; EF-1, S-1									
	sendPaymentFailedNotice()										5–6, AF-1; AF-1
Staff Account Service	signIn()								✓		
	signOut()								✓		
	createStaffAccount()								✓		
	listStaffAccounts()								✓		
	updateStaffAccount()								✓		
	disableStaffAccount()								✓		
Jobs on a timer (not operations)	hold-expiry job of the Booking Service	EF-1									EF-4
	retry job of the Notification Service	EF-3									

A.2 Step by Step

Tables A.2 to A.8 are the evidence behind Table A.1: each row follows one step, or the consecutive steps served by one operation, of a use case: who invokes the operation (an actor, through the API Gateway, or a service), the service and operation that carry the step out, the collaborations the operation needs, the data it stores, and the requirements it realises. Steps in which the actor acts without the system, such as UC-02 step 5, are left out unless a rule applies to them. Only the flows that the MVP builds are listed (Section 1.3), and Table A.8 covers UC-05 to UC-08, which have no description. Every operation of Table 5.3 appears in at least one row, the two jobs appear as jobs, and every MVP requirement of Section 3.1 is realised by at least one row.

Table A.2 Traceability of Reserve a Specific Table

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
1–2, EF-2	**Customer**	API Gateway: check the LINE ID token that LINE Login gave the LIFF app	LINE Login Adapter, which asks the **LINE Platform**	—	FR-01, FR-02
3, AF-2	**Customer**	Concert Round Service: getUpcomingRounds(), with the status of each round	Table Availability Service: countAvailableTables(), for the sold-out status	—	FR-03
4, AF-1	**Customer**	Concert Round Service: getRound()	—	—	FR-03, FR-04
5, AF-2	**Customer**	Concert Round Service: getRoundTables() Table Availability Service: getRoundTableStatus(), polled every 2 seconds	—	—	FR-05, FR-06
6–8, AF-3	**Customer**	Booking Service: createHeldBooking()	Concert Round Service: getRound(), to check that booking is open Table Availability Service: holdTable()	Booking DB: booking Held until the end of the **hold** (one active booking per table per round, ADR-13) Table Status DB: table held	FR-04, FR-07, FR-08
9	**Customer**	Booking Service: getBooking(), the booking summary and the remaining **hold** time	—	—	FR-07

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
10–11	**Customer**	Booking Service: setPartySize(), which computes the **full table fee**	Concert Round Service: getRoundPricing()	Booking DB: **party size** and **full table fee**	FR-09
12	**Customer**	Included use case UC-09 Maintain Customer Profile (Table A.6)	—	—	FR-10
13	**Customer**	Booking Service: getBookingTerms()	Concert Round Service: getCheckInWindow()	—	FR-12
14	**Customer**	Booking Service: acceptBookingTerms()	—	Booking DB: **booking terms** accepted	FR-12
15	**Customer**	Included use case UC-10 Pay the Full Table Fee (Table A.7)	—	—	FR-13
16–17	Payment Service, at UC-10 step 6	Booking Service: confirmBookingPayment(), which also issues the **e-ticket**	Table Availability Service: markTableBooked()	Booking DB: payment recorded, booking Confirmed, **e-ticket** with the signed **booking reference** Table Status DB: table booked	FR-16, FR-19
18	**Customer**	Booking Service: getBooking(), getETicket()	—	—	FR-19
19, S-1	Booking Service	Notification Service: sendBookingConfirmation()	LINE Messaging Adapter: pushLineMessage()	Notification DB: message and delivery result	FR-20
AF-4, AF-5	**Customer**	Booking Service: cancelBooking()	Table Availability Service: releaseHold()	Booking DB: booking Cancelled Table Status DB: table available	FR-10, FR-11
EF-1	Time	Booking Service: the hold-expiry job, every 5 seconds (ADR-08); not an operation	Table Availability Service: releaseHold() Notification Service: sendHoldExpiredNotice()	Booking DB: booking Expired Table Status DB: table available	FR-21, FR-23
EF-1, S-1	Booking Service	Notification Service: sendHoldExpiredNotice()	LINE Messaging Adapter: pushLineMessage()	Notification DB: message and delivery result	FR-21

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
EF-3	Time	Notification Service: the retry job, 3 times within 5 minutes; not an operation	LINE Messaging Adapter: pushLineMessage()	Notification DB: delivery result	FR-22

Table A.3 Traceability of Check In with E-Ticket

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
2–4, S-1, AF-2, AF-3, AF-5, EF-1, EF-2	**Front Staff**	Booking Service: verifyBookingReference(), with the **booking reference** scanned (step 2) or typed (AF-2); its answer is the result of step 4 or the reason of AF-3, AF-5, EF-1 and EF-2	Concert Round Service: getCheckInWindow()	— (verification changes nothing)	FR-24, FR-25, FR-26, FR-27
6, EF-5	**Front Staff**	Booking Service: checkInBooking(); a retry finds the existing check-in and does not repeat it (EF-5)	Table Availability Service: markTableOccupied()	Booking DB: booking Checked-in with the time and the staff account Table Status DB: table occupied	FR-28
7	**Manager**	Booking Service: getRoundBookings() Table Availability Service: getRoundTableStatus(), polled every 2 seconds	—	—	FR-06, FR-42
AF-4	**Front Staff**	No operation of SEATS: extra guests are handled by hand (BRULE-09)	—	—	—

Table A.4 Traceability of Create Concert Round

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
1–2	**Manager**	Concert Round Service: createRound()	—	Round DB: a new round, Draft	FR-34

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
3–5	**Manager**	Concert Round Service: updateRound(), with the concert details, the times and the **booking-open time**; it derives the **check-in window** from getBusinessParameters()	—	Round DB: schedule, **booking-open time**, **check-in window**	FR-34, FR-38
6	**Manager**	Concert Round Service: listZoneMaps(), the Active maps	—	—	FR-35
7	**Manager**	Concert Round Service: getZoneMap()	—	—	FR-35
8	**Manager**	Concert Round Service: updateRound(), with the **zone map** and the tables not for sale	—	Round DB: the round's **zone map** and tables for sale	FR-35
9–10	**Manager**	Concert Round Service: updateRound(), with the **package price** and content of each **table type** in each **zone**; it answers with the tables for sale and the capacity of each **zone**	—	Round DB: **package prices**	FR-37
11–12, S-1, EF-1	**Manager**	Concert Round Service: validateRound()	—	—	FR-75
13	**Manager**	Concert Round Service: getRound(), getRoundTables(), the round as the **Customer** will see it	—	—	—
14–15, EF-2	**Manager**	Concert Round Service: publishRound(); a retry finds the published round and does not create a second one (EF-2)	Table Availability Service: createRoundTableStatus()	Round DB: round Published Table Status DB: every table for sale available	FR-34, FR-75
AF-1	**Manager**	Concert Round Service: updateRound(); the round stays Draft	—	Round DB: the round as entered	FR-34

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
AF-3	**Manager**	Concert Round Service: getRound(), then updateRound(), which allows a Published round only the changes of AF-3 (BRULE-07)	Table Availability Service: getRoundTableStatus(), whose booked tables are the Confirmed bookings	Round DB: the changed fields	FR-34, FR-35

Table A.5 Traceability of Create Venue Zone Map

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
1–2	**Manager**	Concert Round Service: createZoneMap()	—	Round DB: a new **zone map**, Draft	FR-37
3, EF-3	**Manager**	Concert Round Service: uploadZoneMapImage()	Media Storage Adapter: storeZoneMapImage()	Cloud Object Storage: image of the venue Round DB: its address	FR-39
4	**Manager**	Concert Round Service: updateZoneMap(), with the **zones** drawn and named	—	Round DB: **zones**	FR-37
5–6	**Manager**	Concert Round Service: listTableTypes() to choose a **table type**; updateZoneMap(), with each table's position, number, **table type** and capacity; defineTableType() for a **table type** the venue does not have yet	—	Round DB: tables and **table types**	FR-37, FR-39
7	**Manager**	Concert Round Service: getZoneMap(), with the number of tables and the capacity of each **zone**	—	—	FR-37
8–9, S-1, EF-1	**Manager**	Concert Round Service: validateZoneMap()	—	—	FR-74
10	**Manager**	Concert Round Service: getZoneMap(), the preview	—	—	—
11–12, EF-2	**Manager**	Concert Round Service: activateZoneMap(); a retry finds the Active map and does not create a second one (EF-2)	—	Round DB: **zone map** Active	FR-74

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
AF-1	**Manager**	Concert Round Service: listZoneMaps(), getZoneMap(), then updateZoneMap(), which allows only the changes of AF-1	Table Availability Service: getRoundTableStatus(), for the Published rounds that use the map	Round DB: the changed **zones** and tables	FR-37
AF-3	**Manager**	Concert Round Service: updateZoneMap(); the map stays Draft	—	Round DB: the map as entered	FR-37

Table A.6 Traceability of Maintain Customer Profile

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
1–2	**Customer**	Booking Service: getCustomerProfile(), which tells whether a profile exists	—	—	FR-10
3–5, AF-1, AF-2, EF-1	**Customer**	Booking Service: createCustomerProfile(), with the consent, the name and the phone; AF-2 is its validation; AF-1 stores nothing	—	Booking DB: **customer profile** and consent	FR-10
6–7, AF-2, EF-1	**Customer**	Booking Service: getCustomerProfile(), then updateCustomerProfile() when the **Customer** corrects the profile	—	Booking DB: **customer profile**	FR-10

Table A.7 Traceability of Pay the Full Table Fee

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
1–2	**Customer**	Booking Service: startPayment()	Payment Service: createPaymentRequest()	Booking DB: payment started	FR-13
2	Booking Service	Payment Service: createPaymentRequest()	Payment Gateway Adapter: createCheckoutSession()	Payment DB: payment request	FR-13

Steps	Invoked by	Operation	Collaborations	Data stored	Requirements
3–4	**Customer**	No operation of SEATS: the **Customer** pays in the hosted checkout of the **Payment Gateway**, simulated in the MVP (ADR-11)	—	—	FR-13
5–6, AF-1	**Payment Gateway**	Payment Service: receivePaymentResult(), the signed webhook routed by the API Gateway; a duplicate result is ignored	Payment Gateway Adapter: verifyWebhookSignature() Booking Service: confirmBookingPayment(), when paid (UC-01 step 16) Notification Service: sendPaymentFailedNotice(), when declined	Payment DB: payment result, processed once	FR-14, FR-16, FR-21
AF-1	**Customer**	Payment Service: getPaymentStatus(), the decline and the remaining **hold** time	—	—	FR-14
AF-1	Payment Service	Notification Service: sendPaymentFailedNotice()	LINE Messaging Adapter: pushLineMessage()	Notification DB: message and delivery result	FR-21
EF-4	Time, through UC-01 EF-1	The hold-expiry job of the Booking Service; not an operation	—	—	FR-23

Table A.8 Traceability of UC-05 to UC-08, the use cases without a description

Use case	Invoked by	Operation	Collaborations	Data stored	Requirements
UC-07	**Manager**	Concert Round Service: getBusinessParameters(), updateBusinessParameters()	—	Round DB: the **business parameters**; a round uses the values in force at its **booking-open time**	FR-38
UC-06	**Customer**	Booking Service: getCustomerBookings(), getETicket()	—	—	FR-40

Use case	Invoked by	Operation	Collaborations	Data stored	Requirements
UC-05	**Manager**, **Owner**	Booking Service: getRoundBookings() Table Availability Service: getRoundTableStatus()	—	—	FR-42
UC-08	**Manager**	Staff Account Service: createStaffAccount(), listStaffAccounts(), updateStaffAccount() for the role, disableStaffAccount()	—	Staff Account DB: accounts, roles and password hashes	FR-65
UC-08	**Front Staff**, **Manager**, **Owner**	Staff Account Service: signIn(); the API Gateway checks the role of every request	—	—	FR-65, FR-66
UC-08	All users	Staff Account Service: signOut(); a **Customer** logs out of LINE Login in the web app	—	—	FR-73

Appendix B Business Rules

The business rules that the use cases (Section 2), the requirements (Section 3) and the domain model (Section 6.1) refer to. Each rule is a policy of the venue, not a design decision; the design realises it (for example BRULE-03 in ADR-08).

Table B.1 Business rules

ID	Name	Rule
BRULE-01	Full fee confirms	A booking is confirmed only once the **full table fee**, the **package price** plus any **extra-person fees**, has been received and verified; there is no deposit and no balance to pay at the venue. While a **transfer slip** awaits the **Manager**'s decision in **degraded mode**, the **hold** does not expire.
BRULE-02	15-minute hold	Selecting a table gives a **hold** of 15 minutes from the moment of selection. If no payment is confirmed within the **hold**, it expires, the table returns to available and the customer is notified; an expired hold costs the customer nothing.
BRULE-03	First hold wins	When several customers try to take the same table for the same round, the first successful hold wins; the later customer is told that the table was just taken and sees a refreshed map. A second hold on a held or confirmed table is never granted.
BRULE-04	Check-in window	The **check-in window** opens 2 hours before the concert start time; before that time an **e-ticket** cannot be checked in.
BRULE-05	Grace period	A customer who arrives late is still given the booked table up to 30 minutes after the concert start.
BRULE-06	No-show	When the **grace period** ends without a check-in, the booking is marked **No-show** and the table is shown as free; the fee is not refunded. The table is resold by hand to **walk-in** guests, who pay at the venue outside the system, and it is never offered to a **waitlist**.
BRULE-07	Booking-open time	The tables of a round can be selected only from the **booking-open time** set by the **Manager**; before it the round is visible but not bookable. The **zone map**, the table numbers and the prices are complete before booking opens and do not change during the round.
BRULE-08	Package pricing	The price of a table is the **package price** of its **table type** in its **zone**, for example 2,400 THB for a 2-person round table, 4,800 THB for a 4-person square table and 7,200 THB for a 6-person sofa in Zone A. The **Manager** maintains the prices and the **package** contents per round.
BRULE-09	Extra-person fee	Each person above the capacity of the **table type** costs 600 THB, added to the table fee before payment. Extra guests at the door are handled by hand, outside the system.
BRULE-11	Personal data	Name, phone, LINE user id and booking and payment history are collected once into a **customer profile**, for booking, payment, check-in and contact about the booking. The purpose is shown and consent obtained before the first booking; the customer can view and correct the profile; the data is deleted or anonymised after the retention period.

ID	Name	Rule
BRULE-12	Identity	One LINE account is one customer. LINE Login is the customer's identity; there is no separate registration, and bookings and **e-tickets** are always tied to a LINE account.
BRULE-16	Terms before paying	Before paying, the customer is shown and must accept the **booking terms** (full payment confirms the booking, the **check-in window**, the **grace period**, no refund for a **no-show**); the same terms are repeated in the confirmation message.
BRULE-17	Late payment	If a successful payment result arrives after the **hold** expired, the booking is confirmed anyway when the table is still available; if the table was taken meanwhile, the payment is refunded automatically through the gateway and the customer is told by LINE.

Appendix C Glossary

Table C.1 lists the business terms of this document: the words of the venue, its customers and its rules. In the text they are set in bold, and in the PDF each bold term links to its entry. Table C.2 lists the technology and project terms that the ADRs and the microservice design use; they are not set in bold in the text.

Table C.1 Business terms

Term	Definition
Back-office	The staff web application of the venue, used on a desktop or a phone by the Front Staff and the Manager; the Owner reads it without changing anything.
Booking	A Customer's reservation of one table for one concert round, the central record. Its states are Held, Confirmed, Checked-in, Expired, Cancelled and No-show; Transferred belongs to a later release.
Booking reference	The signed reference of a booking encoded in the QR code of its e-ticket, valid only for its concert round and table; it is printed under the code so that it can be typed when a scan fails.
Booking terms	The terms shown before payment and accepted by the Customer: full payment confirms the booking, the check-in window, the grace period, and no refund for a no-show (BRULE-16).
Booking-open time	The time, set by the Manager for each concert round, from which its tables can be selected; before it the round is visible but not bookable (BRULE-07).
Business parameters	The venue's settings that apply to every round opening for booking afterwards: hold period, grace period, check-in window and extra-person fee (FR-38).
Check-in window	The period in which an e-ticket can be checked in: from 2 hours before the concert start to the end of the grace period (BRULE-04, BRULE-05).
Concert round	One scheduled live performance on a given date and start time, featuring one artist, for which every table is sold in advance; it has a booking-open time and a fixed zone map.
Customer	A LINE user who reserves a table for a concert night, for a party of 1 to 6 people.
Customer profile	The Customer's name, phone and LINE user id, collected once with consent and pre-filled for later bookings (BRULE-11).
Degraded mode	The payment mode used while the payment gateway is unreachable: the Customer transfers the full table fee to the venue's bank account and attaches the transfer slip, and the Manager confirms or rejects it (Increment 2).
E-ticket	The electronic ticket with a QR code issued when a booking is confirmed; it carries the booking reference.
Escalation	The hand-over of an invalid or duplicate ticket from the Front Staff to the Manager, who accepts or refuses entry (Increment 2).
Extra-person fee	600 THB for each person above the capacity of the table type, added to the full table fee at purchase (BRULE-09).

Term	Definition
First lock wins	The rule that, of several customers selecting the same table, only the first successful hold keeps it; the others are told that the table was just taken (BRULE-03).
Front Staff	The venue staff who work the door on concert nights and check guests in with the back-office on a phone.
Full table fee	The whole price of the booked package plus any extra-person fees, paid in advance and in full; there is no deposit and nothing is paid at the venue (BRULE-01).
Grace period	The 30 minutes after the concert start during which a late Customer still gets the booked table (BRULE-05).
Hold	The temporary, exclusive lock a Customer obtains on a table by selecting it; it lasts the hold period and is released automatically when no payment is confirmed (BRULE-02).
Hold period	The 15 minutes a hold lasts from the selection of the table (BRULE-02).
LINE Official Account	The venue's account on LINE; Customers who are its friends open the web app from its Rich Menu and receive its messages.
LINE Platform	The external LINE services the system uses: LINE Login for the Customer's identity, LIFF and the Messaging API.
Live view	The back-office screen that shows the tables and bookings of the current concert round as they change; the Owner reads it read-only.
Manager	The venue manager who runs the back-office: zone maps, concert rounds and prices, the live view, and rulings on escalated tickets.
No-show	A confirmed booking whose Customer has not checked in by the end of the grace period; the fee is not refunded and the table is resold by hand to walk-in guests (BRULE-06).
Owner	The owner of the venue, who reads the back-office without changing anything.
Package	A table type as sold: its capacity, contents and package price; "package" and "table type" are used interchangeably (BRULE-08).
Package price	The price of a table type in a zone, maintained by the Manager for each round (BRULE-08).
Party size	The number of guests of a booking, entered by the Customer; it determines the extra-person fee.
Payment Gateway	The external payment service that takes the full table fee and reports the payment result by signed webhook: simulated in the MVP, Beam Checkout from Increment 2.
Table type	The type of a table, for example a 2-person round table, a 4-person square table or a 6-person sofa, which fixes its capacity and its package.
Transfer slip	The proof of a bank transfer attached by the Customer in degraded mode (Increment 2).
Waitlist	A queue of customers for a sold-out round, planned for a later release; a no-show table is never offered to it.
Walk-in	Guests without a booking; they can only take the tables of no-shows, seated and paid by hand at the venue (FR-72).
Zone	A pricing area of the venue, for example Zone A near the stage and Zone B behind it.

Term	Definition
Zone map	The plan of the venue with its zones, tables and table types, drawn on an uploaded image of the venue and shown to the Customer as the real-time map of a concert round; the "floor plan" of ADR-02 and ADR-05.

Table C.2 Technology and project terms

Term	Definition
Adapter	A module of a service, or of the API Gateway, that talks to one external system (LINE Login, LINE Messaging, the Payment Gateway, Media Storage), so that the business logic never depends on the provider's API (ports and adapters, Section 5).
API Gateway	The single public entry of the Backend and its only REST API: it authenticates the caller, checks its role and turns each request into a gRPC call to the service that owns the operation (ADR-07, ADR-12).
Bounded context	A part of the domain with its own model and vocabulary; each service owns one (Section 5.1).
gRPC	Remote procedure calls over HTTP/2 with Protocol Buffers messages; the only API of every service, called by the API Gateway and by the other services (ADR-12).
Hosted checkout	The payment page of the Payment Gateway, opened inside the web app, where the Customer chooses a payment method and pays; no card data passes through the system (FR-71).
ID token	The signed token that LINE Login gives the web app; the API Gateway verifies it and reads the LINE user id from it (FR-01, NFR-36).
Idempotent	Said of an operation that can be repeated without changing the result, so that a retry is safe: releaseHold(), publishRound() and confirmBookingPayment() are idempotent (ADR-08, NFR-22).
Increment	A planned build of the system: the MVP first, then Increment 2 (Section 1.3).
LIFF	LINE Front-end Framework: the customer web app runs as a LIFF app inside LINE's in-app browser (ADR-01).
LINE Login	LINE's sign-in (OpenID Connect); the Customer's identity is the LINE user id it returns (BRULE-12).
LINE Messaging API	The API through which the Notification Service pushes messages to the Customer (ADR-10).
MongoDB	The document database of every service (ADR-06); a zone map or a round is stored as one document.
MVP	The first increment: the smallest system that runs the four business use cases end to end (Section 1.3).
node-cron	The scheduler in the Booking Service that runs the hold-expiry job every 5 seconds (ADR-03, ADR-08).
Polling	The web app asks the API Gateway for the table status every 2 seconds (ADR-09); the answer carries a version, so an unchanged map costs one small response.
Protocol Buffers	The typed message format of gRPC; the .proto file of a service is the contract of its gRPC API.
REST	HTTP requests with JSON bodies; the way the web apps call the API Gateway and the Payment Gateway sends its webhook; the API Gateway is the only REST API of the system (ADR-12).

Term	Definition
Rich Menu	The menu of the LINE Official Account from which the Customer opens the web app.
Session token	The token that the Staff Account Service issues at sign-in and that the API Gateway checks on every back-office request (ADR-07).
Webhook	A call that an external system makes to the API Gateway when something happens; the Payment Gateway reports every payment result by a signed webhook (NFR-38).
WebSocket	A persistent connection over which the server pushes table status changes to the open maps; Increment 2 (ADR-09).

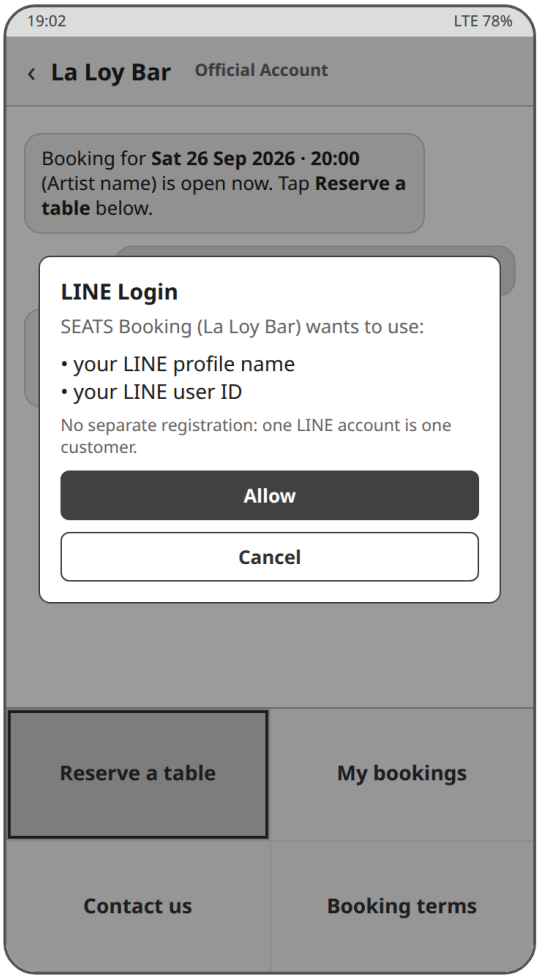
Appendix D Screens and the Routes They Call

The two web apps of Table 5.1 are, with the payment webhook, the only callers of the API Gateway. Tables D.1 to D.9 give the screens of the Customer Web App and Tables D.10 to D.16 those of the Back-office Web App, one page per screen: the wireframe, the use case steps and flows the screen serves, its main elements, the routes of Table 6.11 it calls, and an example of one call as the web app makes it, with the answer of the gateway. The wireframes are low fidelity: grayscale, English placeholder data, the phone screens at the 360 by 640 viewport of the LIFF app and the back-office screens in a desktop window. In the examples the identity headers of every call (`x-user-id` and `x-role` in progress 1) are left out, **…** stands for the fields not shown, and the gRPC method behind each route is in Table 6.11. A route is called when the screen opens or when the named element is used; a polled route is called again every 2 seconds while the screen is open (ADR-09). Section D.3 checks the mapping in the other direction.

D.1 Customer Web App

The Customer Web App is the LIFF app inside LINE (ADR-01). Its screens follow the basic flow of UC-01 in order; C9 stands alone.

Table D.1 Screen C1, Rich Menu and LINE Login

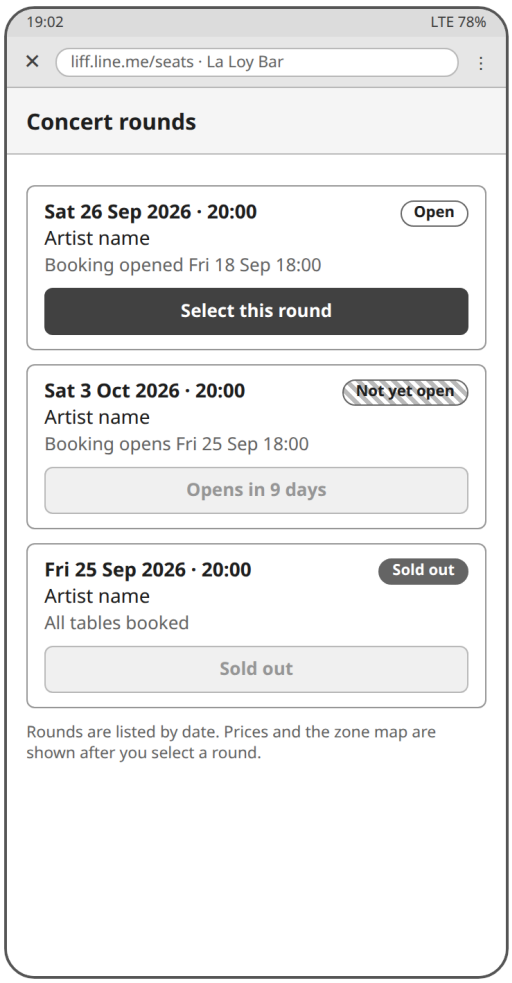


Use case steps	UC-01 steps 1–2, EF-2
Main elements	LINE chat, Rich Menu "Reserve a table", LINE Login dialog
Routes called	None: LINE Login runs in the LIFF app, and the ID token travels in the header of every later call (Table 5.2)

Example call

```
GET /api/rounds   progress 1; later Authorization: Bearer <LINE ID token> (ADR-01)
x-user-id: U-somchai
x-role: customer
→ 200
[…]
```

Table D.2 Screen C2, Concert rounds

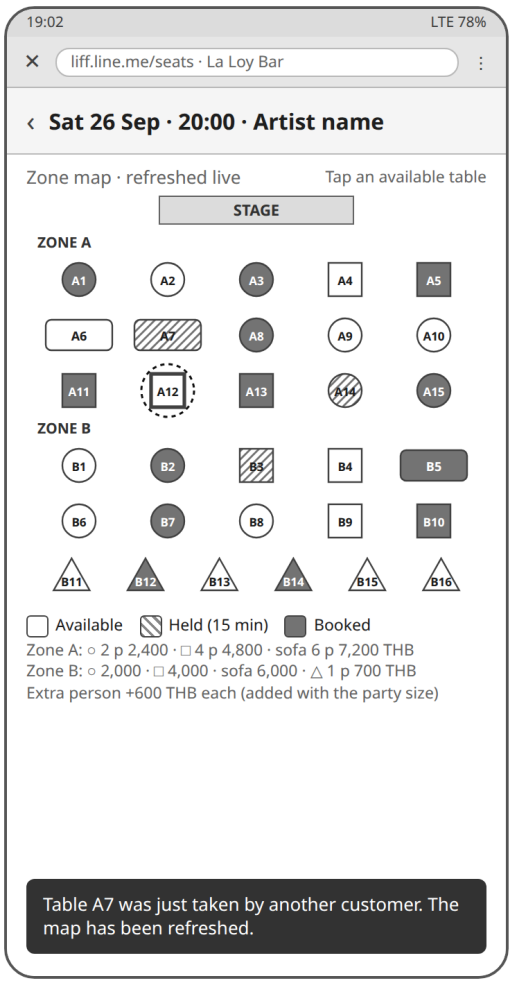


Use case steps	UC-01 steps 3–4, AF-1, AF-2
Main elements	Round list with artist, date, start time, **booking-open time** and status; a round not yet open shows its **booking-open time**, a sold-out round is marked
Routes called	`GET /rounds`

Example call

```
GET /api/rounds
→ 200
[
  {
    "id": "r-friday",
    "name": "Friday Live",
    "artist": "The Band",
    "date": "2026-10-09",
    "startAt": "2026-10-09T20:00:00Z",
    "bookingOpenAt": "2026-10-02T18:00:00Z",
    "status": "open",
    "availableTables": 28,
    "tablesForSale": 28
  }
]
```

Table D.3 Screen C3, Table map



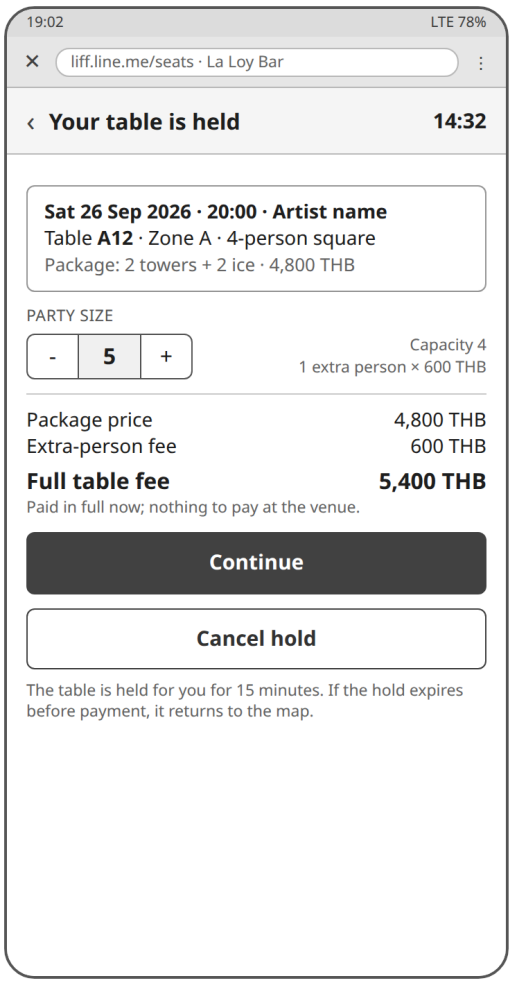
Use case steps	UC-01 steps 5–8, AF-3
Main elements	Stage, **zones**, a shape per table with number, **table type**, **package price** and status; a tap on an available table holds it
Routes called	`GET /rounds/{id}`, `GET /rounds/{id}/tables`; `GET /rounds/{id}/table-status` polled; `POST /bookings` on the tap

Example call

```
GET /api/rounds/r-friday/table-status
If-None-Match: 4
→ 304 unchanged since version 4
→ 200 ETag: 5
{
  "roundId": "r-friday",
  "version": 5,
  "tables": [
    {"tableNumber": 1, "status": "HELD", "bookingId": "b-1", "holdEndsAt": "…T19:15:00Z"},
    …
  ]
}

POST /api/bookings
{"roundId": "r-friday", "tableNumber": 2}
→ 200
{"id": "b-2", "status": "Held", "tableNumber": 2, "remainingHoldSeconds": 900, …}
→ 409 UC-01 AF-3
{"error": "the table has just been taken by another customer"}
```

Table D.4 Screen C4, Hold and booking summary



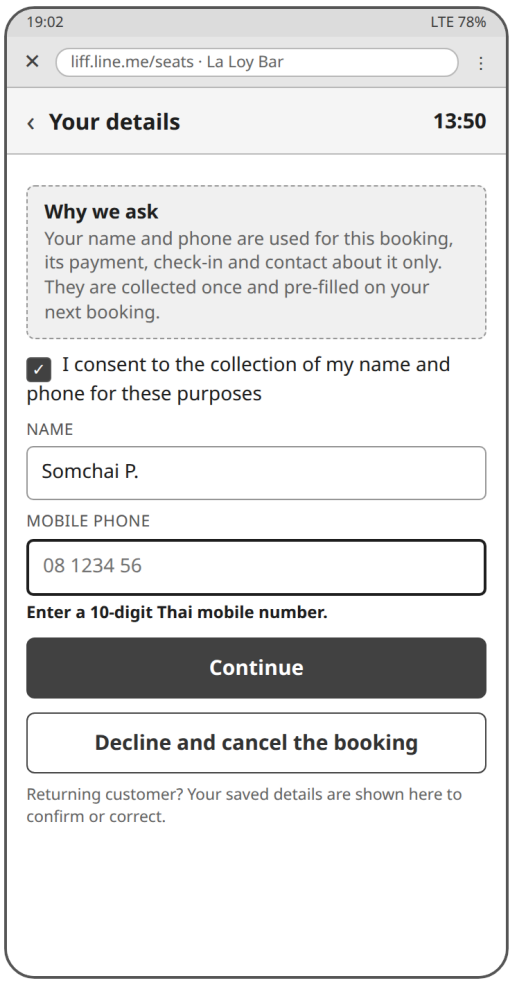
Use case steps	UC-01 steps 8–11, AF-4, EF-1
Main elements	**Hold** countdown, booking summary, **party size** stepper, fee breakdown with the **full table fee**, Cancel the hold; on expiry the message of EF-1
Routes called	`GET /bookings/{id}`, `PUT /bookings/{id}/party-size`, `POST /bookings/{id}/cancel`

Example call

```
PUT /api/bookings/b-2/party-size
{"partySize": 3}
→ 200
{
  "id": "b-2",
  "status": "Held",
  "partySize": 3,
  "fee": {"packagePrice": 2400, "extraPersons": 1, "extraPersonFee": 600, "fullTableFee": 3000},
  …
}

POST /api/bookings/b-2/cancel
→ 200
{
  "id": "b-2",
  "status": "Cancelled",
  "history": […, {"status": "Cancelled", "by": "U-somchai", …}],
  …
}
```

Table D.5 Screen C5, Customer profile and consent



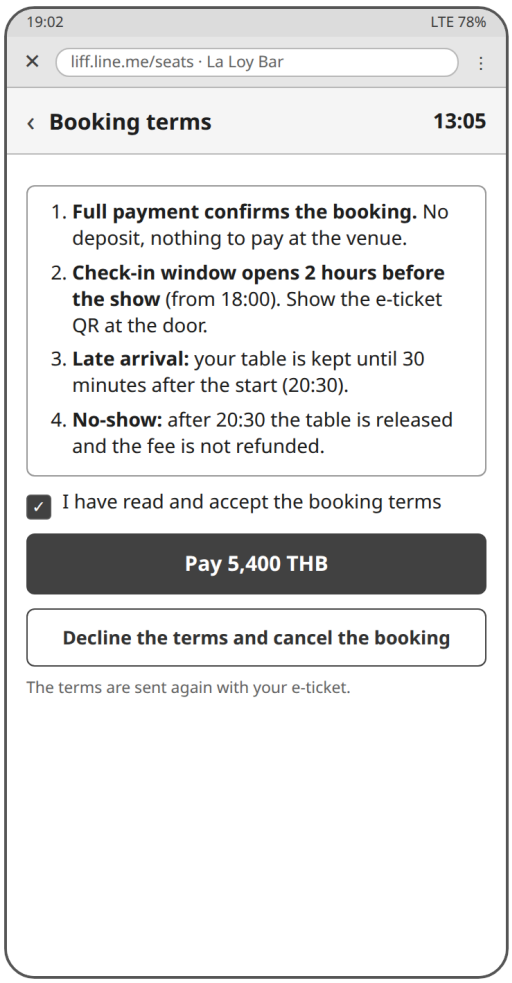
Use case steps	UC-01 step 12, AF-5; UC-09 steps 1–8, AF-1, AF-2
Main elements	On the first booking: the purpose of the data collection, consent, name and phone with validation, Decline; later: the stored name and phone to confirm or correct
Routes called	`GET /customers/me`, `POST /customers/me`, `PUT /customers/me`; `POST /bookings/{id}/cancel` on Decline

Example call

```
GET /api/customers/me
→ 404
{"error": "no profile yet: this is the first booking"}

POST /api/customers/me
{"name": "Somchai P.", "phone": "0812345678", "consent": true}
→ 200
{
  "customerId": "U-somchai",
  "name": "Somchai P.",
  "phone": "0812345678",
  "consentAt": "2026-10-03T12:00:00Z"
}
→ 400 UC-09 AF-2
{"error": "invalid profile", "details": ["phone must be a Thai mobile number"]}
```

Table D.6 Screen C6, Booking terms



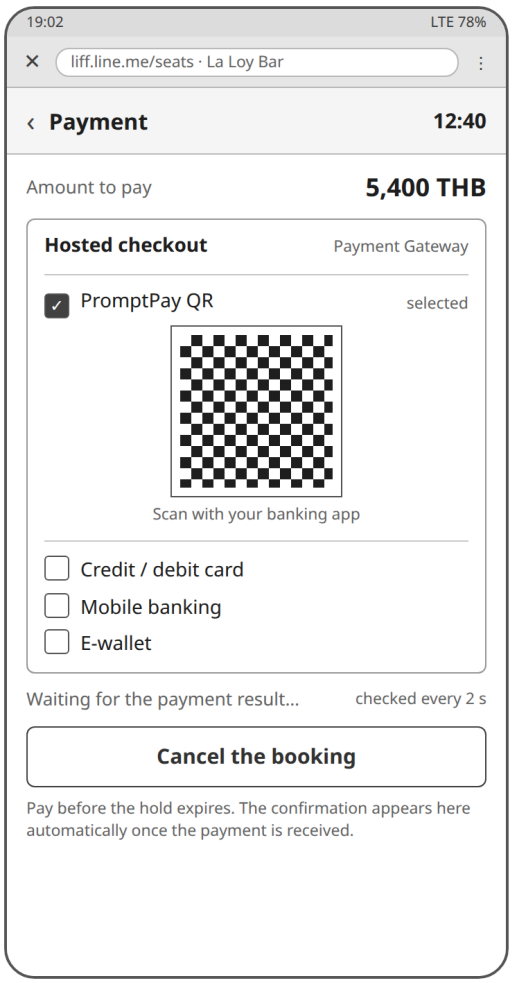
Use case steps	UC-01 steps 13–14
Main elements	The **booking terms** with the **check-in window**, Accept, Pay with the amount, Decline and cancel the booking
Routes called	`GET /bookings/{id}/terms`, `POST /bookings/{id}/terms-acceptance`; `POST /bookings/{id}/cancel` on Decline

Example call

```
GET /api/bookings/b-2/terms
→ 200
{
  "bookingId": "b-2",
  "terms": ["Full payment confirms the booking; …", …],
  "checkInWindow": {
    "opensAt": "2026-10-09T18:00:00Z",
    "startAt": "2026-10-09T20:00:00Z",
    "graceEndsAt": "2026-10-09T20:30:00Z"
  }
}

POST /api/bookings/b-2/terms-acceptance
→ 200
{"id": "b-2", "status": "Held", "termsAccepted": true, …}
```

Table D.7 Screen C7, Payment



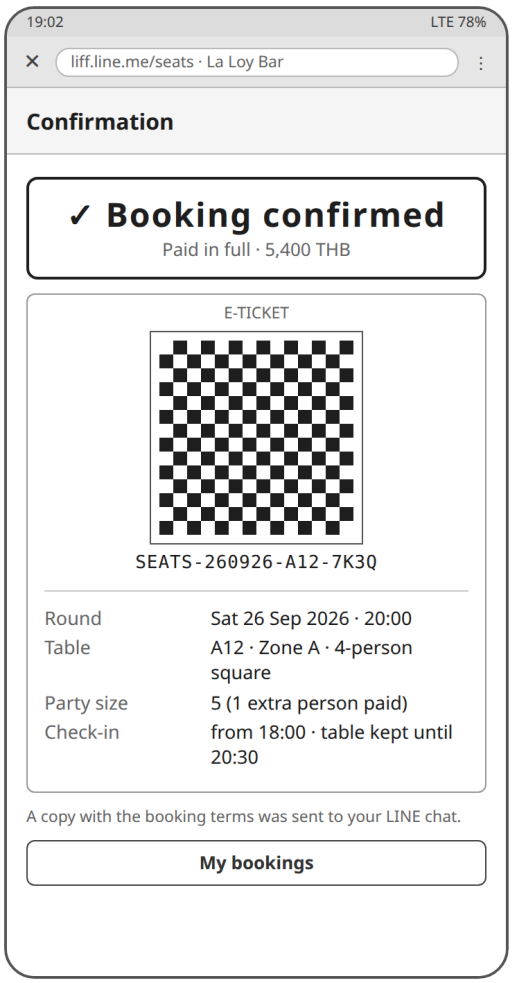
Use case steps	UC-01 step 15; UC-10 steps 1–7, AF-1, EF-4
Main elements	Amount, the hosted checkout of the **Payment Gateway** opened inside the web app, Cancel the booking; the payment status while the result is awaited
Routes called	`POST /bookings/{id}/payment` (501 in Increment 1), `GET /payments/{id}` polled until the result; `POST /bookings/{id}/cancel`

Example call

```
POST /api/bookings/b-2/payment
→ 200
{"paymentId": "p-7", "checkoutUrl": "https://checkout.example/pay/p-7"}
→ 501 Increment 1
{"error": "startPayment() is built in progress 2"}

GET /api/payments/p-7   polled until the result
→ 200
{"paymentId": "p-7", "bookingId": "b-2", "status": "Paid", "amount": 3000}
```

Table D.8 Screen C8, Confirmation and e-ticket



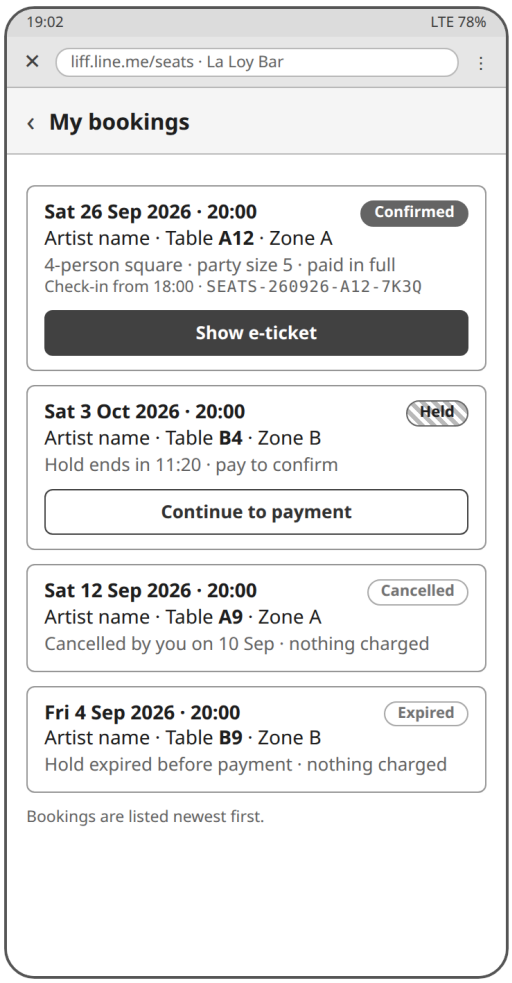
Use case steps	UC-01 steps 16–20, EF-3
Main elements	Confirmed banner, QR **e-ticket** with the **booking reference**, booking details, **check-in window**, note that the LINE copy was sent
Routes called	`GET /bookings/{id}`, `GET /bookings/{id}/e-ticket`

Example call

```
GET /api/bookings/b-2
→ 200
{
  "id": "b-2",
  "status": "Confirmed",
  "tableNumber": 2,
  "zoneName": "Zone A",
  "partySize": 3,
  "fee": {…, "fullTableFee": 3000},
  …
}

GET /api/bookings/b-2/e-ticket
→ 200
{"bookingId": "b-2", "bookingReference": "SEATS-261009-A02-7K3Q", "qrPayload": …}
```

Table D.9 Screen C9, My Bookings



Use case steps	UC-06
Main elements	The **Customer**'s bookings with status; a Confirmed booking opens its **e-ticket**
Routes called	`GET /customers/me/bookings`, `GET /bookings/{id}/e-ticket`

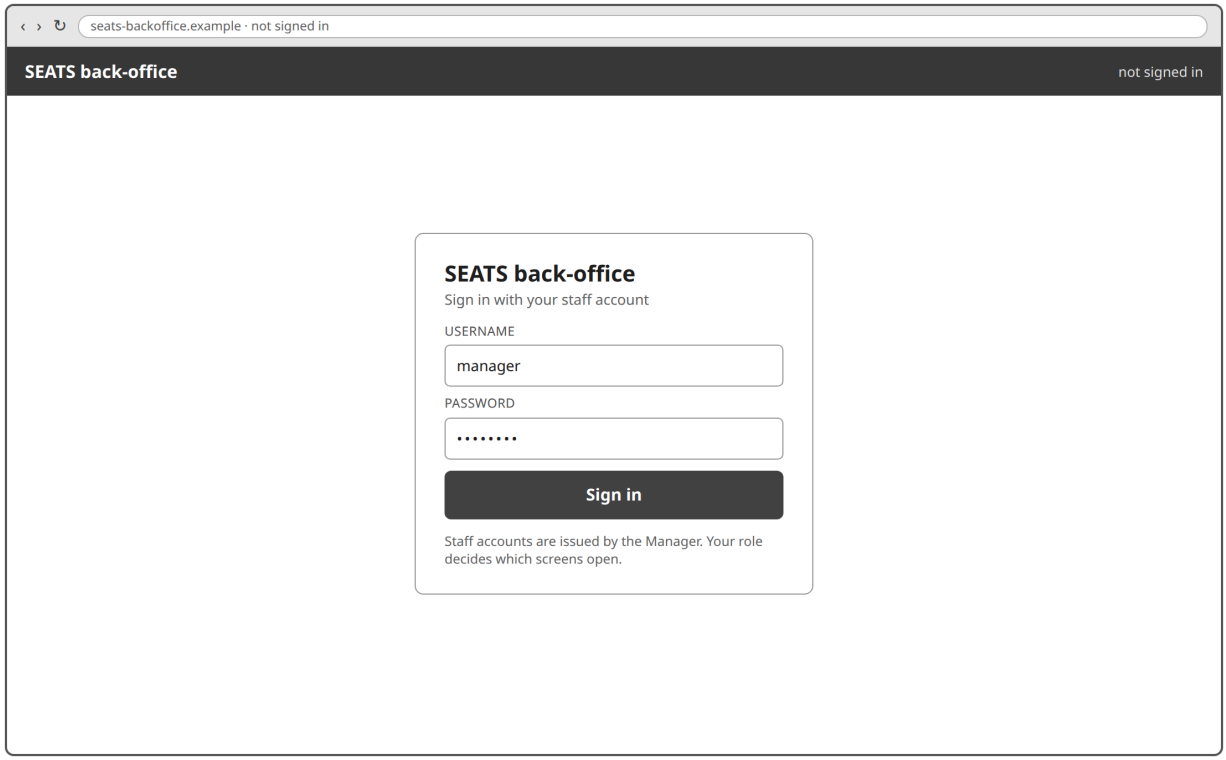
Example call

```
GET /api/customers/me/bookings
→ 200
[
  {"id": "b-2", "roundId": "r-friday", "tableNumber": 2, "status": "Confirmed", …},
  {"id": "b-1", "roundId": "r-sat", "tableNumber": 9, "status": "Expired", …}
]
```

D.2 Back-office Web App

The Back-office Web App runs in the browsers and phones of the **Manager**, the **Front Staff** and the **Owner**; every screen but B1 needs a signed-in staff account, and the role decides which screens open (FR-66). B5 and B6 run on the staff phone at the door; B6 is drawn in its two states.

Table D.10 Screen B1, Sign-in



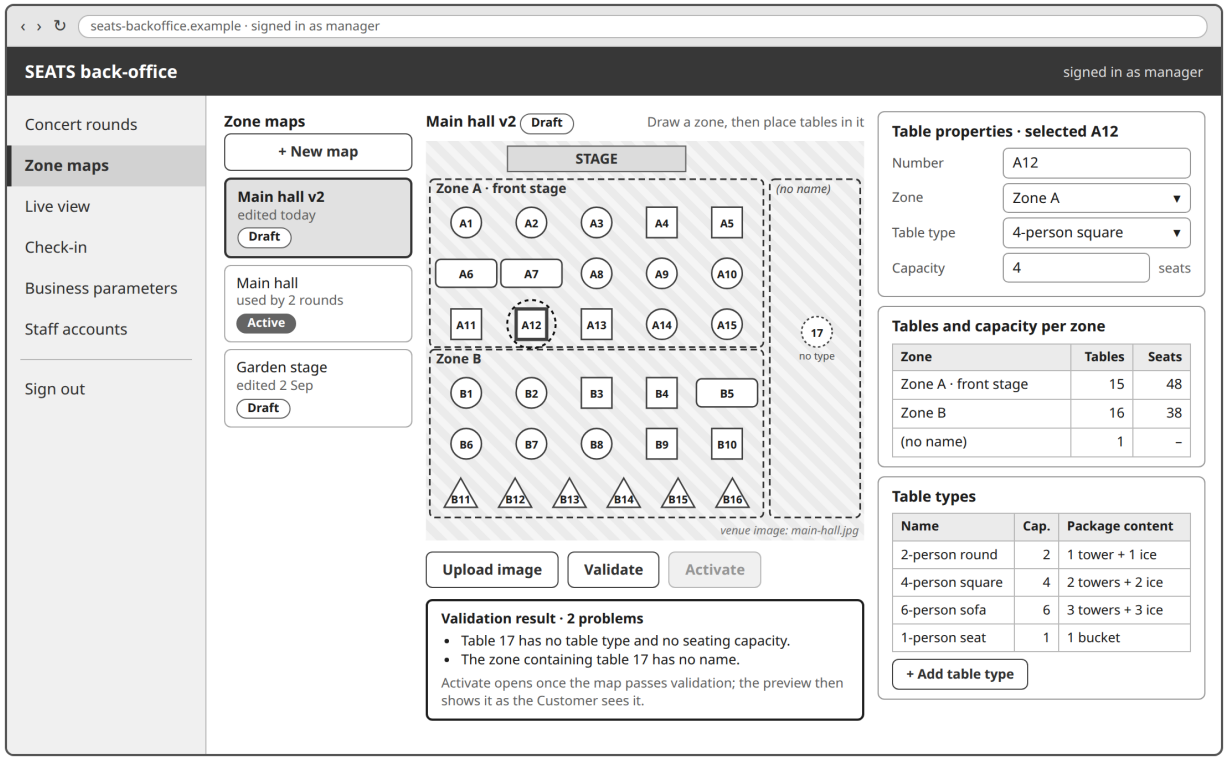
Use case steps	UC-08 (sign in and log out)
Main elements	Username, password; Sign out on every other screen
Routes called	`POST /sessions`, `DELETE /sessions/current`

Example call

```
POST /api/sessions
{"username": "manager", "password": …}
→ 200
{"token": "5e1f…", "role": "manager", "staffAccountId": "s-1"}
→ 401
{"error": "wrong username or password"}

DELETE /api/sessions/current
Authorization: Bearer 5e1f…
→ 200
```

Table D.11 Screen B2, Zone map editor



Use case steps	UC-04 steps 1–13, S-1, AF-1, AF-3, EF-1 to EF-3; **table types** (FR-37)
Main elements	Map list with status, new map, image upload, **zones** drawn on the image, tables placed with number, **table type** and capacity, tables and capacity per **zone**, validation result, preview, Activate; the **table types** with capacity and **package** content
Routes called	`GET /zone-maps`, `POST /zone-maps`, `GET /zone-maps/{id}`, `PUT /zone-maps/{id}`, `POST /zone-maps/{id}/image`, `POST /zone-maps/{id}/validate`, `POST /zone-maps/{id}/activate`, `DELETE /zone-maps/{id}`; `GET /table-types`, `PUT /table-types/{id}`

Example call

```
POST /api/zone-maps
{"name": "Main hall v2"}
→ 200
{
  "id": "m-2",
  "name": "Main hall v2",
  "status": "Draft",
  "imageUrl": "",
  "zones": [],
  "tables": [],
  "summary": [],
  …
}

PUT /api/zone-maps/m-2
{
  "zones": [{"id": "A", "name": "Zone A · front stage"}],
  "tables": [
    {
      "tableNumber": 1,
```

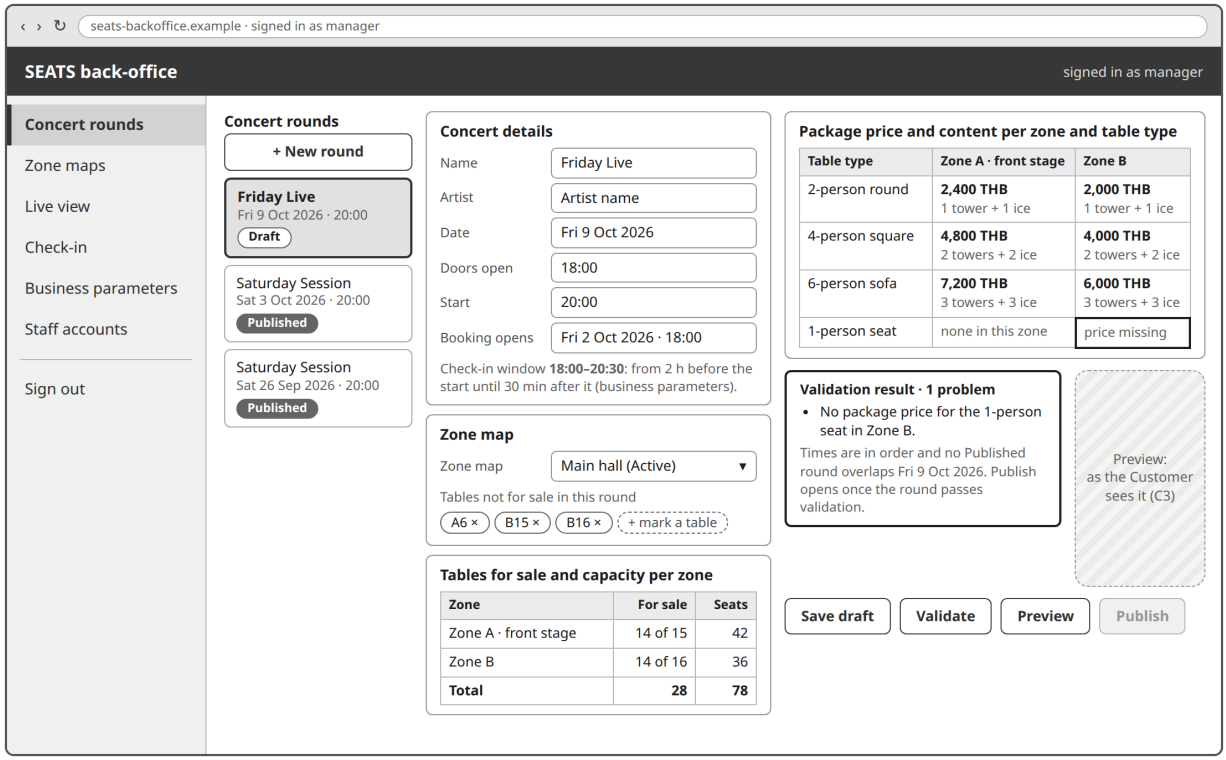
```
      "zoneId": "A",
      "tableTypeId": "round2",
      "capacity": 2,
      "x": 40,
      "y": 60
    }
  ]
}
→ 200
{
  "id": "m-2",
  "status": "Draft",
  "zones": [{"id": "A", "name": "Zone A · front stage"}],
  "tables": [
    {
      "tableNumber": 1,
      "zoneId": "A",
      "tableTypeId": "round2",
      "capacity": 2,
      "x": 40,
      "y": 60
    }
  ],
  "summary": [{"zoneId": "A", "name": "Zone A · front stage", "tables": 1, "capacity": 2}],
  …
}

POST /api/zone-maps/m-2/validate   UC-04 S-1
→ 200
{
  "valid": false,
  "problems": ["table 17 has no table type", "the zone containing table 17 has no name"]
}

POST /api/zone-maps/m-2/activate
→ 400
{"error": "the zone map is not valid", "details": ["table 17 has no table type", …]}
→ 200 once valid
{"id": "m-2", "status": "Active", …}

PUT /api/table-types/round2
{"name": "2-person round table", "capacity": 2, "packageContent": "1 tower + 1 ice"}
→ 200
{
  "id": "round2",
  "name": "2-person round table",
  "capacity": 2,
  "packageContent": "1 tower + 1 ice"
}
```

Table D.12 Screen B3, Round editor



Use case steps	UC-03 steps 1–16, S-1, AF-1, AF-3, EF-1, EF-2
Main elements	Round list with status, new round, concert details and times, **zone map** choice, tables not for sale, **package price** and content per **zone** and **table type**, tables for sale and capacity per **zone**, validation result, preview as the **Customer** sees it (the elements of C3), Publish
Routes called	`GET /rounds`, `POST /rounds`, `GET /rounds/{id}`, `PUT /rounds/{id}`, `GET /zone-maps?status=Active`, `GET /zone-maps/{id}`, `POST /rounds/{id}/validate`, `GET /rounds/{id}/tables` for the preview, `POST /rounds/{id}/publish`, `DELETE /rounds/{id}`

Example call

```
POST /api/rounds
{"name": "Friday Live"}
→ 200
{
  "id": "r-friday",
  "name": "Friday Live",
  "status": "Draft",
  "checkInWindow": null,
  "parameters": null,
  "tables": [],
  …
}

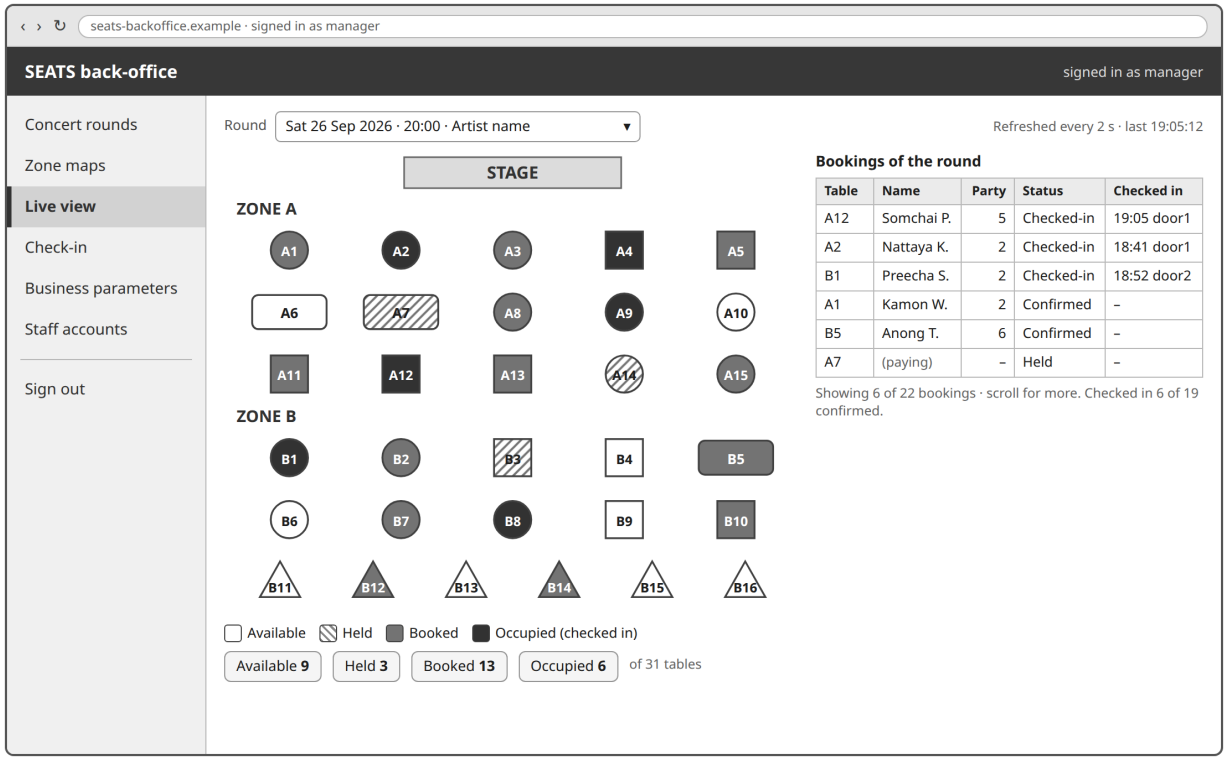
PUT /api/rounds/r-friday
{
  "artist": "The Band",
  "date": "2026-10-09",
  "doorsOpenAt": "2026-10-09T18:00:00Z",
  "startAt": "2026-10-09T20:00:00Z",
  "bookingOpenAt": "2026-10-02T18:00:00Z",
```

```
  "zoneMapId": "m-1",
  "tablesNotForSale": [6],
  "prices": [
    {
      "zoneId": "A",
      "tableTypeId": "sofa6",
      "packagePrice": 7200,
      "packageContent": "3 towers + 3 ice"
    },
    …
  ]
}
→ 200
{
  "id": "r-friday",
  "status": "Draft",
  "artist": "The Band",
  "date": "2026-10-09",
  "zoneMapId": "m-1",
  "tablesNotForSale": [6],
  "checkInWindow": {
    "opensAt": "2026-10-09T18:00:00Z",
    "startAt": "2026-10-09T20:00:00Z",
    "graceEndsAt": "2026-10-09T20:30:00Z"
  },
  "tables": [
    {
      "tableNumber": 1,
      "zoneId": "A",
      "tableTypeId": "sofa6",
      "capacity": 6,
      "forSale": true,
      "packagePrice": 7200,
      …
    },
    …
  ],
  …
}
→ 409 a Published round, UC-03 AF-3
{
  "error": "after the booking-open time these fields are fixed: zoneMapId",
  "details": {"confirmedBookings": 12}
}

POST /api/rounds/r-friday/validate   UC-03 S-1
→ 200
{"valid": false, "problems": ["no package price for table type seat1 in zone B"]}

POST /api/rounds/r-friday/publish   creates the table map of the round; idempotent
→ 200
{
  "id": "r-friday",
  "status": "Published",
  "parameters": {
    "holdPeriodMinutes": 15,
    "checkInWindowHours": 2,
    "gracePeriodMinutes": 30,
    "extraPersonFee": 600
  },
  …
}
→ 400
{"error": "the round is not valid", "details": […]}
```

Table D.13 Screen B4, Live view



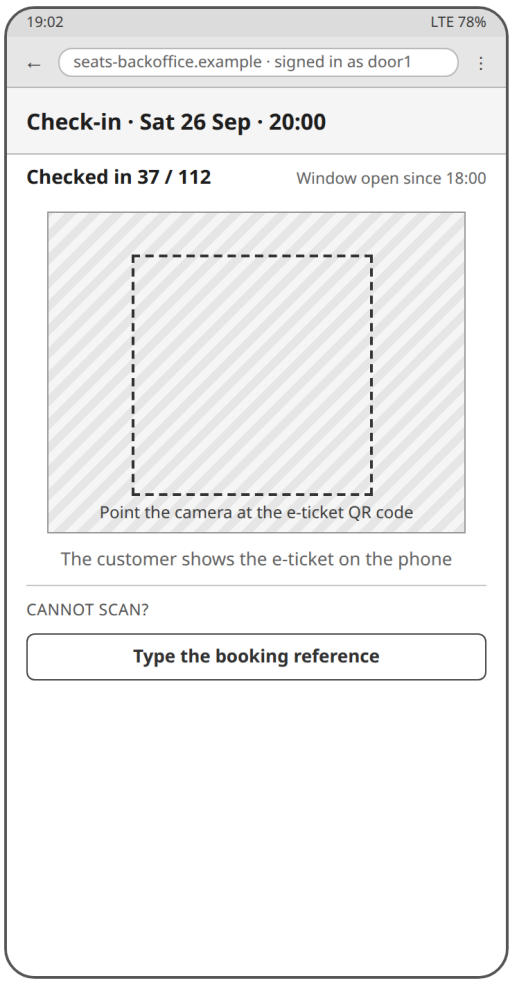
Use case steps	UC-05; UC-02 step 7
Main elements	Round choice, the table map with the status of every table, the bookings of the round with **party size** and check-in time, counts of available, held, booked and occupied tables
Routes called	`GET /rounds`, `GET /rounds/{id}/tables`, `GET /rounds/{id}/table-status` polled, `GET /rounds/{id}/bookings`

Example call

```
GET /api/rounds/r-friday/table-status   polled every 2 seconds
If-None-Match: 41
→ 304 unchanged since version 41; the 200 answer is that of C3

GET /api/rounds/r-friday/bookings
→ 200
[{"id": "b-2", "tableNumber": 1, "partySize": 3, "status": "Checked-in", …}, …]
```

Table D.14 Screen B5, Check-in scanner



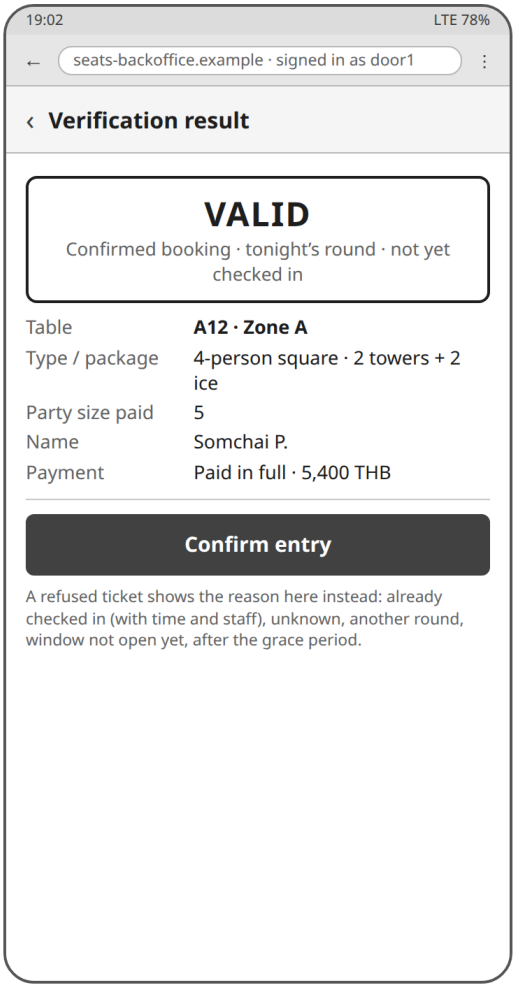
Use case steps	UC-02 steps 1–2, AF-2
Main elements	Round and count, camera viewfinder, type the **booking reference**
Routes called	`POST /check-ins/verify`

Example call

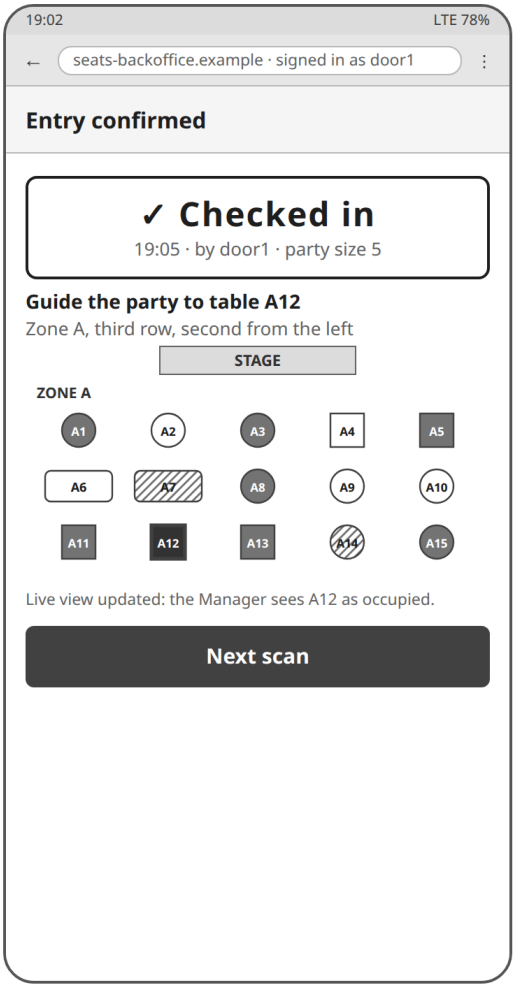
```
POST /api/check-ins/verify
{"bookingReference": "SEATS-261009-A02-7K3Q"}
→ 200
{
  "valid": true,
  "booking": {
    "id": "b-2",
    "tableNumber": 1,
    "zoneName": "Zone A",
    "partySize": 3,
    "status": "Confirmed",
    …
  },
  "reason": ""
}
→ 200 UC-02 EF-1
{"valid": false, "booking": null, "reason": "already checked in at 19:42 by door1"}
→ 501 progress 1
{"error": "verifyBookingReference() is built in progress 2"}
```

Table D.15 Screen B6, Verification result and entry confirmed

Verification result



Entry confirmed

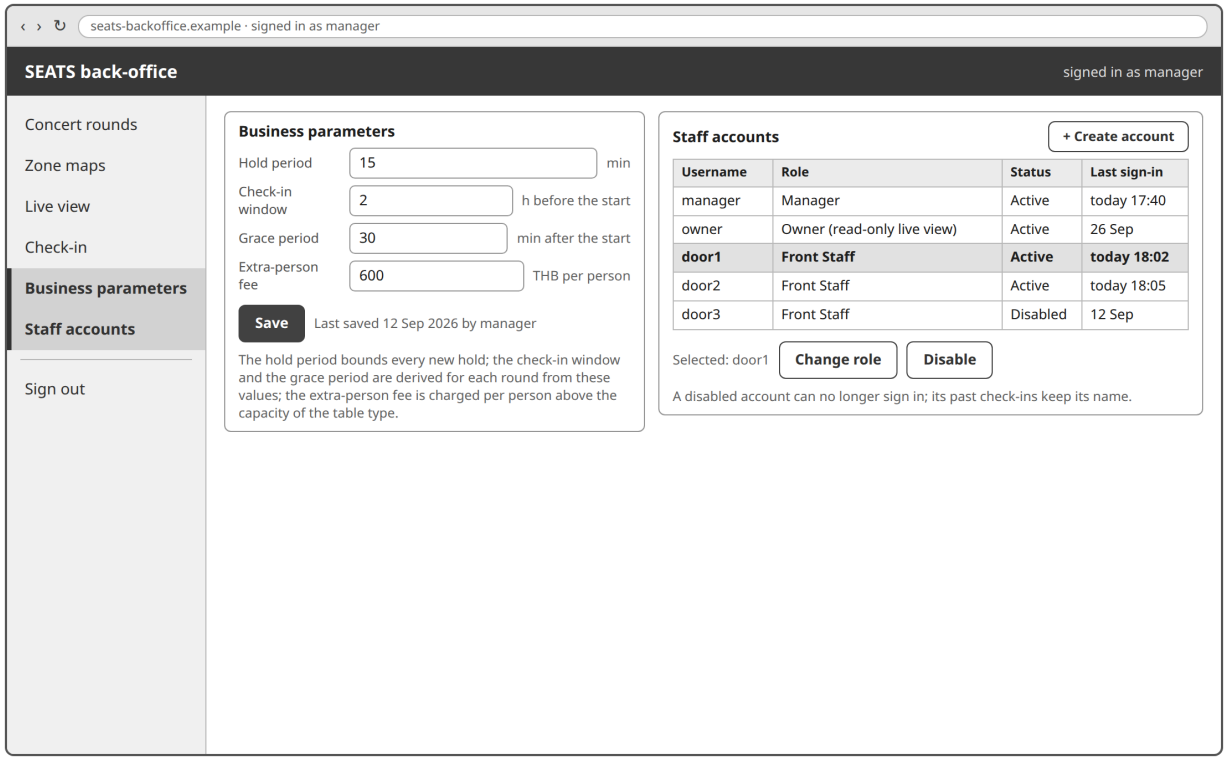


Use case steps	UC-02 steps 3–8, S-1, AF-3 to AF-5, EF-1, EF-2, EF-5
Main elements	Result panel with the reason of a failure, booking details with the **party size** paid for, Confirm entry; then the Checked-in banner with time and staff, Next scan
Routes called	`POST /check-ins`

Example call

```
POST /api/check-ins
{"bookingReference": "SEATS-261009-A02-7K3Q"}
→ 200
{
  "id": "b-2",
  "status": "Checked-in",
  "history": […, {"status": "Checked-in", "at": "2026-10-09T19:05:00Z", "by": "door1"}],
  …
}
→ 501 progress 1
{"error": "checkInBooking() is built in progress 2"}
```

Table D.16 Screen B7, Business parameters and staff accounts



Use case steps	UC-07; UC-08
Main elements	Hold period, check-in window, grace period, extra-person fee; the staff accounts with role, create, change the role, disable
Routes called	`GET /business-parameters`, `PUT /business-parameters`; `GET /staff-accounts`, `POST /staff-accounts`, `PUT /staff-accounts/{id}`, `DELETE /staff-accounts/{id}`

Example call

```
GET /api/business-parameters
→ 200
{
  "holdPeriodMinutes": 15,
  "checkInWindowHours": 2,
  "gracePeriodMinutes": 30,
  "extraPersonFee": 600
}

PUT /api/business-parameters
{"extraPersonFee": 700}
→ 200
{
  "holdPeriodMinutes": 15,
  "checkInWindowHours": 2,
  "gracePeriodMinutes": 30,
  "extraPersonFee": 700
}

GET /api/staff-accounts
→ 200
[
  {"staffAccountId": "s-1", "username": "manager", "role": "manager", "status": "Active"},
```

```
  {"staffAccountId": "s-2", "username": "door1", "role": "front_staff", "status": "Active"},
  …
]

POST /api/staff-accounts
{"username": "door2", "role": "front_staff", "password": …}
→ 200
{"staffAccountId": "s-4", "username": "door2", "role": "front_staff", "status": "Active"}
→ 409
{"error": "username door2 is taken"}

DELETE /api/staff-accounts/s-4
→ 200
{"staffAccountId": "s-4", "username": "door2", "role": "front_staff", "status": "Disabled"}
```

D.3 Coverage

Read the other way, every route of Table 6.11 is called by at least one screen of Tables D.1 to D.16, except the payment webhook, which the **Payment Gateway** calls. The screens also show one route that the back-office needs and Table 6.11 does not give: the round list of B3 must show the Draft rounds of the venue as well as the Published ones, while `GET /rounds` answers the **Customer**'s upcoming rounds only (getUpcomingRounds()). A listRounds() operation of the Concert Round Service, or a status query on the route, is to be decided before the back-office is built; until then B3 reopens a Draft round by its id.